

DOKUZ EYLÜL ÜNİVERSİTESİ

FEN BİLİMLERİ ENSTİTÜSÜ

METASEZGİSEL TABANLI YENİ BİR
OPTİMİZASYON YÖNTEMİ: YAPAY KAN
DOLAŞIM SİSTEMİ ALGORİTMASI

Nermin ÖZCAN

Aralık, 2024

İZMİR

METASEZGİSEL TABANLI YENİ BİR OPTİMİZASYON YÖNTEMİ: YAPAY KAN DOLAŞIM SİSTEMİ ALGORİTMASI

Dokuz Eylül Üniversitesi Fen Bilimleri Enstitüsü

Doktora Tezi

Biyomedikal Teknolojiler Anabilim Dalı, İleri Biyomedikal Teknolojiler

Sanayi Ortak Bütünleşik Doktora Programı

Nermin ÖZCAN

Aralık, 2024

İZMİR

DOKTORA TEZİ SINAV SONUÇ FORMU

NERMİN ÖZCAN tarafından **DOÇ.DR. SEMİH UTKU** yönetiminde hazırlanan
**“METASEZGİSEL TABANLI YENİ BİR OPTİMİZASYON YÖNTEMİ:
YAPAY KAN DOLAŞIM SİSTEMİ ALGORİTMASI”** başlıklı tez tarafımızdan
okunmuş, kapsamı ve niteliği açısından bir Doktora tezi olarak kabul edilmiştir.

Doç.Dr. Semih UTKU

Danışman

Prof.Dr. Recep Alp KUT

Tez İzleme Komitesi Üyesi

Prof.Dr. Efendi NASİBOĞLU

Tez İzleme Komitesi Üyesi

Prof. Dr. Mehmet Nurullah ORMAN

Jüri Üyesi

Doç. Dr. Özgü CAN

Jüri Üyesi

Prof. Dr. Abdullah SEÇGİN

Müdür

Fen Bilimleri Enstitüsü

TEŞEKKÜR

Kuşkusuz bu doktora tez çalışmamın ortaya çıkmasındaki en büyük katkı danışmanım Doç. Dr. Semih Utku'ya aittir. Fikirlerime olan inancı, değerli incelemeleri, akademik yönlendirmeleri ve bilimsel katkıları için kendisine minnettarım. Çalışmamın geliştirilmesi aşamasında her türlü bilgi ve değerli görüşlerini benimle paylaştığı için Prof. Dr. Recep Alp Kut ve Prof. Dr. Efendi Nasiboğlu hocalarıma en içten teşekkürlerimi sunarım. Ayrıca Yüksek Öğretim Kurumu tarafından sağlanan YÖK 100/2000 Doktora Bursu için Yüksek Öğretim Kurumu'na teşekkürler ederim. Son olarak, eğitim sürecimde bana her türlü fedakârlığı sağlayan sevgili aileme ve desteklerini esirgemeyen arkadaşlarıma teşekkürü bir borç bilirim.

Nermin ÖZCAN

METASEZGİSEL TABANLI YENİ BİR OPTİMİZASYON YÖNTEMİ: YAPAY KAN DOLAŞIM SİSTEMİ ALGORİTMASI

ÖZ

Metasezgisel (MS) algoritmalar, mühendislik uygulamaları da dahil olmak üzere çeşitli alanlarda yaygın olarak kullanılmaktadır. Bu yöntemlerin etkili sonuçlar elde ettiği açıktır, ancak stokastik yapıları gereği her problem için evrensel bir çözüm sunamadıkları bilinmektedir. Bu durum, araştırmacıları alternatif teknikler keşfetmeye, mevcut yöntemleri geliştirmeye yönlendiren önemli bir motivasyon kaynağıdır. Bu tez çalışması, Yapay Kan Dolaşım Sistemi (YKDS) algoritması adlı yeni bir MS optimizasyon yöntemi tanıtmaktadır. YKDS algoritması, dolaşım sisteminin kontrolünden esinlenmekte, düzenleme sürecinde görev alan sinirsel ve hormonal düzenleyicilerin davranışlarını taklit etmektedir. Çalışma önerilen yaklaşımın etkinliğini, iki boyutlu ve on boyutlu olmak üzere farklı özelliklere sahip iki setten oluşan toplam 28 adet test fonksiyonu üzerinde değerlendirmektedir. Daha sonra yöntem çeşitli zorluk ve kısıtlamalara sahip 6 mühendislik tasarım problemine uygulanarak incelenmiştir. Ayrıca çalışmada YKDS algoritmasının performansı farklı yaklaşımlardan türetilen 64 MS yöntemle istatistiksel olarak karşılaştırılmıştır. Mevcut literatürde en yaygın kullanılan yöntemler mercek altına alınmış ve önerilen algoritmanın 11 algoritmaya kıyasla üstünlüğünü incelemek için bir dizi karşılaştırma yapılmıştır. Sonuçlar, YKDS stratejisinin küresel optimuma hızlı bir şekilde ulaşabildiğini, yerel optimumlarda sıkışıp kalmaktan kaçındığını ve keşif ile sömürü arasındaki dengeyi etkin bir şekilde koruduğunu göstermektedir. Bunun yanı sıra, YKDS algoritması post-hoc testlerine göre 42 algoritmadan istatistiksel olarak daha iyi performans göstermiş, 9 algoritmadan da niceliksel olarak daha iyi sonuçlar elde etmiştir. Ayrıca bulgular, önerilen yöntemin popüler yöntemlere rekabetçi çözümler sunduğunu göstermiştir. Sonuç olarak önerilen yöntemin performansı göz önünde bulundurulduğunda, YKDS algoritması çeşitli alanlarındaki optimizasyon problemlerini ele almak için önemli bir alternatif olarak hizmet edebilir.

Anahtar kelimeler: Optimizasyon, meta-sezgisel, stokastik, evrimsel algoritma, sürü zekası

A NOVEL METAHEURISTIC-BASED OPTIMIZATION METHOD: ARTIFICIAL BLOOD CIRCULATION SYSTEM ALGORITHM

ABSTRACT

Metaheuristic (MH) algorithms have gained significant attention in various fields, particularly engineering applications. Although these methodologies are effective in solving complex problems, their stochastic nature limits their ability to provide universally applicable solutions. This limitation motivates researchers to explore alternative techniques and improve existing methods. This thesis introduces a novel meta-heuristic optimization technique called the Artificial Blood Circulatory System (ABCS) algorithm. The ABCS algorithm is inspired by the regulatory mechanisms of the circulatory system, emulating the roles of neural and hormonal regulators. The efficacy of the proposed method is evaluated through its application to 28 benchmark functions, categorized into two sets: two-dimensional and ten-dimensional. Additionally, the algorithm is tested on six engineering design problems of varying complexity and constraints. This study also conducts a statistical comparison of the ABCS algorithm against 64 MH methods from diverse approaches, examining prevalent methods in the literature and performing comparative analyses with 11 established algorithms. Results indicate that the ABCS strategy can rapidly converge to the global optimum, effectively avoid local optima, and maintain a balance between exploration and exploitation. Post-hoc analyses show that the ABCS algorithm significantly outperformed 42 competing algorithms and quantitatively surpassed 9 others. The findings demonstrate that the proposed method provides competitive solutions compared to established techniques, making the ABCS algorithm a viable alternative for addressing optimization challenges across various domains.

Keywords: Optimization, meta-heuristics, stochastic, evolutionary algorithms, swarm intelligence

İÇİNDEKİLER

	Sayfa
DOKTORA TEZİ SINAV SONUÇ FORMU	ii
TEŞEKKÜR	iii
ÖZ	iv
ABSTRACT	v
İÇİNDEKİLER	vi
ŞEKİLLER LİSTESİ	ix
TABLolar LİSTESİ	x
KISALTMALAR	xii
BÖLÜM BİR - GİRİŞ	1
BÖLÜM İKİ - METASEZGİSEL (MS) YÖNTEMLER.....	8
2.1 MS Algoritmaların Gelişimi	8
2.1.1 Tarihsel Bakış	8
2.1.2 NFL Teoremi	9
2.1.3 Yeni Nesil Algoritmalar.....	10
2.2 MS Algoritmaların Sınıflandırılması	11
2.2.1 İlham Kaynağına Dayalı Sınıflandırma	12
2.2.2 Popülasyon Boyutuna Dayalı Sınıflandırma.....	14
2.2.3 Popülasyon Hareketine Dayalı Sınıflandırma.....	15
2.2.4 Parametre Sayısına Dayalı Sınıflandırma	16
2.3 MS Algoritmaların Yapısı	17
BÖLÜM ÜÇ - YAPAY KAN DOLAŞIM SİSTEMİ ALGORİTMASI.....	19
3.1 İlham Kaynağı	19
3.2 Düzenleyicilerin Modellemesi.....	21
3.3 Model Parametreleri ve Uyarım Fonksiyonu	22
3.4 Keşif ve Sömürü Aşaması	24

BÖLÜM DÖRT - DENEYSEL TASARIM..... 28

4.1 DeneySEL Prosedür	28
4.2 Test Fonksiyonları	29
4.2.1 Klasik Test Fonksiyonları	29
4.2.2 Bileşik Test Fonksiyonları	31
4.3 Mühendislik Uygulamaları	32
4.3.1 P1: Germe-Sıkıştırma Yayı Tasarımı.....	33
4.3.2 P2: Basıncılı Kap Tasarımı	33
4.3.3 P3: Kaynaklı Kiriş Tasarımı	34
4.3.4 P4: Hız Düşürücü Tasarımı.....	35
4.3.5 P5: Çoklu Disk Debriyaj Freni Tasarımı	36
4.3.6 P6: Üç Çubuklu Kafes Tasarımı	36
4.4 Değerlendirme Kriterleri	37
4.4.1 Arama Geçmişi	38
4.4.2 Yakınsama Eğrileri	38
4.4.3 Keşif-Sömürü Grafikleri	39
4.4.4 Çözüm Kalitesi.....	39
4.4.5 Sağlamlık	39
4.4.6 Hesaplama Süresi.....	40
4.4.7 Sıralama Testi	40
4.4.8 İşaret Testi.....	40
4.4.9 Diğer İstatistiksel Testler	41

BÖLÜM BEŞ - BULGULAR 42

5.1 Niteliksel Sonuçlar.....	42
5.2 Niceliksel Sonuçlar	44

BÖLÜM ALTI - MEVCUT YÖNTEMLER İLE KARŞILAŞTIRMA..... 47

6.1 Çoklu-İstatistiksel Karşılaştırma	47
6.2 Popüler Algoritmalar ile Karşılaştırma.....	51
6.2.1 Yakınsama Eğrileri	52

6.2.2 Sıralama Testi Sonuçları	55
6.2.3 İşaret Testi Sonuçları	58
6.3 Hesaplama Süresi.....	59
BÖLÜM YEDİ - SONUÇLAR VE TARTIŞMA	61
BÖLÜM SEKİZ - KISIT, ÖNERİ VE GELECEK ÇALIŞMALAR	64
KAYNAKLAR.....	66
EKLER.....	77
Ek 1: Klasik Test Fonksiyonları Matematiksel İfadeleri.....	77
Ek 2: Bileşik Test Fonksiyonları Matematiksel İfadeleri	78
Ek 3: Germe-Sıkıştırma Yayısı Tasarımının Matematiksel İfadeleri.....	79
Ek 4. Basıncılı Kap Tasarımının Matematiksel İfadeleri	80
Ek 5: Kaynaklı Kiriş Tasarımının Matematiksel İfadeleri	81
Ek 6. Hız Düşürücü Tasarımının Matematiksel İfadeleri.....	82
Ek 7. Çoklu Disk Debriyaj Freni Tasarımının Matematiksel İfadeleri	83
Ek 8. Üç Çubuklu Kafes Tasarımının Matematiksel İfadeleri	84

ŞEKİLLER LİSTESİ

Sayfa

Şekil 1.1 Temel bileşenlerine göre optimizasyon problemleri.....	3
Şekil 2.1 MS algoritmaların sınıflandırılması.....	12
Şekil 3.1 Dolaşım sisteminin düzenlenmesi.....	20
Şekil 3.2 Bir arama uzayında düzenleyici davranışlarının gösterimi.....	21
Şekil 3.3 YKDS algoritmasına ait akış şeması	27
Şekil 4.1 Bazı klasik test fonksiyonlarının grafiksel gösterimleri	30
Şekil 4.2 Bazı bileşik test fonksiyonlarının grafiksel gösterimleri	31
Şekil 4.3 Germe-sıkıştırma yayı tasarımının grafiksel gösterimi.....	33
Şekil 4.4 Basınçlı kap tasarımının grafiksel gösterimi	34
Şekil 4.5 Kaynaklı giriş tasarımının grafiksel gösterimi.....	34
Şekil 4.6 Hız düşürücü tasarımının grafiksel gösterimi	35
Şekil 4.7 Çoklu disk debriyaj freni tasarımının grafiksel gösterimi	36
Şekil 4.8 Üç çubuklu kafes tasarımının grafiksel gösterimi	37
Şekil 5.1 Önerilen modele ait grafiksel sonuçlar	43
Şekil 6.1 Klasik test fonksiyonlarına ait yakınsama grafikleri	53
Şekil 6.2 Bileşik test fonksiyonlarına ait yakınsama grafikleri.....	54
Şekil 6.3 Mühendislik tasarım problemlerine ait yakınsama grafikleri	54
Şekil 6.4 En hızlı ve en yavaş yöntemlerin hesaplama süreleri	57
Şekil 6.5 En hızlı ve en yavaş yöntemlerin hesaplama süreleri	60
Şekil 6.6 En popüler yöntemlerin hesaplama süreleri.....	60

TABLÖLAR LİSTESİ

Sayfa

Tablo 3.1 YKDS algoritmasının parametreleri	24
Tablo 3.2 YKDS algoritmasının sözde kodu	26
Tablo 4.1 Deneysel arka planın detayları.....	28
Tablo 4.2 Klasik test fonksiyonlarının genel özellikleri	30
Tablo 4.3 Bileşik test fonksiyonlarının genel özellikleri	32
Tablo 5.1 YKDS algoritmasının klasik test fonksiyonu sonuçlarının özeti.....	45
Tablo 5.2 YKDS algoritmasının bileşik test fonksiyonu sonuçlarının özeti.....	45
Tablo 5.3 YKDS algoritmasının mühendislik uygulama sonuçlarının özeti	46
Tablo 6.1 Karşılaştırılan biyolojik canlı tabanlı algoritmaların listesi.....	47
Tablo 6.2 Karşılaştırılan evrim temelli yöntemlerin listesi.....	47
Tablo 6.3 Karşılaştırılan insan kökenli algoritmaların listesi	48
Tablo 6.4 Karşılaştırılan fiziksel yasalara dayanan yöntemlerin listesi	48
Tablo 6.5 Karşılaştırılan sürü zekâsına sahip yöntemlerin listesi	48
Tablo 6.6 Karşılaştırılan diğer yöntemlerin listesi.....	49
Tablo 6.7 Optimal sonuçları istatistiksel olarak daha kötü olan modeller.....	49
Tablo 6.8 Optimal sonuçları kötü olan modeller	50
Tablo 6.9 Optimal sonuçları istatistiksel olarak daha iyi olan modeller.....	51
Tablo 6.10 Optimal sonuçları iyi olan modeller	51
Tablo 6.11 Popüler algoritmalara ait detaylar.....	51
Tablo 6.12 Klasik test fonksiyonları için sıralama sonuçları.....	55

Tablo 6.13 Bileşik test fonksiyonları için sıralama sonuçları	56
Tablo 6.14 Mühendislik tasarım problemleri için sıralama sonuçları.....	56
Tablo 6.15 F18 fonksiyonu için üç yöntemin ulaştığı beş minimum sonuç	58
Tablo 6.16 İşaret testi sonuçları	59



KISALTMALAR

MS	: Meta-sezgisel
YKDS	: Yapay kan dolařım sistemi
NFL	: No Free Lunch
CEC	: Congress on Evolutionary Computation
BBO	: Biogeography-based Optimization
BMO	: Barnacles Mating Optimizer
EOA	: Earthworm Optimisation Algorithm
IWO	: Invasive Weed Optimization
SMA	: Slime Mould Algorithm
SOA	: Seagull Optimization Algorithm
SOS	: Symbiotic Organisms Search
WHO	: Wildebeest Herd Optimization
CRO	: Coral Reefs Optimization
DE	: Differential Evolution
ES	: Evolution Strategies
FPA	: Flower Pollination Algorithm
GA	: Genetic Algorithm
MA	: Memetic Algorithm
BSO	: Brain Storm Optimization
CA	: Culture Algorithm

CHIO	: Coronavirus Herd Immunity Optimization
FBIO	: Forensic-Based Investigation Optimization
LCO	: Life Choice-based Optimization
SARO	: Search And Rescue Optimization
SPBO	: Student Psychology Based Optimization
SSDO	: Social Ski-Driver Optimization
TLO	: Teaching Learning-based Optimization
ASO	: Atom Search Optimization
CDO	: Chernobyl Disaster Optimizer
EFO	: Electromagnetic Field Optimization
EO	: Equilibrium Optimizer
HGSO	: Henry Gas Solubility Optimization
MVO	: Multi-Verse Optimizer
NRO	: Nuclear Reaction Optimization
SA	: Simulated Annealing
WDO	: Wind Driven Optimization
ABC	: Artificial Bee Colony
ACO	: Ant Colony Optimization
ALO	: Ant Lion Optimizer
ARO	: Artificial Rabbits Optimization
AVOA	: African Vultures Optimization Algorithm
BA	: Bat-inspired Algorithm

BES	: Bald Eagle Search
CSA	: Cuckoo Search Algorithm
DO	: Dragonfly Optimization
EHO	: Elephant Herding Optimization
FFA	: Firefly Algorithm
FOX	: Fox Optimizer
GOA	: Grasshopper Optimization Algorithm
GWO	: Grey Wolf Optimizer
HHO	: Harris Hawks Optimization
JA	: Jaya Algorithm
MFO	: Moth-Flame Optimization
NMRA	: Naked Mole-Rat Algorithm
PFA	: Pathfinder Algorithm
PSO	: Particle Swarm Optimization
SSO	: Salp Swarm Optimization
STO	: Siberian Tiger Optimization
TDO	: Tasmanian Devil Optimization
WOA	: Whale Optimization Algorithm
ZOA	: Zebra Optimization Algorithm
AOA	: Arithmetic Optimization Algorithm
CGO	: Chaos Game Optimization
HC	: Hill Climbing

SCA	: Sine Cosine Algorithm
TS	: Tabu Search
AEO	: Artificial Ecosystem-based Optimization
GCO	: Germinal Center Optimization



BÖLÜM BİR

GİRİŞ

Bir problemin çözümünde istenilen durumu minimize etmek veya maksimize etmek için en iyi seçeneklerin araştırılması ve uygun çözümlerin tespit edilmesi işlemine optimizasyon denir. Gerçek hayat problemleri optimizasyon yöntemlerinin en çok kullanıldığı alanlardan biridir (Altay vd., 2024). Örneğin bir yere giderken en kısa yoldan gitmeyi isteriz ya da satın almak istediğimiz bir ürünün en iyisini en ucuza almak isteriz. Bir şirket en fazla kâr elde etmek isterken en az iş gücü kullanmak ve maliyeti düşürmek ister. Optimizasyon yöntemlerinin sıklıkla kullanıldığı bir diğer alan olan mühendislik uygulamalarında ise bir mühendis probleme en uygun sistemi planlamak veya mevcut kaynakları kullanarak ihtiyaca yönelik en iyi tasarımı yapmak ister (Y. Li vd., 2024; Raja vd., 2023). Tasarımında hatayı en aza indirmeyi ve en iyi tahminleri öngörmeyi ister. Tüm bu örnekler, aslında hayatımızın her alanında optimizasyon süreçlerini farkında olarak ya da olmayarak kullandığımızı gösterir ve anlaşılacağı üzere optimizasyon alternatif durumlar arasında en iyiyi bulmayı amaç edinir.

Optimizasyon süreçleri planlanırken uygulanacak yöntemleri belirlemeden önce, bir optimizasyon probleminin yapısı ve bileşenleri hakkındaki anlayışımızı matematiksel açıdan geliştirmemiz büyük önem taşımaktadır. Bir optimizasyon probleminin matematiksel gösterimi amaç fonksiyonu, çözüm (olası çözüm, aday çözüm veya optimal çözüm olarak da kullanımlar mevcuttur), karar değişkenleri, kısıtlamalar, durum değişkenleri gibi temel bileşenleri içerir (X. S. Yang, 2010b).

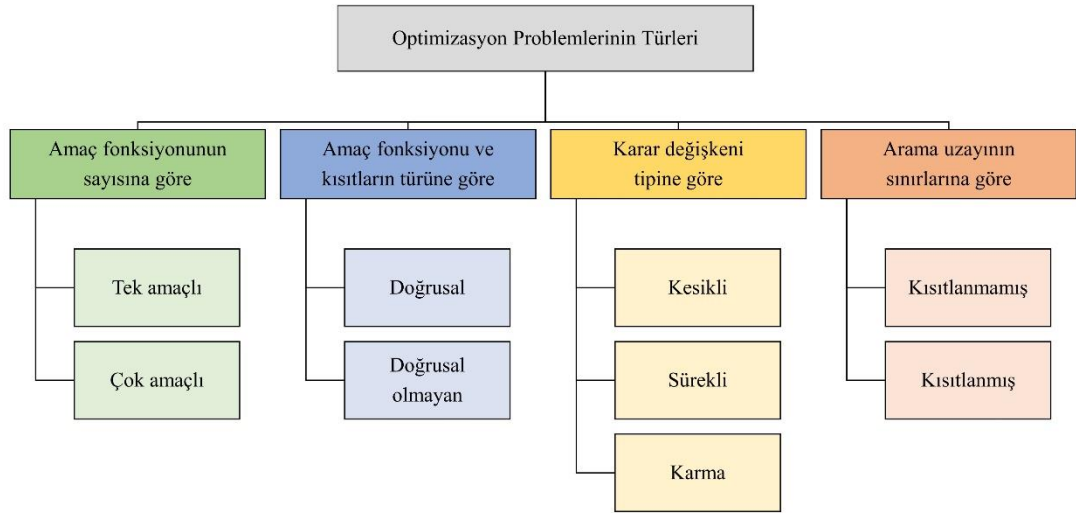
Amaç fonksiyonu, optimize edilmesi gereken, en üst düzeye çıkarılması veya en aza indirilmesi gereken fonksiyondur (Bozorg-Haddad vd., 2017). Bir çözümün ne kadar arzu edilir olduğunu niceliksel olarak belirler. Örneğin, bir optimizasyon probleminde amaç fonksiyonu kâr maksimizasyonunu veya riskin en aza indirilmesini temsil edebilir. Problemden optimize edilmek istenen olgu bir tane ise tek amaçlı, iki veya daha fazla ise çok amaçlı optimizasyon problemi olarak adlandırılır. Ayrıca amaç fonksiyonu doğrusal bir fonksiyon olabildiği gibi doğrusal olmayan ifadeler de amaç fonksiyonu olarak kullanılabilir. Diğer bir önemli bileşen çözümdür, çözüm probleme ait en iyi sonucu ifade eder. En iyi çözümü bulma aşamasında birden fazla

olası çözüm amaç fonksiyonu ile kontrol edilir. Aday çözümler, bir dizi olarak temsil edilebilir ve genellikle çok boyutludur. Aday çözümlerin her biri karar değişkeni olarak tanımlanır (Zolghadr-Aslı, 2024). Bunlar, tam sayılar veya kategoriler gibi ayrık değerler alıyorsa kesikli değişkenler, gerçek sayı kümesinden çekiliyor ve belirli bir aralıktaki herhangi bir değeri alıyorsa sürekli değişkenler olarak tanımlanabilir. Bir çözümdeki karar değişkenleri tek tipte olabileceği gibi sürekli ve kesikli değerler birlikte de yer alabilir, bu durumda karma karar değişkeni olarak isimlendirilir. Kısıtlamalar, karar değişkenlerine uygulanan ve onları kabul edilebilir eşikler içinde tutan sınırlamalardır. Kısıtlamalar, problem kurulumunda dikkate alınması gereken kaynaklardaki, yasal gerekliliklerdeki veya fiziksel koşullardaki sınırlamalar nedeniyle ortaya çıkar. Olası çözümler eğer sonsuz uzayda aranıyorsa ve bir sınırlandırmaya sahip değilse kısıtlanmamış olarak tanımlanır. Bir değer aralığında sınırlandırılıyor veya problemin ifadesinde matematiksel bir sınırlandırma bulunuyor ise bu problemin karar değişkenleri kısıtlanmış olarak tanımlanır. Durum değişkenleri ise karar değişkenleri değiştikçe değişen bağımlı değişkenlerdir. Optimizasyon sürecinde doğrudan kontrol edilmeseler de, karar değişkenlerinin sistem performansını nasıl etkilediğine dair içgörüler sağlarlar.

Bir optimizasyon probleminin bileşenleri, olası çözümlerin sistematik olarak değerlendirilmesine ve karşılaştırılmasına olanak tanıyan standart bir optimizasyon modeli formüle etmek için birlikte çalışır. Bu bileşenleri anlamak ve kavramlara ait parametrelerin türlerini detaylandırmak amacıyla optimizasyon problemlerinin türleri dört kategoride (Şekil 1.1) özetlenmiştir (X. S. Yang, 2010b).

Amaç fonksiyonunun türü, karar değişkenlerinin tipi ve arama uzayının sınırları optimizasyon problemlerinin çözümünde kullanılacak modellerin belirlenmesinde oldukça önemlidir. Optimizasyon yöntemleri deterministik ve stokastik olmak üzere iki şekilde incelenebilir (X. S. Yang, 2010a). Deterministik yöntemler, durum değişkenlerinin modeldeki parametreler ve bu değişkenlerin önceki durum kümeleri tarafından aynı şekilde belirlendiği modellerdir. Bu nedenle, bir deterministik model belirli bir parametre ve başlangıç koşulları kümesi için aynı şekilde çalışır ve kesin çözümler üretir (Renard vd., 2013). Bununla birlikte, deterministik modeller bazen kararsızdır. Başlangıç koşullarının veya problemi yöneten parametrelerin küçük

bozulmaları nihai çözümde büyük değişikliklere yol açar (Lorenz, 1963). Bu nedenle, çözümün benzersiz olmasına rağmen, etki alanının tek bir noktasında tek bir yönetici parametreyi veya başlangıç koşulunu hafifçe bozarak önemli ölçüde farklı çözümler elde edilebilir. Bunun tersine, stokastik yöntemler parametreleri tek bir değer yerine rastgele değişkenler veya dağılımlarla tanımlar (Shabani vd., 2019a). Buna karşılık, durum değişkenleri de olasılık dağılımlarıyla tanımlanır. Bu nedenle, stokastik bir model, modelleyicinin modellenen doğal sistemin içsel belirsizliğini değerlendirmesine olanak tanıyan eşit derecede olası çözümlerden oluşan bir manifold üretir. Stokastik modeller, tek bir kesin çözüm sunmazlar, bunun yerine en iyi çözüme olası sonuçlar sağlayarak karar almayı destekleyebilir. Bu durum dezavantaj olarak görülse de deterministik yöntemlerle yakalanamayan düzensizlikler ve karmaşıklıklar içeren sistemleri modellemek için uygundur. Karmaşık çevresel olayların gerçekçi bir temsili sağlayarak süreçlerin daha iyi anlaşılmasını sağlarlar. Bunun yanı sıra stokastik yaklaşımlar esnektir, çeşitli durumlara uyum sağlayabilir ve bu da onları çeşitli alanlarda uygulanabilir hale getirir (Renard vd., 2013).



Şekil 1.1 Temel bileşenlerine göre optimizasyon problemleri

Stokastik yaklaşımlar sezgisel yöntemler ve meta-sezgisel (MS) yöntemler olarak iki kategoriye ayrılır. Genişlik öncelikli arama ve derinlik öncelikli arama bu teknik kapsamındaki iki sezgisel yöntemdir (Sivaraj & Gopalakrishnan, 2003). Kör veya bilgisiz arama olarak adlandırılan bu sezgisel yöntemler, genellikle arama için tüm

özüm alanını gerektirir. özümü bulmak zaman alır ve daha fazla bellek alanı tüketir. En iyi-ilk arama (Sen vd., 2004), A* arama (Goldenberg, 2017), çift yönlü arama (Ghosh & Mahanti, 1991), ışın arama (Passos, 2000) teknikleri, çeşitli algoritmalara sahip olan bilgilendirilmiş arama olarak adlandırılır. Bu sezgisel arama teknikleri, uygulamaya göre gerekli alan, özel bilgi sağlanırsa ve düzgün bir şekilde uygulanırsa etkili teknikler olarak kullanılabilir. Buna rağmen sezgisel algoritmalar özümleri yinelemeler yoluyla belirler ve açgözlü yöntemlerdir (Savsani & Savsani, 2016). Öte yandan, "meta-sezgisel" terimi, "daha yüksek seviye" ve "yeni stratejiler keşfetme sanatı" anlamına gelen Yunanca kökleri birleştirerek, problem özmeye yönelik yapılandırılmış bir yaklaşımı belirtir (Sörensen, 2015). Sezgisel yöntemler genellikle geçici ve probleme bağlıdır. MS yöntemler sezgisel yöntemlerin bazı dezavantajlarını miras almalarına rağmen evrensel olarak daha uygulanabilir bir çerçeve sağlamayı amaçlar.

MS algoritmaların tam kökenini kesin olarak belirlemek zordur. Ancak belki de geleneksel optimizasyon yöntemlerinin ilkelerinden sapmak için en dikkat çekici girişim, desen arama algoritmasının tanıtılmasıydı. Hooke ve Jeeves (1961) tarafından teorileştirilen, “rastgele arama” olarak da bilinen desen arama algoritması, tartışmasız bir şekilde, optimizasyon tekniklerinin tarihindeki temel noktalardan birini temsil eder, ünkü bu algortmada kullanılan ilkeler, MS optimizasyon algoritmaları olarak bilinen bir sonraki nesil optimizasyon yöntemlerinden gelen ilkelere oldukça benzemektedir. 1990'ların başlarında, birçok araştırmacı optimizasyon problemlerini ele almak için hesaplamalı zekaya dayalı çerçeveleri kavramsallaştırmaya başlamıştı. Bu yöntemler, rastgele örneklemenin teorik temeline dayanıyordu. Bu yöntemlerin ayırt edici özelliğı, kullanıcı tarafından belirli bir optimizasyon problemine göre uyarlanabilecek şekilde ayarlanabilen rehberli bir arama yürütmeleri idi. Bu nedenle Glover (1986), optimizasyon yöntemlerinin bu dalını ayırt etmek için "meta-sezgisel" ifadesini ortaya attı. Fikir, bu yöntemlerin, sezgisel bir çerçevenin yapısını değıştirebilmeleri ve kontrol edebilmeleri anlamında sezgisel yöntemlerin kavramsal olarak üstünde sıralanması ve onları geçici yöntemden daha evrensel olarak uygulanabilir bir çerçeveye yükseltmeleri idi.

MS optimizasyon algoritmaları, arama alanlarını ve sorunlarını daha iyi ele almak için ince ayarlı olacak şekilde tasarlanmıştır. Arama alanının benzersiz özelliklerini yansıtacak ve bunlara uyum sağlayacak şekilde ayarlanabilen optimum çözüme doğru bizi yönlendirmek için bir dizi deneme-yanılma kullanırlar. Bu algoritmalar, arama aşamasında elde edilen bilgileri kullanarak arama alanında gezinmelerine olanak tanıyan bir yönlendirme mekanizmasına sahiptir. Diğer optimizasyon yöntemlerinden farklı olarak, bu algoritmalar arama alanı hakkında bilgi edinmek ve arama süreçlerinde gezinmek için doğrudan bir arama kullanır. Bu doğrudan arama, algoritmanın bir fonksiyonun türetilmesinden elde edilen bilgilere başvurmak yerine, arama sürecinde doğrudan fonksiyon değerlerini kullanacağını gösterir. Bu fikirleri pratik bir şekilde bir araya getirmek için, bu algoritmaların, en iyiye yakın çözümlere ulaşana kadar bir dizi komutu tekrar tekrar yürütebilecek şekilde tasarlanmaları gerekir. Aslında, tüm MS algoritmalarında görülen yinelemeli süreç, bu algoritmaların her denemede alanı kapsamlı bir şekilde aramalarına ve performanslarını ayarlamalarına olanak tanır.

MS algoritmalar, karmaşık gerçek dünya optimizasyon problemlerini çözmede en temel ve pragmatik yollarından biri olarak kendilerini kanıtlamışlardır. Bu yöntemler teorik olarak yüksek boyutlu, sürekli olmayan, kısıtlı veya türevlenemeyen hedef fonksiyonlarını ele alabilir (Zhang vd., 2019). Bu, bu yöntemlerin dezavantajları olmadığı anlamına gelmez. İlk olarak, MS algoritmalar küresel optimum çözümler yerine yakın optimumları takip edebilir. İkinci olarak, bu sürecin stokastik doğası olan deneme-yanılma bileşeni göz önüne alındığında, bu algoritmaların optimum çözüme yakınsayabileceğine dair garanti yoktur. Ayrıca, yerel ve küresel optimumları ayırt etmenin kesin bir yolu olmadığından, algoritmanın yerel optimumlara hapsolme olasılığı her zaman vardır. Elbette, algoritmaların parametreleri ayarlanarak problem odaklı uygun bir ayar bulunabilir. Sonuç olarak, bu eksiklikler bu yöntemlerin önemini ve karmaşık gerçek dünya problemlerini ele alma kapasitelerinin göz ardı edilmesine neden olmamalıdır (Zolghadr-Aslı, 2024).

Bu tez çalışmasında, optimizasyon problemlerinin çözümü için yeni bir MS yöntem önerilmektedir. Önerilen bu yöntem Yapay Kan Dolaşım Sistemi (YKDS) algoritması olarak adlandırılır. İnsan dolaşım sistemindeki kan basıncı ve kan hacminin kontrolüne

odaklanır ve bu hayati işlevlerin anlık olarak düzenlenerek optimize edilme sürecinden ilham alır. Algoritma matematiksel alt yapısında ilham kaynağına benzer şekilde sinirsel ve hormonal sistemleri taklit eden düzenleyicileri barındırır, arama alanını özel gruplardan oluşan bir popülasyon kullanarak keşfederler. Ancak, arama uzayı içinde belirli grupları hedef alan diğer yöntemlerin aksine, bu yöntem sadece en iyi çözümü bulmaya öncelik vermekle kalmaz, aynı zamanda en kötü çözümlerin etkisini göz önünde bulundurmanın ve keşif aşamasında potansiyel yerel minimumlardan kaçınmanın önemini vurgular. Önerilen algoritma, ilham ve matematiksel ifade, popülasyon ayrımı ve optimizasyon stratejisi gibi ayırt edici özellikleri nedeniyle mevcut yöntemlerden ayrılmaktadır. Bu farklılıklar, önerilen yöntemin özgünlüğünü ortaya koymaktadır. Önerilen algoritma ve yürütülen çalışmaların mevcut literatüre katkıları aşağıda özetlenmiştir:

- ✓ Literatüre MS temelli doğadan ilham alan özgün bir algoritma kazandırılması amaçlanmıştır, bu algoritma çeşitli alanlardaki zorlu problemleri ele almak için önemli bir alternatif olarak hizmet edebilir.
- ✓ MS algoritmaların test edilebilmesi için farklı tipteki test fonksiyonları tek bir çalışmada toplanmıştır, çalışmanın kodları açık kaynak kodlu olarak paylaşılmıştır. Bu sayede gelecekte yapılacak olan çalışmalara yol gösterici olabilir.
- ✓ Gerçek hayat problemi olarak karşımıza çıkan tasarım kısıtlamaları içeren yaygın mühendislik problemleri belirlenip bunların ayrıntıları incelenmiş ve zorlukları karşılaştırılmıştır. Çalışma mühendislik alanında çalışan araştırmacıları MS yöntemlerin kullanımına teşvik edebilir.
- ✓ Literatürde MS yöntemlerin değerlendirilmesinde kullanılan farklı metrikler detaylı olarak ele alınmıştır. Bu, benzer alanda çalışan araştırmacılar için kaynak oluşturabilir.
- ✓ Popüler MS algoritmaların yanı sıra daha az bilinen yöntemleri de çalışmaya dâhil edip kapsamlı bir karşılaştırma yürütülmüştür. Bu sayede düşük popülasyon boyutu ve yinleme sayısı gibi ilkel koşullar altında tekniklerin performansları gözlemlenebilir. Ayrıca sağlamlık, yakınsama davranışı, hesaplama maliyeti gibi kritik ölçütlerde MS algoritmaların avantaj ve dezavantajlarını ortaya konmuştur. Bu, belirli bir alanda MS algoritmalar ile

alıřan arařtırmacılar iin az bilinen ve esnek, verimli algoritmaları ne ıkarılmasını saėlayabilir.

alıřmanın detayları ileriki blmlerde detaylı olarak ele alınmıřtır. Blm 2, MS algoritmalara kapsamlı bir bakıř sunar. Blm 3, nerilen YDSA algoritmasını detaylı bir řekilde ele alır. Blm 4, algoritmaya ait deneysel sreleri tanımlar. Blm 5, nerilen modelin etkinliėini inceler; Blm 6 ise nerilen YDSA algoritmasını literatrdeki mevcut MS yntemler ile karřılařtırır. Blm 7 elde edilen sonuların bir zetini sunar, sonuları tartıřarak ıkarımlar yapar ve son olarak Blm 8 alıřmanın kısıtlamalarına yer verir, gelecekte yapılabilcek uygulamalar iin nerilerde bulunur.



BÖLÜM İKİ

METASEZGİSEL (MS) YÖNTEMLER

Bu bölümde MS yöntemler ele alınmıştır. Bölüm 2.1 literatürdeki MS algoritmaları tarihsel bir gelişim ile ele alır, geçmişten günümüze ışık tutar. Bir çok MS algoritmanın türetilmesinde motivasyon kaynağı olan teoremi ele alır, yeni yöntemler önerilmesi ya da yöntemlerin geliştirilmesinin önemini vurgular. Bölüm 2.2 algoritmaları farklı özelliklerini göz önünde bulundurarak sınıflandırır, bu sayede yöntemlerin benzerlik ve farklılıklarını gözler önüne serer. Bölüm 2.3 MS algoritmaların yapısı hakkında bilgi vererek yöntemlerin en iyileme aşamalarının ve çalışma prensibinin anlaşılmasını sağlar.

2.1 MS Algoritmaların Gelişimi

2.1.1 Tarihsel Bakış

Genetik algoritma tartışmasız en iyi bilinen MS optimizasyon yöntemlerinden biridir ve günümüzde hala en çok kabul gören algoritmalarından biri olmaya devam etmektedir. Genetik algoritma özünde, belirli bir türün yeni bir çevreye uyum sağlayabildiği kademeli bir süreç olan evrimi taklit etmeye çalışır. Holland'ın çığır açan çalışması (Holland, 1975), genetik algoritma olarak bilinen yaklaşımın temelini resmileştirilmiş ve popülerleştirilmiştir ancak teknik olarak genetik algoritmanın temel fikri daha eski yıllara dayanmaktadır. 1957'de Fraser tarafından önerilmiş olan kavram (Fraser, 1957a, 1957b), 1962'de Fogel'in evrimsel programlama yaklaşımına (Fogel, 1962), 1965'te ise Rechenberg'in evrimsel strateji fikrine (Rechenberg, 1965) öncü olmuştur. Ve evrimsel hesaplama fikri karmaşık problemleri çözmek için o yıllarda uygulanmaya başlamıştı (Barricelli, 1962; Fogel vd., 1966). 1983 yılında Kirkpatrick ve arkadaşları tarafından ortaya atılan benzetilmiş tavlama (Kirkpatrick vd., 1983), evrimsel bakış açısına farklı bir soluk getirerek metalurjideki tavlama işleminden esinlenilen yörünge tabanlı bir algoritma olarak dikkat çekmiştir. 1989 yılında ise genetik algoritma ile aynı temele dayanan ancak bireylerin ağaç yapıları olarak kodlandığı genetik programlama (Koza, 1989) ve erken

yakınsama olasılığını azaltmak için yerel bir arama tekniği kullanan memetik algoritma gelişmeleri izlemiştir (Moscato, 1989).

Hayvanların iletişim ve beslenme gibi sosyal davranışlarının dikkat çekmesi MS gelişmeleri evrimsel yöntemlerden sürü zekâsı yaklaşımlarına çevirmiştir. İlk olarak 1992 yılında yiyecek arayan bir karıncaların davranışı üzerine modellemiş olan karınca kolonisi optimizasyonu (Dorigo, 1992), daha sonra da 1995 yılında kuşların uçuş ve beslenme gibi sosyal davranışı simüle etmek için tasarlanan parçacık sürüsü optimizasyonunu (Eberhart & Kennedy, 1995) geliştirilmiştir. Sonraki iki yıl içinde, optimizasyon alanında oldukça dikkat çeken ve tartışmalı bir çalışma olan “No Free Lunch (NFL)” teoremi (Wolpert, 1996; Wolpert & Macready, 1997) tanıtıldı. Bazı araştırmacılar NFL'nin önemli bir içgörüye sahip olduğunu savunurken, diğerleri bu alandaki araştırmalarla çok az ilgisi olduğunu ileri sürmüştür.

2.1.2 NFL Teoremi

Günümüzde çok sayıda MS optimizasyon algoritmasının varlığı, bu algoritmaların iyi sonuçlar üretmesiyle ilişkilidir. Ancak, optimizasyon problemlerini ele almak için birden fazla algoritmaya ihtiyaç duyulmasının nedeni, veri bilimi alanında bilinen NFL teoremidir (Wolpert & Macready, 1997). Bu teorem, belirli bir problem hakkında önceden bir varsayım olmaksızın, hiçbir algoritmanın diğerlerine üstünlük sağlayamayacağını ifade eder (Culberson, 1998). Dolayısıyla, MS algoritmalar için genel amaçlı bir optimizasyon stratejisinin varlığı mümkün değildir.

NFL teoremi, evrensel olarak üstün bir algoritmanın olamayacağını ve bu nedenle her algoritmanın tüm optimizasyon problemlerini çözebileceği ideal bir ayarın da mevcut olmadığını ortaya koyar (Ho & Pepyne, 2002). En iyi performansı elde etmek için, her bir problem için uygun parametre ayarlarını bulmak gerekmektedir. Bu bağlamda, bir algoritmanın ince ayarını yapmak, MS optimizasyonun ayrılmaz bir parçasıdır. Başlangıç noktası olarak sezgi ve deneyim kullanılabilir; ancak, en iyi sonuçları almak için deneme-yanılma sürecine başvurmak şarttır.

2.1.3 Yeni Nesil Algoritmalar

2000'li yıllarda önerilen yeni MS optimizasyon yöntemi sayısındaki sıçramalar, NFL'nin bir çok araştırmacıya motivasyon oluşturduğunun kanıtıdır. NFL alana özgü algoritmalar geliştirmek ve daha fazla araştırma yapmak için altın bir fırsatın kilidini açmıştır.

İlk yıllarda bir grup araştırmacı insan davranışını mercek altına aldı ve insan temelli MS yöntemler önerdi. 2001 yılında caz müzisyenlerinin doğaçlama sürecinden esinlenen harmoni arama algoritması (Geem vd., 2001) dikkat çekti. 2002'de insanların başkalarının davranışlarını gözlemleyerek öğrendiğini ve davranışı yönlendirmek için başkalarının davranışlarını sembolize ettiğini varsayan sosyal bilişsel optimizasyon algoritması (Xie vd., 2002) önerildi. 2003 yılında, bireyler arasındaki toplumsal etkileşimlerden esinlenen toplum ve medeniyet optimizasyon algoritması türetildi (Ray & Liew, 2003). Bu insan davranışlarını taklit yaklaşımını emperyalizm kavramından esinlen emperyalist rekabetçi algoritma (Atashpaz-Gargari & Lucas, 2007) ve sınıftaki öğretme-öğrenme ortamından esinlenen öğretme-öğrenmeye dayalı optimizasyonu (R. V Rao vd., 2011) izledi.

Parçacık sürü optimizasyonundan sonra, mevcut yaklaşımların yüzlerce yeni sürü tabanlı benzerleri, varyantları ve melezleri önerildi. Bazı önemli algoritmalar şunlardır: bal arılarının yiyecek arama davranışından esinlenen yapay arı kolonisi (Karaboğa, 2005), ateş böceklerinin yanıp sönme davranışından esinlenen ateş böceği algoritması (X.-S. Yang, 2009), guguk kuşlarının diğer kuşların yuvalarına yumurtlama davranışından ilham alan guguk kuşu arama algoritması (X.-S. Yang & Deb, 2009) ve yarasa türlerinin yankı konumlama davranışından esinlenerek oluşturulan yarasa algoritması (X. S. Yang, 2010a). Sonraki yıllarda gri kurtların sosyal hiyerarşisinden ve avlanma davranışlarından esinlenerek türetilen gri kurt optimizatörü (Mirjalili vd., 2014), araştırmacılar tarafından oldukça ilgi gördü. Daha sonra, aynı araştırmacı kambur balinaların kabarcık ağlarıyla beslenme davranışından esinlenerek balina optimizasyon algoritmasını (Mirjalili & Lewis, 2016), bir sonraki yıl ise çekirge sürülerinin davranışını matematiksel olarak modelleyen çekirge optimizasyon algoritmasını (Saremi vd., 2017) önerdi. Bunun dışında benzer yıllarda, matematik ve fizik yasalarından ilham alan bazı yöntemler de

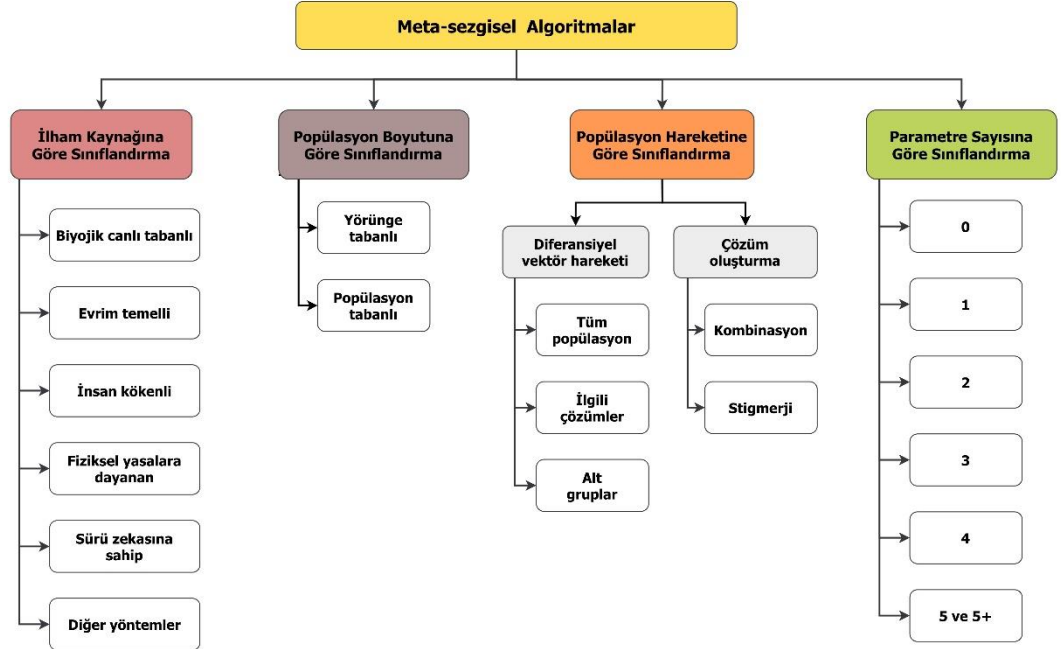
dikkat çekti. Evrenin yaratılış, genişleme ve küçülme teorisinden esinlenen büyük patlama-büyük çöküş algoritması (Erol & Eksin, 2006), Newton'un yerçekimi yasasından esinlenen yerçekimsel arama algoritması (Rashedi vd., 2009) ve trigonometrik fonksiyonlardan esinlenen sinüs kosinüs algoritması (Mirjalili, 2016b) bunların popüler olanlarıdır.

Tüm bu algoritmaların yanı sıra MS yöntemler, son 20 yılda durdurulamaz bir eğilimle hızla artmıştır. Bu alanda yapılan bibliyometrik analizler, 500'den fazla farklı algoritma olduğunu göstermiştir (G. Li vd., 2024; Rajwar vd., 2023). Mevcut tüm algoritmaları incelemek ve matematiksel alt yapılarını özetlemek zordur. Bu yüzden MS yöntemlerin temellerini ve genel prosedürlerini anlamak için sonraki alt bölümlerde sınıflandırma türleri ve genel yapıları kapsamlı olarak ele alınmıştır.

2.2 MS Algoritmaların Sınıflandırılması

Doğadaki yeni keşifler ve dünya çapında araştırma çabalarının artışı, biyolojik süreçlere ve bunların hesaplamalı sorunlara odaklanılmasına olan ilgisini ateşlemiştir. Bu durum, literatürde birçok yeni MS yöntemin önerilmesine yol açmıştır. MS yöntemlerin sayısının hızla artmasıyla birlikte, araştırmacılar bu algoritmaları anlama ve ilkeli, tutarlı kriterler altında inceleme ihtiyacı benimsemiş, çeşitli sınıflandırmalar önermiştir. İlham kaynağına dayalı sınıflandırma bunların en yaygın olanıdır, yöntemlerin doğal esin süreçleri hakkında bilgi sağlamaktadır (G. Li vd., 2024). Ancak algoritmaların sayıca artması, her geçen gün farklı kategorilerin tanımlanmasına sebep olmuş ve bazı yöntemlerin ortak farklı kategorilere dâhil olması sınıflandırmayı zorlaştırmıştır. Ayrıca bu taksonominin algoritmaların matematiksel içyapısı hakkında bilgi sağlamaması araştırmacıları farklı bakış açılarına yönlendirmiştir. Popülasyon boyutuna dayalı başka bir sınıflandırma, bir yinelemede arama uzayına dağıtılan araçların sayısına ilişkin içgörü sağlar (Mirjalili, 2016b). Ancak, nispeten az sayıda algoritma bir gruba girdiğinden bu taksonomi oldukça düzensizdir. Bir grup araştırmacı (Molina vd., 2020), MS yöntemleri algoritma davranışlarına göre sınıflandırmayı önermiş; algoritmaların vektörel hareketlerine ve iç işleyişi hakkında bilgi sağlayan çözüm oluşturma stratejilerine göre ele almışlardır. Başka bir araştırma ise MS yöntemlerin performanslarında parametrelerin önemini vurgulamış ve

parametre sayısına dayalı bir sınıflandırma sunmuşlardır (Rajwar vd., 2023). Sonraki bölümlerde, bu dört farklı kategorizasyon tipleri (Şekil 2.1) detaylandırılmış ve bu sınıflara ait MS yöntemlere örnekler verilmiştir.



Şekil 2.1 MS algoritmaların sınıflandırılması

2.2.1 İlham Kaynağına Dayalı Sınıflandırma

İlham kaynağına göre taksonomi en eski sınıflandırmadır. Ayrıca, doğadan ilham alan algoritmalar veya MS kavram öncelikle doğal veya biyolojik olaylara dayandığından yararlı bir sınıflandırma olarak görülebilir. İlham kaynağına göre, MS algoritmalar farklı yazarlar tarafından çeşitli şekillerde kategorize edilmiştir. Bir grup araştırmacı (Fister vd., 2013), bunu sürü zekası tabanlı algoritmalar, biyolojik tabanlı algoritmalar, fizik-kimya tabanlı algoritmalar ve diğerleri olarak dört kategoriye sınıflandırırken, başka bir grup (Siddique & Adeli, 2015) bunu fizik tabanlı, kimya tabanlı ve biyoloji tabanlı algoritmalar olarak üç alt gruba ayırmıştır. Bir araştırma ise MS algoritmaları geniş bir koleksiyona sahip platformlar arası bir python kütüphanesi önermiştir. Araştırmacılar çalışmalarında yöntemleri sekiz farklı temele (biyo-tabanlı, evrimsel-tabanlı, insan-tabanlı, matematik-tabanlı, müzik-tabanlı, fizik-tabanlı, sürü-tabanlı, sistem-tabanlı) dayandırmışlardır (Van Thieu & Mirjalili, 2023).

Tez kapsamında yaygın olarak tanınan beş kategoriye bağlı kalınarak sayıca daha az olan kategoriteler birleştirilmiş ve altı sınıfta incelenmiştir:

Biyolojik canlı tabanlı yöntemler (1), optimum çözümler bulmak için canlı organizmaların davranışlarını taklit eden algoritmalarıdır. Bağlantıcılık, sosyal davranış ve mikrobiyolojiden ilham alan bu yöntemler biyolojik süreçleri simüle ederek biyolojik modeller oluşturmayı amaçlar. Virüs kolonisi araması (M. D. Li vd., 2016), sümüksü küf algoritması (S. Li vd., 2020) ve simbiyotik organizmalar arama (Cheng & Prayogo, 2014) algoritması bu gruba örnek gösterilebilir.

Evrimsel temelli yöntemler (2), en iyi çözümü bulmak için nesiller boyunca evrimleşen aday çözüm popülasyonlarını içerir. Doğal evrimsel süreçlere dayanan bu algoritmalar, doğal seçilimi ve genetik çeşitliliği taklit ederek çözümleri yinelemeli olarak iyileştirir. Bu kategorideki en popüler yöntem, Darwin evrim teorisinden esinlenen genetik algoritma'dır (Holland, 1992). Evrim stratejisi (Schwefel, 1984), diferansiyel evrim (Storn & Price, 1997) ve mercan kayalıkları optimizasyonu (Salcedo-Sanz vd., 2014) da diğer popüler evrimsel temelli yöntemlerdir.

İnsan kökenli algoritmalar (3), genellikle öğrenme ve bilgi paylaşımı kavramlarını birleştirerek insan etkileşimlerini, adaptasyonlarını ve iş birliğini taklit eder. Dahası, kaos veya emperyalizm gibi bireysel ve sosyal davranışları modellemeyi ve insan etkileşimini, adaptasyonunu ve duygusal gösterimi simüle etmeyi amaçlarlar. Kültür algoritması (Reynolds, 1994), beyin fırtınası optimizasyonu (Shi, 2011), arama kurtarma optimizasyonu (Shabani vd., 2019b), yaşam seçimi tabanlı optimizasyon (Khatri vd., 2020a), adli temelli soruşturma optimizasyonu (Chou & Nguyen, 2020), öğrenci psikolojisine dayalı optimizasyon (Das vd., 2020) bu tekniklere örnek olarak gösterilebilir.

Fiziksel yasalara dayanan yöntemler (4), yerçekimi, elektrik yükü ve kimyasal reaksiyonlar gibi doğa yasaları ve olguları temel alan algoritmalarıdır. Bu algoritmalar, çözümleri optimize etmek için yerçekimi veya hareket gibi doğal süreçleri simüle eder. Bu grupta en eski ve popüler yöntem, benzetilmiş tavlama algoritmasıdır (Laarhoven & Aarts, 1987). Elektromanyetik alan optimizasyonu (Abedinpourshotorban vd., 2015), nükleer reaksiyon optimizasyonu (Wei vd., 2019), atom arama optimizasyonu

(Zhao vd., 2019), denge optimizasyonu (Faramarzi vd., 2020) fizik yasalardan ilham alarak geliştirilen diğer yöntemlerdir.

Sürü zekasına sahip yöntemler (5), hayvanların işbirliğini ve etkileşimini gözlemleyerek optimum çözümler bulmayı amaçlayan, doğadaki sürülerin kolektif davranışını taklit eden MS algoritmalarıdır. En popüler sürü temelli yöntemlerden biri olan parçacık sürü optimizasyonudur (Eberhart & Kennedy, 1995). Bu kategorideki diğer geleneksel popüler iki yöntem karınca koloni optimizasyonu (Socha & Dorigo, 2008) ve yapay arı kolonisi (Karaboga & Basturk, 2007) yöntemleridir. Hayvanların kendi aralarındaki bu ilginç davranışları özellikle araştırmacıların dikkatini çekmiş ve bu alanda son yıllarda oldukça yenilikçi teknikler geliştirilmiştir. Gri kurt optimizasyonu (Mirjalili vd., 2014), balina optimizasyon algoritması (Mirjalili & Lewis, 2016), yarası algoritması (X. S. Yang, 2010a) ve çekirge optimizasyon algoritması (Saremi vd., 2017) bu yenilikçi yaklaşımlardan yalnızca birkaçıdır.

Son olarak diğer yöntemler (6) kategorisi matematiksel, sistem odaklı ve müzikten ilham alan yaklaşımlardan türetilen teknikleri kapsar. Matematik tabanlı algoritmalar (Abualigah vd., 2021; Mirjalili, 2016b; Qais vd., 2022) cebirsel ve geometrik prensipleri kullanırken, sistem tabanlı yöntemler (Eskandar vd., 2012; Villaseñor vd., 2018; Zhao vd., 2020) çeşitli sistemlerin çalışma prensiplerinden türetilir. Müzik tabanlı metodolojiler (Geem vd., 2001), çözümleri geliştirmek için müzik kavramlarından yararlanır.

2.2.2 Popülasyon Boyutuna Dayalı Sınıflandırma

Bir bölgeyi araştırmak söz konusu olduğunda birden fazla etkenin işbirliği, tek bir etkenden daha iyi çalışır. Bilgi paylaşımı, hafıza vb. gibi çeşitli avantajlara sahiptir. Bu düşünce ile yola çıkan araştırmacılar, mevcut algoritmaları yörünge tabanlı ve popülasyon tabanlı algoritmalar olarak iki kategoriye ayırılır.

Bazı klasik algoritmalar yörüngeler üzerine kuruludur, bu da çözümün her yineleme sırasında hareketinin tek bir yörünge oluşturduğu anlamına gelir. Prosedürün başında, rastgele bir tahmin yapılır ve sonuç her sonraki adımda iyileştirilir. Bu tip algoritmalar yörünge tabanlı algoritmalar olarak sınıflandırılır. Örneğin, benzetilmiş tavlama

(Laarhoven & Aarts, 1987), uygulandığı tasarım veya arama alanında parça parça hareket eden tek bir etken veya çözüm içerir. Daha iyi hamleler ve çözümler her zaman hoş karşılanırken, idealden daha az hamlelerin kabul edilme olasılığı daha yüksektir. Bu eylemler arama alanında bir yol oluşturur ve bu yolun küresel en iyi çözüme ulaşma olasılığı sıfırdan farklıdır. Tabu araması (Glover, 1990), büyük tufan algoritması (Dueck, 1993), yinelemeli yerel arama (Lourenço vd., 2003) ve açgözlü rastgele uyarlanabilir arama prosedürleri (Feo & Resende, 1995) bu kategorinin birkaç örneğidir.

Popülasyon tabanlı algoritmalar kategorisi ise MS algoritmaların büyük bir bölümünü kapsar. Popülasyon tabanlı algoritma birden fazla arama aracı kullandığından, arama alanının çeşitliliğinin olağanüstü bir şekilde keşfedilmesini sağlar. En iyi çözümler gelecek iterasyonlara kolayca aktarılabilir, elitizm olarak adlandırılan bu durum bu kategorideki algoritmaların bir avantajıdır. Genetik algoritma (Holland, 1992), parçacık sürüsü optimizasyonu (Eberhart & Kennedy, 1995) ve karınca kolonisi optimizasyonu (Dorigo vd., 1996) bu kategorinin birkaç örneğidir.

2.2.3 Popülasyon Hareketine Dayalı Sınıflandırma

Popülasyon hareketine dayalı sınıflandırma, ilham kaynağı ya da popülasyonun büyüklüğü yerine yöntemleri algoritmanın davranışına göre kategorize etmeye çalışır. Bir sonraki yinelemede popülasyonun nasıl güncellendiği, benzer türdeki algoritmaları anlamak ve farklı prensibe dayanan yöntemleri karşılaştırmak için iyi bir araçtır. Bu doğrultuda MS yöntemler diferansiyel vektör hareketine dayalı algoritmalar ve çözüm oluşturmaya dayalı algoritmalar olmak üzere iki alt sınıfta incelenebilir (Molina vd., 2020).

Diferansiyel vektör hareketi, yeni çözümlerin önceki bir çözüme yapılan bir kaydırma veya mutasyonla üretildiği bir durumdur. Parçacık sürü optimizasyonu algoritması, her çözümün arama alanında daha iyi çözümler tespit etmek için diferansiyel vektör hareketini kullandığı en temel yöntemlerden biridir, bu hareket bir hız vektörü ile temsil edilir ve birçok yönetime öncülük etmiştir. Bu yöntemlerin ilk

kategorisinde, popülasyon çözümlerinin hareketini oluşturmak için popülasyondaki tüm bireylerin kullanılır, tüm bireylerin diğer çözümlerin hareketi üzerinde bir etki derecesi vardır. Ateşböceği algoritması önemli bir örnek olarak gösterilebilir (X.-S. Yang, 2009). Bazı yöntemler, küçük bir temsili çözüm grubu belirleyerek tüm çözümlerin hareketini bu gruba göre belirler. Bu temsili çözümlerin, algoritma tarafından bulunan en iyi çözümler olarak seçilmesi sıklıkla karşılaşılan bir durumdur ve popülasyon en iyi birey yada bireyler tarafından yönlendirilir. Diferansiyel vektör hareketi temel alan en kalabalık sınıflandırma bu kategoridir. Yarasa esinli algoritma (X. S. Yang, 2010a), beyin fırtınası optimizasyon algoritması (Shi, 2011), çekirge optimizasyon algoritması (Saremi vd., 2017) bu yöntemlerden yalnızca birkaçıdır. Diferansiyel vektör hareketini temel alan son alt kategoride ise popülasyondaki çözümlerin bir alt kümesinin veya grubunun çözümlerini dikkate alır. Guguk kuşu optimizasyon algoritması (X.-S. Yang & Deb, 2009), Bakteriyel besin arama optimizasyonu (Liu & Passino, 2002) bu kategorinin örnekleri arasında yer alır.

Çözüm oluşturmaya dayalı algoritmalar tek bir referans çözüm yerine birçok çözümün birleştirilmesiyle veya benzer mekanizmalarla oluşturulur. Genetik algoritma (Holland, 1992), harmoni araması (Geem vd., 2001) gibi algoritmalar ilk kategoride yer alır, birkaç çözümün kombinasyonu veya çaprazlaması ile çözümlerini günceller. İkinci kategori ise dolaylı koordinasyonun olduğu stigmerjiyi kullanır, karınca kolonisi optimizasyonu (Dorigo vd., 1996), kültürel algoritma (Reynolds, 1994)s bu grubun örnekleridir.

2.2.4 Parametre Sayısına Dayalı Sınıflandırma

Parametreler, MS algoritmaların yapılandırılmasında kritik bir rol oynar ve algoritmanın performansı bu ayarlara bağlıdır. En iyi parametre değerlerini seçmek, karmaşık bir süreçtir ancak algoritmanın esnekliği ile sağlamlığı üzerinde etkilidir. Parametre değerleri, optimizasyon problemine özgüdür. Daha az parametre, ayarı basitleştirir fakat probleme uyarlanabilirliği azaltabilir. Bu nedenle MS algoritmaların popülasyon boyutu ve iterasyon dışındaki ortak olmayan parametre sayısına göre bir sınıflandırma çerçevesi, yöntemlerin performanslarının karşılaştırılması için önem taşır.

Literatürdeki MS algoritmalar parametrelerine göre incelendiğinde çoğunun 0 ile 5 arasında birincil parametresi olduğu görülmüştür. Sonuç olarak, algoritmalar 0 (birincil parametresi olmayan) (Cheng & Prayogo, 2014; Chou & Nguyen, 2020; R. V. Rao & Patel, 2012; X.-S. Yang, 2012), 1 parametrelili (Heidari vd., 2019; Mirjalili vd., 2017; X.-S. Yang & Deb, 2009), 2 parametrelili (Mirjalili, 2015a; Mirjalili vd., 2014; Saremi vd., 2017; Storn & Price, 2009), 3 parametrelili (Eskandar vd., 2012; Faramarzi vd., 2020; X.-S. Yang, 2009), 4 parametrelili (Hashim vd., 2021; Mirjalili, 2016b; Rashedi vd., 2009; Socha & Dorigo, 2008) ve 5 veya daha fazla parametreye sahip algoritmalar (Eberhart & Kennedy, 1995; Hashim vd., 2019; Simon, 2008) olmak üzere altı grupta incelenebilir.

2.3 MS Algoritmaların Yapısı

MS algoritmaların yapısı genel olarak başlatma, arama ve sonlandırma aşaması olmak üzere üç ana bölümde incelenebilir (Cuevas vd., 2021). Başlatma aşamasında, algoritma tüm giriş bilgilerini alır ve algoritmada tanımlanan parametreler doğrultusunda ajanlar arama uzayına rastgele olarak dağıtılır. Başlatma aşamasından sonra, algoritma arama sürecini yürütür ve bu aşama çözüm güncelleme veya yeniden üretme olarak da adlandırılabilir. Algoritmaların birbirinden farklılaştığı yer çoğunlukla bu aşamadır. MS algoritmalar, özgün bir arama süreci modellemelerine yardımcı olacak bir strateji oluşturmak için genellikle dış dünyaya bakarlar. Bu yinelemeli arama sürecinde model, ajanları amaç fonksiyonuna göre değerlendirerek en uygun çözüme yaklaşmasına yardımcı olur. Yani sonlandırma aşamasına kadar aynı strateji tekrarlamaya devam eder. Arama sürecinde, MS optimizasyon algoritmalarının keşif ve istismar aşaması olmak üzere iki farklı aşamadan geçmesi gerektiğini bilmek önemlidir. Keşif aşamasında, algoritma genellikle daha rastgele hareketlerle arama alanını keşfetme eğilimindedir çünkü arama süreci doğası gereği daha büyük adımlarla ve düzen bir şekilde gerçekleştirilir. Bu aşamadan sonra, algoritma arama alanının optimum çözümü barındırma olasılığının daha yüksek olduğu belirli bölgelere daha fazla odaklanacaktır. Bu, sömürü aşaması olarak da adlandırılan istismar aşamasının görevidir. Bu iki aşamada dengeli bir stratejiye sahip olmak, verimli ve kapsamlı bir arama süreci için çok önemlidir. Genellikle, algoritmalar keşif aşamasından istismar

aşamasına sorunsuz bir şekilde geçiş yapma eğilimindedir. MS yöntemlerin son adımında algoritmaların sonlandırma aşaması yer alır. Bu bir sonlandırma kriterinin karşılandığı anlamına gelir ve arama sürecinin sona erebileceğini gösterir. Böyle bir sonlandırma aşaması olmadan algoritmanın sonsuz bir döngüde tekrarlanacaktır. Aslında sonlandırma aşaması, algoritmanın optimum veya kabul edilebilir bir çözüme ulaşmış olup olmadığını belirler, bu nedenle oldukça önemlidir. Bazı yöntemler kendine özgü şekilde tasarlanmış sonlandırma kriterlerine sahip olsa da, çoğu MS algoritma benzer bir sonlandırma kriteri kullanır. Bu konuda en sık alıntılanan seçeneklerden bazıları yineleme sayısını, çalışma süresini sınırlamak veya ardışık yinelemeler arasındaki en iyi çözüme yapılan iyileştirmeyi izlemektir. Bu seçenekler arasında, yineleme sayısını sınırlamak, MS algoritmalar için bir sonlandırma aşaması oluşturmak için tartışmasız en bilinen mekanizmadır. Bu yaklaşım, algoritmanın önceden tanımlanmış prosedürlerinin, durmadan ve süreçte bulunan en iyi çözümü döndürmeden önce yalnızca belirli sayıda kez yürütülmesidir.

BÖLÜM ÜÇ

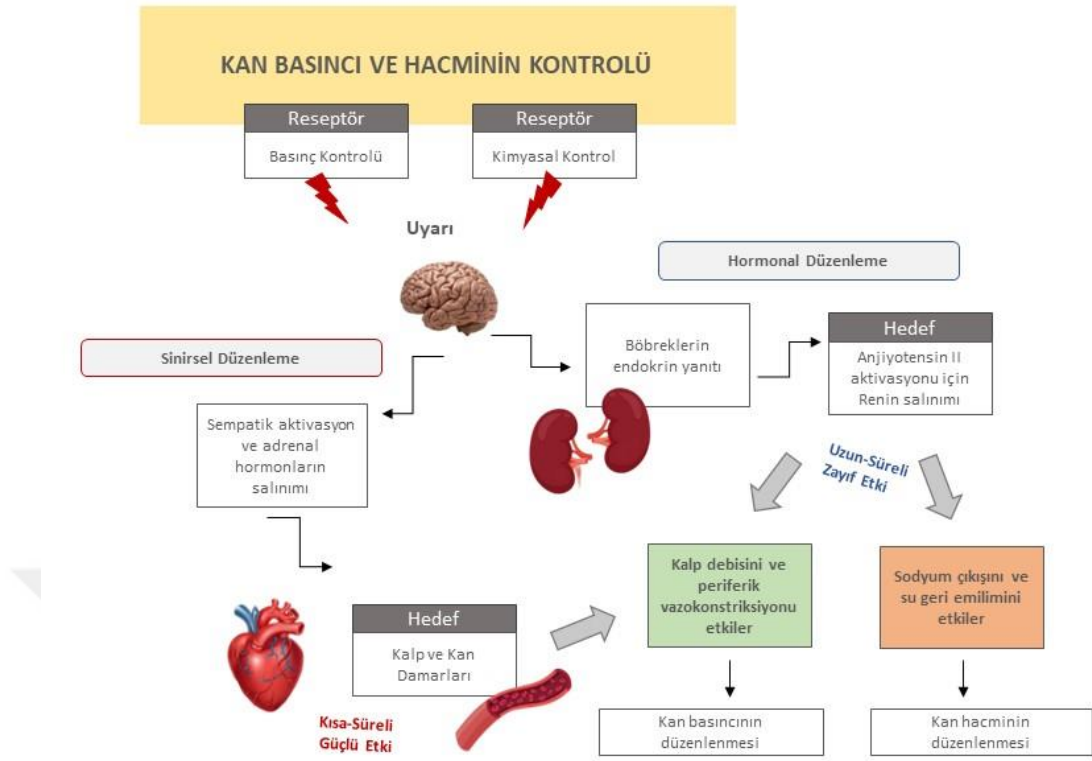
YAPAY KAN DOLAŞIM SİSTEMİ ALGORİTMASI

Bu bölümde tez kapsamında önerilen YKDS algoritması detaylı bir şekilde incelenmiştir. Bölüm 3.1 algoritmanın ilham kaynağını tanımlar. Bölüm 3.2 sürecin matematiksel olarak modellenmesine değinir. Bölüm 3.3 model parametreleri ve arama sürecinde etkin bir rol oynayan uyarım fonksiyonuna yer verir. Son olarak Bölüm 3.4 algoritmanın keşif-sömürü aşamalarındaki davranışları hakkında bilgi sağlar. Modelin matematiksel ifadelerini tanımlar.

3.1 İlham Kaynağı

Dolaşım sistemi, insan vücudundaki işleyişleri dengede tutmakla görevli temel sistemlerden biridir. Bu sistem, vücut için gerekli olan besin ve oksijeni doku ve hücrelere taşır; ihtiyaç halinde hormonları vücudun ilgili bölümüne ulaştırır. Bunun yanı sıra artık maddeleri hücrelerden uzaklaştırır. Dolaşımın sürekliliği canlılık için hayati önem taşır, bu nedenle kan akımı doku ve organların ihtiyacını karşılayacak şekilde düzenlenir. Dolaşımın işlevlerinin ayrıntıları karmaşık olmakla birlikte, tez kapsamında yapının modellenmesi için arteriyel basınç ve kan hacminin düzenlenmesine odaklanılmıştır.

Arter basıncı, diğer bir adıyla kan basıncı, kanın kalpten pompalanmasından itibaren başlayan süreçte kanın damarlara uyguladığı basınçtır. Vücuttaki kan miktarına, damar direncine, damarların çeperlerindeki esnekliğe ve diğer birçok faktöre bağlıdır. Kan basıncı, damar tipine, kişiye ve yaşa göre değişir hatta aynı kişi için dinlenme veya fiziksel bir aktivitede olma durumuna göre farklılık gösterir. Özetle, bu hayati değer vücutta bulunan çeşitli türdeki reseptörler tarafından kontrol edilir ve anlık olarak düzenlenir. Bu karmaşık kontrol sürecinde birçok faktör etkilidir, ancak temel olarak iki sistemin bu düzenleme sürecinde rol aldığı söylenebilir. Şekil 3.1, insan vücudundaki kan basıncını düzenleyen sistemleri tasvir ederek algoritmanın çalışma prensiplerine ilişkin fikir verir.



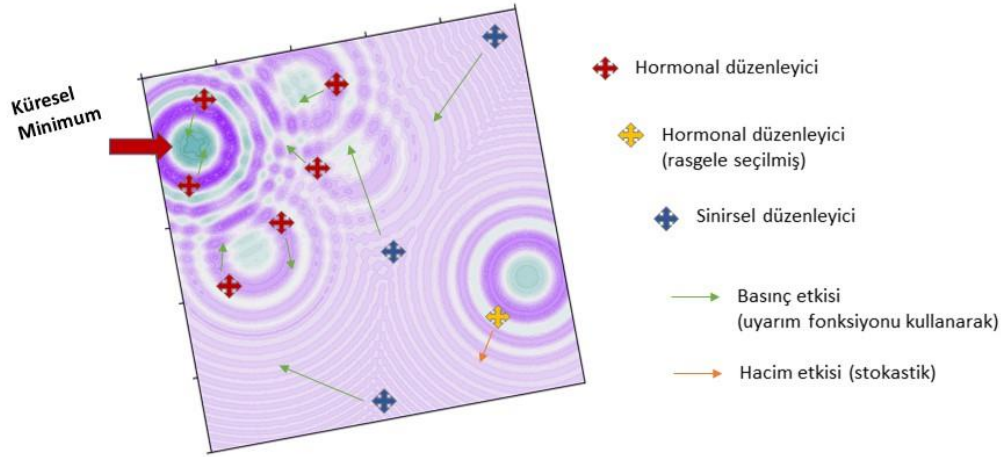
Şekil 3.1 Dolaşım sisteminin düzenlenmesi

Kan basıncında ani bir değişiklik ihtiyacı olduğunda ilk olarak sinirsel kontrol devreye girer. Otonom sinir sistemi, damar çevresindeki sinir liflerini uyararak gereksinime göre damarların hacmini azaltır veya artırır. Sinirsel kontrol mekanizmaları, ciddi bir düzenleme sağlar ancak birlikte çalıştığı baro reseptörlerin zamanla inaktif olması nedeniyle uzun süreli bir etki yaratmaz. Dolaşımın normal düzeyine ulaşması için yeterli değildir. Bu aşamada, uzun süreli kontrol mekanizması olan hormonal sistem devreye girer. Hormonların salgılanma süreci, taşınması ve belirli bir seviyeye ulaşması gereksiniminden dolayı hormonal düzenleme sinirsel kontrole kıyasla daha yavaştır. Hormonal düzenleme, damarları uyarıcı kimyasal etkinin yanı sıra böbrekteki yerel kontrol sistemlerini etkileyerek vücuttaki su ve tuz geri emilimini kontrol eden ikinci bir işleve sahiptir. Gerektiğinde, tuz ve suyun kan dolaşımına emilimini kolaylaştırır, böylece vücuttaki kan hacmini artırır. Aksini gerektiren durumlarda, böbreklerden suyun dışarı akışını teşvik ederek kan miktarını azaltır. Sonuç olarak, kan hacmini düzenleyerek dolaylı yoldan kan basıncını dengeler. Hormonal düzenleme, nispeten daha yavaş ancak son derece hassas bir ayarlama

sürecidir. Sinir sisteminin aksine, düzenleme sürecinin tamamlanması haftalar veya aylar boyunca sürebilir.

3.2 Düzenleyicilerin Modellemesi

MS yöntemlerde en önemli aşama ilham kaynağının modellenmesidir. Bu matematiksel veya mantıksal ifadeler, rastgele başlatılan aday çözümlerin konumlarının güncellenmesi ve iyileştirilmiş çözümler üretilmesi için gereklidir. YKDS algoritması, bir optimizasyon sorununu optimize etmek için ilhama benzer nörolojik ve hormonal düzenleyici sistemler kullanır. Şekil 3.2, temel bir minimizasyon problemi üzerinde rastgele dağıtılan düzenleyicileri göstermekte ve davranışlarını simüle etmektedir.



Şekil 3.2 Bir arama uzayında düzenleyici davranışlarının gösterimi

Başlangıçta arama alanına rastgele yerleştirilen düzenleyiciler, her iterasyonda optimum değere ulaşmak için güncellenir. Optimizasyon prosedürü üç farklı teknik kullanılarak gerçekleştirilir. İlk iki mekanizmada sinirsel düzenleyiciler ve bazı hormonal düzenleyiciler, kan basıncını düzenlemek için bir uyarım fonksiyonu kullanır. Hormonal ve sinirsel düzenleyiciler, aynı matematiksel ifadeyi kullanmalarına rağmen kendilerine özgü parametreleri nedeniyle farklı etki şiddetine sahip olur. Sinirsel düzenleyiciler en kötü çözümlerden kaçınmak ve arama uzayını

keşfetmek için önemli sıçramalar yaparak güçlü bir eğilim sergiler. Hormonal düzenleyiciler ise daha zayıf bir etkiyle en iyi aday çözümün çevresini keşfederek iyileştirilmiş çözümleri takip etmeye devam eder. Üçüncü mekanizma, önceki bölümlerde tartışılan kan hacminin düzenlenmesi metaforunu taklit eden bazı hormonal düzenleyicilerin faaliyetini içerir. Bu düzenleyiciler uyarım işlevini kullanmazlar, bunun yerine kendi konum bilgilerini kullanarak sinirsel düzenleyiciler tarafından sağlanan daha avantajlı çözümleri geliştirmeye çalışırlar. Algoritma, optimum çözümlere yaklaşmak için bu mekanizmaların yinelemeli olarak tekrarlanmasını temel alır. Bu kavramların davranışlarına ait matematiksel ifadeler ve algoritmada kullanılan parametreler ilerleyen bölümlerde ayrıntılı olarak incelenmektedir.

3.3 Model Parametreleri ve Uyarım Fonksiyonu

Parametre belirleme, optimizasyon algoritmalarının tasarlanmasında kritik bir aşamadır. Uygun parametreler kullanılmazsa optimizasyon modeli istenmeyen davranışlar sergileyebilir ve bu da optimizasyon performansı üzerinde olumsuz bir etki yaratabilir. Temel olarak iki tip parametre vardır: ayarlanabilir parametreler ve sabit parametreler. Ayarlanabilir parametreler belirli bir aralıkta kullanıcının seçme özgürlüğüne sahipken, sabit parametreler teorik bilgilerden veya deneysel hesaplamalardan elde edilen önceden belirlenmiş sabit değerlerdir. Sabit parametre yöntemi optimizasyon sürecinde kullanıcıya söz hakkı tanımaz ve her probleme uygulamada başarılı olamayabilir. Ayarlanabilir parametreler ise probleme ayarlanabilirlik açısından esneklik sağlar, ancak parametreleri yapılandırma süreci zaman alıcı olabilir.

YKDS algoritmasında düzenleyicilerin sinirsel ve kimyasal uyarılmasını ifade etmek amacıyla bir formülasyon kullanılmıştır. Bu matematiksel ifade sigmoid fonksiyona benzer (Dombi & Jónás, 2022), ancak bu fonksiyonun genişletilmiş halidir. Uyarım fonksiyonu (UF) olarak adlandırılan (Denklem 3.1) bu matematiksel ifade esin kaynağına benzer şekilde üç farklı parametreden oluşur. E_m etki miktarını ve E_s ise etki süresini temsil eder. O_f olarak gösterilen üçüncü parametre, etki miktarının hesaplanması sırasında arama uzayına uygulanan ölçeklendirme faktörüdür.

Algoritmanın son parametresi olan ***SH_{orani}*** uyarım fonksiyonunda kullanılmaz, ancak algoritmada arama uzayında sinirsel ve hormonal düzenleyicilerin sayıca bulunma oranını temsil eder.

$$UF(X_i) = \frac{(us-as) * O_f}{1 + e^{(E_m * (-X_i + E_s))}} \quad (3.1)$$

burada ***X_i***, ***i***. düzenleyicinin konumunu tanımlar, ***as*** ve ***us*** sırasıyla problemin alt ve üst sınır değerleridir. ***O_f*** ölçekleme faktörüdür. ***E_m*** düzenleyicinin türüne bağlı olarak etki miktarını ve ***E_s*** etki süresini temsil eder.

E_m ve ***E_s*** sabit parametrelerdir, düzenleyicinin tipine göre sinirsel ve hormonal olacak şekilde aldıkları değerler birbirinden farklıdır. Aşağıda detayları verilen hayvan deneylerinden elde edilen bulgular kullanılarak belirlenmiştir:

Bir çalışmada (J. H. Hall, 2011), spinal anestezi ile sinir uyarılarının geçişi engellenen bir hayvan deneyinde arter basıncının 100 milimetre cıvadan 50 milimetre cıvaya düştüğü görülmüştür. Bir süre sonra sinir sistemini uyarıcı bir maddenin enjekte edilmesiyle damarlarda kasılma görülmüş, kan basıncı 1-3 dakika içerisinde normal değerinin de üzerine çıkmıştır. Çalışma maddenin aktifleşmesinden sonra etkinin saniyeler içerisinde başladığı ve sıklıkla 5-10 saniye içerisinde basıncı normal değerlerin iki katına kadar arttırdığını göstermektedir. Bir başka çalışmada (J. H. . Hall, 2011) ise herhangi bir kanama durumunda Renin-Anjiyotensin sisteminin arter basıncı üzerine olan etkisini inceleyen tipik bir deney ele alınmıştır. Hormonal sistemin damarlardaki vazokonstriksiyon etkisi sayesinde kalbe venöz dönüşü arttırdığını ve 50 milimetre cıvaya düşen kan basıncının 83 milimetre cıvaya kadar çıkarılabildiği görülmektedir. Bu sayısal bilgilerden yola çıkarak kan basıncındaki değişim miktarı (değişim miktarı/başlangıç değeri) olan ***E_m*** ve değişimin gerçekleşmesi gereken zaman (1/saat) olan ***E_s*** değerleri hesaplanmıştır.

SH_{orani} ve ***O_f*** parametreleri ise diğer iki parametreden farklı olarak ayarlanabilir parametrelerdir, problemin türüne göre belirlenebilir ve esneklik imkanı sunar. Tablo

3.1, E_m ve E_s parametreleri için sabit değerleri ve $SH_{oranı}$ ve O_f parametreleri için kullanılması uygun görülen değer aralıklarını göstermektedir.

Tablo 3.1 YKDS algoritmasının parametreleri

Tanım	Parametre	Parametre tipi	Düzenleyici	Değer/Değer aralığı
Etki miktarı	E_m	Sabit	Sinirsel düzenleyici	1,8
			Hormonal düzenleyici	0,66
Etki süresi	E_s	Sabit	Sinirsel düzenleyici	0,002
			Hormonal düzenleyici	0,33
Ölçekleme faktörü	O_f	Ayarlanabilir	Tümü	0,1-0.9
Düzenleyicilerin sayıca oranı	$SH_{oranı}$	Ayarlanabilir	Tümü	0,01-0,5

3.4 Keşif ve Sömürü Aşaması

Doğadan ilham alan MS algoritmaların çoğu, optimizasyon problemlerinin çözümünde arama uzayını rastgele bir popülasyon ile başlatır. Bu arama süreci, mantıksal olarak keşif ve sömürü eğilimleri ile gerçekleştirilir. Bölüm 3.2’de bahsedildiği düzenleyici olarak adlandırılan aday çözümler, sinirsel ve hormonal olmak üzere iki farklı grup olarak hareket eder. Sinirsel düzenleyiciler daha düzensiz ve büyük sıçramalar ile arama uzayını keşfederken hormonal düzenleyiciler en iyi çözümlere yakın olası iyi çözümleri sömürür. Popülasyonun konumu, aşağıda ana hatlarıyla belirtildiği gibi, düzenleyici türüne göre iki kategoride güncellenir:

Sinirsel düzenleyiciler esin kaynağına benzer davranış sergiler, daha büyük bir uyarılma ile arama uzayını tarar. Bu düzenleyici tipi, amaç fonksiyonuna göre en kötü olan aday çözümleri benzer tipteki düzenleyicilerin ortalama değerine bağlı kalacak şekilde popülasyonun o anki en iyi aday çözümüne ulaştırmaya çalışır. Amacı, en kötü çözümlerin çevresini daha geniş bir alanda tarayarak daha iyi aday çözümleri bulmaktır. Sinirsel düzenleyicilerin konum güncellemesi, Denklem 3.2’deki matematiksel hesaplama ile ifade edilir.

$$X_{i+1} = r_1 * UF(X_i) \quad (3.2)$$

burada X_{i+1} , düzenleyicinin yeni konumunu tanımlar. X_i , i . düzenleyicinin konumudur; r_1 , 0 ile 1 arasında rastgele bir sayıdır ve UF , uyarım fonksiyonunu temsil eder.

Hormonal düzenleyiciler de esin kaynağına benzer davranış gösterir, sinirsel kontrol ile düzenlenmiş olan sistemi daha iyi ve daha kararlı hale getirme özelliğine sahiptir. Aday çözümleri sinirsel düzenleyicilere kıyasla daha iyi hedef değerine sahiptir. İyi çözümlerin çevresini daha küçük adımlarla tarayarak daha iyi çözümleri bulmaya çalışırlar. İki farklı işlevde görev alırlar, bunlardan ilki sinirsel düzenleyicilere benzer şekilde uyarılma fonksiyonunu kullanır. Hormonal düzenleyicilerin ortalama değerine bağlı kalarak popülasyonun o anki en iyi aday çözümüne ulaşmaya çalışır. Diğer işlev ise aday çözümün bir rastgeleliğe bağlı olarak popülasyon ortalamasına yaklaşmasıdır. Hormonal düzenleyicilerin konum güncellemesi sırasıyla Denklem 3.3 ve 3.4'deki matematiksel hesaplamalar ile ifade edilir. Düzenleyicilerin hangi işleve sahip olacağı rastgeleliğe bağlı bir süreç ile yönetilir.

$$X_{i+1} = X_{ort} + r_2 * UF(X_i) \quad (3.3)$$

$$X_{i+1} = X_{iyi} + r_3 * (X_{ort} - X_i) \quad (3.4)$$

burada X_{i+1} , düzenleyicinin yeni konumunu; X_i , i . düzenleyicinin konumunu tanımlar. X_{iyi} ve X_{ort} sırasıyla popülasyonun en iyi aday çözümünün konumu ve popülasyonun konumlarının ortalama vektörüdür. UF , uyarım fonksiyonunu temsil eder; r_2 ve r_3 , 0 ile 1 arasında rastgele bir sayılardır.

YKDS algoritması, daha önce de belirtildiği gibi, çeşitli düzenleme mekanizmaları kullanarak dolaşım sisteminin kontrolünden esinlenen bir optimizasyon algoritmasıdır. Yönteme ait akış şeması Şekil 3.3'te gösterilmiştir ve Tablo 3.2'de gösterildiği gibi sözde kod kullanılarak ifade edilebilir. Ayrıca algoritmaya ve çalışmaya ait tüm kodlara halka açık github hesabımızdan ulaşılabilir (Özcan & Utku, 2024). YKDS algoritması, diğer MS yaklaşımlarına benzer şekilde rastgele bir başlangıç popülasyonu oluşturarak başlar. Her bir düzenleyici tipi, matematiksel

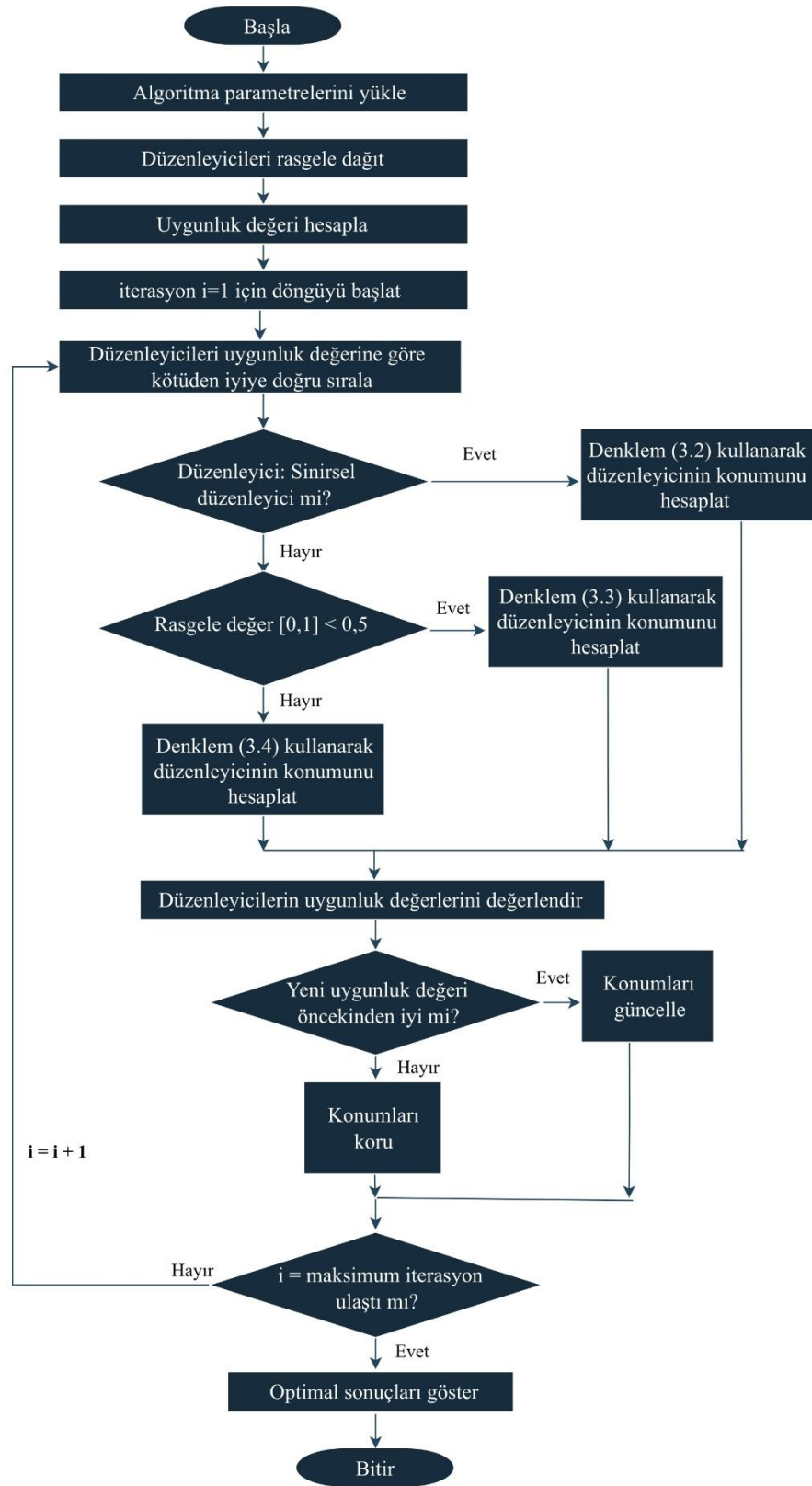
hesaplamalar ve parametreler kullanarak aday çözümlerini amaç fonksiyonuna göre iteratif olarak geliştirir. Çözüm güncellemelerinin her iterasyonunun ardından algoritma, çözümlerin arama uzayının sınırları içinde kaldığını garanti etmek için popülasyonun konumlarını doğrular. Belirlenen bölgenin dışında bir çözüm tespit edilirse, tanımlanan arama alanı içinde kalacak şekilde sınırlandırılır. Her iterasyon sırasında, düzenleyicilerin uygunluk değerleri hedef fonksiyona göre hesaplanır. En yüksek uygunluk değerine sahip düzenleyici daha sonra popülasyon içindeki en iyi çözüm olarak tanımlanır.

Tablo 3.2 YKDS algoritmasının sözde kodu

```

YKDS algoritmasının parametrelerini yükle
Düzenleyicileri başlat
Düzenleyicilerin uygunluk değerlerini hesapla
while iterasyon < maksimum_iterasyon_sayısı
    Düzenleyicileri uygunluk değerine göre kötüden iyiye doğru sırala.
    For indeks to düzenleyici_sayısı:
        sinirsel_düzenleyici = popülasyon_boyutu *  $SH_{oranı}$ 
        If indeks < sinirsel_düzenleyici:
            Denklem (3.2) kullanarak sinirsel düzenleyicilerin konumlarını hesapla ve alt-üst sınırlar içinde olduğunu kontrol et.
        else:
            [0,1] arasında rastgele bir sayı ata.
            If sayı < 0,5:
                Denklem (3.3) kullanarak hormonal düzenleyicilerin konumlarını hesapla ve alt-üst sınırlar içinde olduğunu kontrol et.
            else:
                Denklem (3.4) kullanarak hormonal düzenleyicilerin konumlarını hesapla ve alt-üst sınırlar içinde olduğunu kontrol et.
            end if
        end if
        If düzenleyicinin yeni uygunluk değeri (indeks) düzenleyicinin yeni uygunluk değerinden (indeks) daha iyiye:
            Düzenleyicinin konumunu güncelle.
        else:
            Düzenleyicinin konumunu koru.
        end if
    end for
    iterasyon = iterasyon + 1
end while

```



Şekil 3.3 YKDS algoritmasına ait akış şeması

BÖLÜM DÖRT

DENEYSEL TASARIM

Bu bölümde, çalışmanın deneysel ortam ve süreci hakkında detaylara yer verilmiştir. Deneysel ortam ve prosedürler Bölüm 4.1’de açıklanmıştır. Deneysel süreç iki aşamada ele alınmaktadır. İlk aşama modelin kıyaslama fonksiyonları ile test sürecini içerir, detaylara Bölüm 4.2’den ulaşılabilir. İkinci aşamada ise, algoritmanın performansı altı mühendislik tasarım problemini ele alarak incelenmiştir, problemlere ait detaylara Bölüm 4.3’te yer verilmiştir. Deneysel süreçlerden elde edilen sonuçlar farklı değerlendirme kriterleri kullanılarak incelenmiştir. Değerlendirme kriterlerine Bölüm 4.4’ten ulaşılabilir.

4.1 Deneysel Prosedür

Önerilen YKDS algoritmasının tasarımı, deneysel prosedürlerin gerçekleştirilmesi ve tüm simülasyonlar için Tablo 4.1’de gösterilen donanım ve yazılımlara sahip bir bilgisayar kullanılmıştır.

Tablo 4.1 Deneysel arka planın detayları

Tanım	Düzenleyici	Değer/Değer aralığı
Donanım	İşlemci	Intel Core(TM) i5-12400 2.50 GHz
	Depolama	512 GB
	Ram	16 GB
	Ekran kartı	Intel (R) UHD 730
Yazılım	İşletim sistemi	Windows 11
	Programlama dili	Python (Versiyon 3.10)
	Kütüphane	Mealpy, Opfunu, Enoppy, ve diğer temel kütüphaneler

Önerilen algoritma temel Python kütüphanelerinin yanı sıra Mealpy (Van Thieu & Mirjalili, 2023) kütüphanesinin altyapısı kullanılarak kodlanmıştır. Test aşamasında kıyaslama fonksiyonlarının yapılandırılması için Opfunu (Thieu, 2024) kütüphanesinden yararlanılmıştır. Mühendislik uygulamaları aşamasında ise literatürde kullanılan bir çok mühendislik optimizasyon problemini derleyen Enoppy (Van Thieu, 2023) kütüphanesi kullanılmıştır. Karşılaştırma aşamasında her algoritma

için parametre konfigürasyonları, orijinal yayınlarda önerilen ve kullanılan kütüphanede varsayılan ayarlar olarak belirtilen değerlere göre oluşturulmuştur. Daha fazla bilgi için Mealpy (Van Thieu & Mirjalili, 2023) kütüphanesine erişilebilir. Tüm algoritmalar için ortak parametreler olan maksimum iterasyon sayısı ve popülasyon boyutu sırasıyla 50 ve 100'dür ve her bir çalıştırmada sabittir. Deneysel süreçte yazılım kaynaklı bir üstünlük ihtimalini minimuma indirmek için tüm deneyler eşit şartlarda gerçekleştirilmiştir. Sonuçların güvenilirliğinden emin olmak ve istatistiksel açıdan anlamlı sonuçlar elde edebilmek için önerilen yöntem de dâhil olmak üzere her bir algoritma 25 kez çalıştırılmıştır.

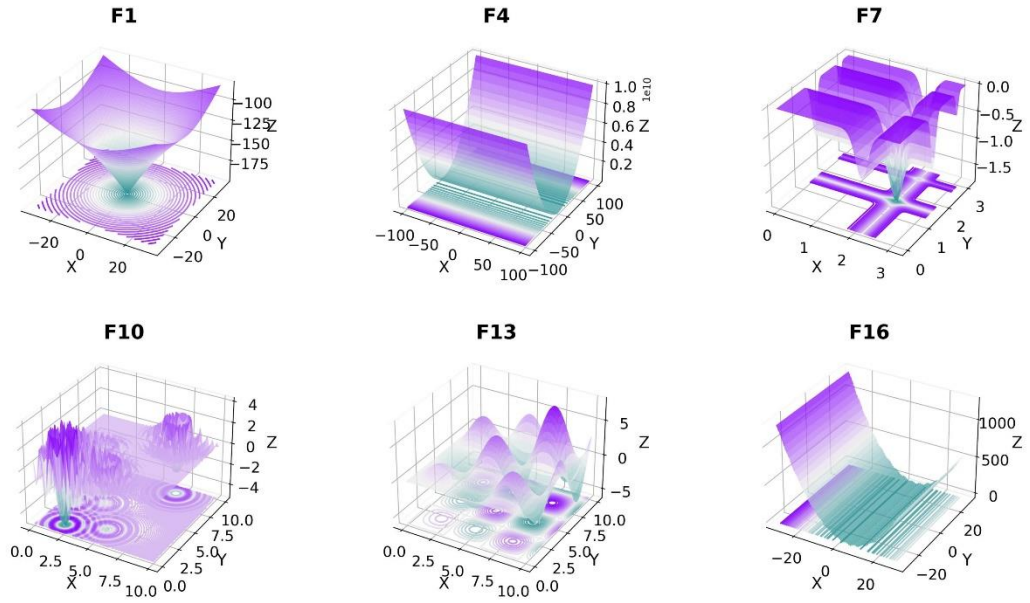
4.2 Test Fonksiyonları

Optimum değeri bilinen matematiksel fonksiyonlar, MS algoritmaların test edilmesinde yaygın bir şekilde kullanılmaktadır. Bununla birlikte, farklı özelliklerdeki test fonksiyonlarının kullanılması, yöntemlerin karakteristikleri hakkında önemli bilgiler sağlar. Örneğin, tek modlu bir fonksiyonun yerel çözümü yoktur ve yalnızca bir küresel çözümü vardır. Bu tür fonksiyonlar, yöntemlerin yakınsama hızları hakkında bilgi sahibi olunmasını sağlar. Çok modlu fonksiyonlar ise birden fazla yerel çözüme sahip olduğundan, modellerin yerel optimumlara takıldığını veya kaçındığını gösterir. Diğer açıdan, iki veya daha fazla ilişkili değişkene bağlı olduğunda arama süreci tek değişkene kıyasla daha zordur. Bu durum, ayrılamayan fonksiyonların optimum çözümünü elde etmesini ayrılabilenlere kıyasla zorlaştırır. Ayrıca matematiksel fonksiyonların konveks olup olmadığı, sürekliliği gibi birçok özelliğinden bahsedilebilir. Kısacası her bir modelin eksik ya da başarılı olan yönlerinin analiz edilmesinde test fonksiyonları oldukça önemlidir. Bu nedenle, çalışmanın test aşamasında önerilen algoritmanın performansını kıyaslamak için farklı özelliklere sahip iki set kıyaslama fonksiyonu kullanılmıştır.

4.2.1 Klasik Test Fonksiyonları

İlk sette literatürde iyi bilinen bazı algoritmaların karşılaştırması için sıklıkla tercih edilen klasik kıyaslama fonksiyonlarından 16 fonksiyon seçilmiştir. Bu fonksiyonlar

bir önceki bölümde belirtilen modalite, ayrılabilirlik, konvekslik ve süreklilik gibi özellikler dikkate alınarak belirlenmiştir. Klasik kıyaslama işlevleri çalışmada iki boyutlu olarak uygulanmış olup optimizasyon problemlerine bir bakış açısı sunabilmek için rastgele seçilmiş altı fonksiyona ait grafiksel gösterimleri Şekil 4.1’de sunulmuştur.



Şekil 4.1 Bazı klasik test fonksiyonlarının grafiksel gösterimleri

Test aşamasında kullanılan klasik kıyaslama fonksiyonlarına ait temel bilgiler Tablo 4.2’de verilmiştir. Fonksiyonların amaç fonksiyonlarına ait matematiksel fonksiyonlar Ek 1’de gösterilmiştir. Diğer detaylara klasik kıyaslama fonksiyonlarını derleyen çalışmalardan ulaşılabilir (Jamil & Yang, 2013), ayrıca daha fazla kıyaslama fonksiyonunu incelemek için Opfunu (Thieu, 2024) kütüphanesi ziyaret edilebilir.

Tablo 4.2 Klasik test fonksiyonlarının genel özellikleri

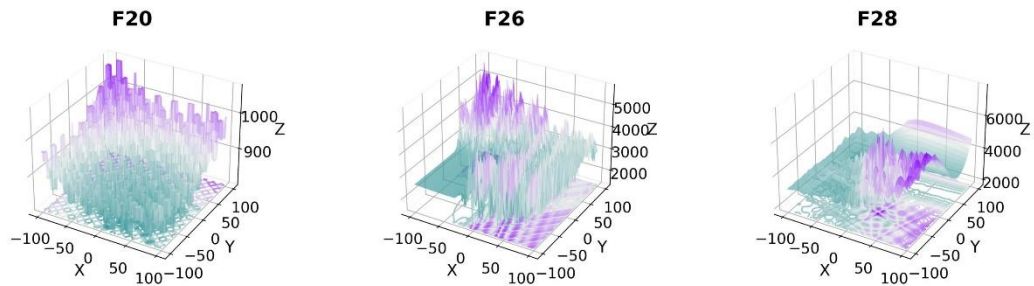
Kod	Fonksiyon adı	Minimum noktası	Tanımlı değer aralığı	Boyut
F1	Ackley 2	-200,00	$[-32; 32]^2$	2
F2	Brent	0,00	$[-10; 10]^2$	2
F3	Chung Reynolds	0,00	$[-100; 100]^2$	2
F4	Cigar	0,00	$[-100; 100]^2$	2
F5	Matyas	0,00	$[-10; 10]^2$	2
F6	Leon	0,00	$[-1,2; 1,2]^2$	2

Tablo 4.2 Klasik test fonksiyonlarının genel özellikleri (devamı)

F7	Michalewicz	-1,80	$[0, \pi]^2$	2
F8	CrossInTray	-2,06	$[-10; 10]^2$	2
F9	Hosaki	-2,34	$[0; 6]^2$	2
F10	Langermann	-5,16	$[0; 10]^2$	2
F11	Levy 5	-176,13	$[-10; 10]^2$	2
F12	Mishra 5	-1,01	$[-10; 10]^2$	2
F13	Alpine 2	-6,12	$[0; 10]^2$	2
F14	Hansen	-176,54	$[-10; 10]^2$	2
F15	Himmelblau	0,00	$[-5; 5]^2$	2
F16	Chichinadze	-42,94	$[-30; 30]^2$	2

4.2.2 Bileşik Test Fonksiyonları

Test aşamasında kullanılan ikinci fonksiyon grubu ise araştırmacılar tarafından gerçek optimizasyon problemlerinin daha iyi taklit etmek amacıyla üretilen ve klasik fonksiyonlara kıyasla daha zorlu problemler olarak kabul edilen bileşik fonksiyonlardır. Bileşik test fonksiyonları klasik kıyaslama fonksiyonlarının kaydırılarak, rotasyona uğratarak veya birden fazla fonksiyonun hibritleştirilmesiyle oluşturulmaktadır. Ayrıca farklı boyutlara uyarlanabilmesi sayesinde yöntemlerin çok boyutlu problemlerle baş edebilirliğini incelemek açısından önem taşımaktadır. Bileşik kıyaslama işlevleri çalışmada on boyutlu olarak uygulanmıştır. Bu türdeki optimizasyon problemlerine bir bakış açısı sunabilmek için iki boyutlu olarak uyarlanabilenler arasından rastgele seçilmiş üç fonksiyona ait grafiksel gösterimleri Şekil 4.2’de sunulmuştur.



Şekil 4.2 Bazı bileşik test fonksiyonlarının grafiksel gösterimleri

Evrimsel hesaplama kongresi kapsamında çeşitli yıllarda CEC koduyla yayımlanan ve birçok araştırmacının ilgisini çeken bileşik kıyaslama fonksiyonlarından oluşan bu set 2022 yılına ait olup 12 adet kıyaslama işlevini içermektedir, fonksiyonların özellikleri Tablo 4.3'te gösterilmektedir. Fonksiyonlara ait matematiksel ifadeler Ek 2'de yer verilmiştir.

CEC2022 fonksiyonlarını tanıtan teknik rapordan diğer detaylara ulaşılabilir (Kumar vd., 2022), ayrıca farklı yıllarda yayımlanan diğer CEC kıyaslama fonksiyonlarını incelemek için Opfunu (Thieu, 2024) kütüphanesi ziyaret edilebilir.

Tablo 4.3 Bileşik test fonksiyonlarının genel özellikleri

Kod	Fonksiyon adı	Minimum noktası	Bias	Tanımlı değer aralığı	Boyut (D)
F17	CEC2022-1	0,00	300	$[-100,100]^D$	$[2, 10 \text{ veya } 20]$
F18	CEC2022-2	0,00	400	$[-100,100]^D$	$2 \leq D \leq 100$
F19	CEC2022-3	0,00	600	$[-100,100]^D$	$2 \leq D \leq 100$
F20	CEC2022-4	0,00	800	$[-100,100]^D$	$2 \leq D \leq 100$
F21	CEC2022-5	0,00	900	$[-100,100]^D$	$2 \leq D \leq 100$
F22	CEC2022-6	0,00	1800	$[-100,100]^D$	$[10 \text{ veya } 20]$
F23	CEC2022-7	0,00	2000	$[-100,100]^D$	$[10 \text{ veya } 20]$
F24	CEC2022-8	0,00	2200	$[-100,100]^D$	$[10 \text{ veya } 20]$
F25	CEC2022-9	0,00	2300	$[-100,100]^D$	$[2, 10 \text{ veya } 20]$
F26	CEC2022-10	0,00	2400	$[-100,100]^D$	$[2, 10 \text{ veya } 20]$
F27	CEC2022-11	0,00	2600	$[-100,100]^D$	$[2, 10 \text{ veya } 20]$
F28	CEC2022-12	0,00	2700	$[-100,100]^D$	$[2, 10 \text{ veya } 20]$

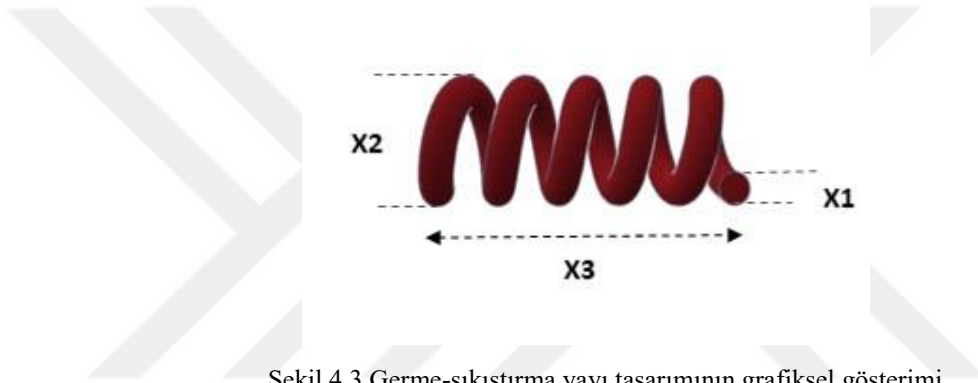
4.3 Mühendislik Uygulamaları

Mühendislik tasarım problemleri, kinematik, geometrik koşullar ve malzeme direnciyle ilgili çeşitli doğrusal olmayan kısıtlamaları içeren karmaşık değişkenlere sahip gerçek hayat problemleridir. Son otuz yılda, çeşitli mühendislik uygulamalarındaki zorlukları ele almak için çok sayıda çalışmada MS yöntemler kullanılmış ve birçok mühendislik tasarım problemi tanımlanmıştır. Tez kapsamında, literatürdeki optimizasyon yöntemlerinin karşılaştırmalı bir analizini yürütmek için altı yaygın kısıtlı mühendislik tasarım problemi ele alınmıştır. Problemlere ait detaylar alt başlıklarda sunulmuştur. Uygunluk fonksiyonunun, kısıtlamaların ve değişken aralığının matematiksel formülasyonları bir araştırma grubu tarafından ele alınan

dışbükey olmayan 57 kısıtlı optimizasyon problemlerinin derlendiği çalışmadan ulaşılabilir (Kumar vd., 2020).

4.3.1 P1: Germe-Sıkıştırma Yayı Tasarımı

Bu problem, Şekil 4.3'de gösterildiği gibi üç değişkene sahip bir yay tasarımına odaklanır. Değişkenlerden X_1 telin çapını, X_2 bobin çapını ve X_3 bobinlerin sarım sayısı temsil eder. Problemin amacı, minimum ağırlığa sahip bir germe-sıkıştırma yayı tasarlamak için değişkenlerin optimum değerlerini belirlemektir.

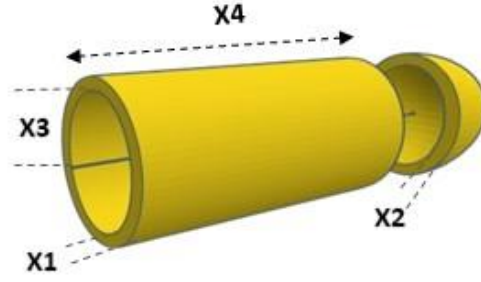


Şekil 4.3 Germe-sıkıştırma yayı tasarımının grafiksel gösterimi

Probleme ait amaç fonksiyonu, amaç fonksiyonunu sınırlandıran dört kısıtlama ve değişkenlerin değer aralıklarına ait bilgiler de detaylı olarak formüle edilmiştir. Probleme ait tüm matematiksel formülasyonlara Ek 3'ten ulaşılabilir.

4.3.2 P2: Basınçlı Kap Tasarımı

Bu problem (Şekil 4.4) uçları kapatılmış ve yarım küre formuna sahip başlığı olan silindirik bir basınçlı kabın tasarımını ele alır. Bu problemde X_1 kabuk kalınlığı, X_2 baş kalınlığı, X_3 iç yarıçap ve X_4 kabın uzunluğu olmak üzere dört karar değişkeni bulunur. Problemin amacı, malzeme ve kaynaklama prosedürü göz önünde bulundurularak genel maliyeti en aza indirmektir. Problem ayrıca dört kısıtlamayı da içerir.

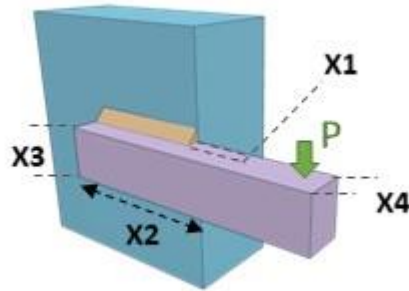


Şekil 4.4 Basınçlı kap tasarımının grafiksel gösterimi

Basınçlı kap tasarımı probleminin amaç fonksiyonu, amaç fonksiyonunu sınırlandıran dört kısıtlama ve değişkenlerin değer aralıklarını içeren bu problemin matematiksel ifadelerine Ek 4’te yer verilmiştir.

4.3.3 P3: Kaynaklı Kiriş Tasarımı

Bu problem kesme gerilimi, burkulma yükü, eğilme gerilimi, kirişin uç sapması ve yanal kısıtlamalar gibi sınırlamalar altında Şekil 4.5’te gösterildiği gibi bir kaynaklı kiriş tasarımına odaklanır. Problem X_1 kaynak kalınlığı, X_2 çubuğun bağlı kısmının uzunluğu, X_3 çubuğun yüksekliği ve X_4 çubuğun kalınlığı olmak üzere dört karar değişkenine sahiptir. Problemin amacı, üretim maliyetini azaltmak için en uygun girdileri belirlemektir.

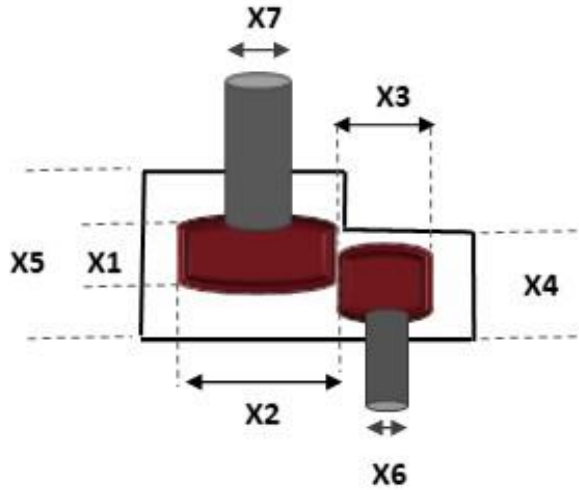


Şekil 4.5 Kaynaklı kiriş tasarımının grafiksel gösterimi

Kaynaklı kiriş tasarım problemine ait amaç fonksiyonu, probleme ait beş kısıtlama, teknik bilgilerden kaynaklanan sabitler ve formülasyonlar, değişkenlerin değer aralıkları gibi diğer detaylara Ek 5'te incelenebilir.

4.3.4 P4: Hız Düşürücü Tasarımı

Bir hız düşürücünün tasarımı, sistemin işlevselliğini optimize etmek, sürtünmeyi en aza indirmek ve ömrünü uzatmak için diş sayısını, konfigürasyonu, yarıçapı ve çeşitli dişli parametrelerini kontrol etmeyi gerektirir. Şekil 4.6'de verilen problemin ve değişkenlerinin şematik bir gösterimine yer verilmiştir. Bu problemde yedi değişken tanımlamaktadır: X_1 dişli genişliği, X_2 dişli modülü, X_3 dişli diş sayısı, X_4 rulmanlar arasındaki birinci eksenin uzunluğu, X_5 rulmanlar arasındaki ikinci eksenin uzunluğu, X_6 birinci eksenin çapı ve X_7 ikinci eksenin çapı. Problem hız düşürücünün ağırlığının en aza indirgenmesini sağlayan girdilerin tespit edilmesini amaçlar.

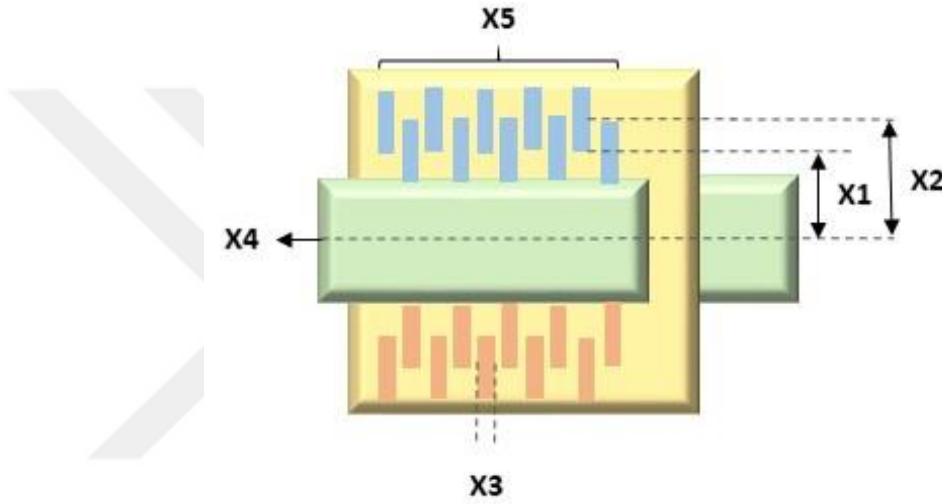


Şekil 4.6 Hız düşürücü tasarımının grafiksel gösterimi

Hız düşürücü tasarımı probleminin uygunluk fonksiyonu, probleme ait on bir teknik kısıtlama ve yedi değişkene ait değer aralıklarını içeren matematiksel formülasyonlar Ek 6'da gösterilmiştir.

4.3.5 P5: Çoklu Disk Debriyaj Freni Tasarımı

Çoklu disk debriyaj freni problemi otomotiv mühendisliğinde yaygın bir kısıtlı tasarım sorunudur. Problemin amacı, sekiz kısıtlamayı göz önünde bulundurarak çoklu disk debriyaj freninin kütlesini en aza indirmek ve performansını artırmaktır. Bunu yaparken beş karar değişkeni göz önünde bulundurulur: X_1 iç yarıçap, X_2 dış yarıçap, X_3 disk kalınlığı, X_4 aktüatör kuvveti ve X_5 sürtünmeli yüzey sayısı. Çoklu disk debriyaj freninin grafiksel gösterimi Şekil 4.7’te görülmektedir.



Şekil 4.7 Çoklu disk debriyaj freni tasarımının grafiksel gösterimi

Bu problemde amaç fonksiyonu Ek 7’de verilmiştir. Ayrıca çoklu disk debriyaj freni tasarım problemine ait sekiz kısıtlama, gerekli sabit değerler ve değişken değer aralıklarına da buradan ulaşılabilir.

4.3.6 P6: Üç Çubuklu Kafes Tasarımı

Üç çubuklu kafes yapısı köprülerde, binalarda, mekanik ekipmanlarda ve diğer çeşitli alanlarda yaygın olarak kullanılan bir tasarımdır. Bu problemin temel amacı çubuklu yapıların ağırlığını azaltmaktır. Bu problem üç karar değişkenini kullanır: X_1 ilk çubuğun uzunluğu, X_2 ikinci çubuğun uzunluğu ve X_3 üçüncü çubuğun uzunluğu. Bu sorun ele alınırken, sistemin çift yönlü özelliği dikkate alınır ve $X_1=X_3$ olarak kabul edilir. Ayrıca, problem toplam destek yükü, malzeme giderleri ve kesit alanı gibi

Üç çubuklu kafes tasarımının amaç fonksiyonu, amaç fonksiyonunu sınırlandıran probleme ait teknik kısıtlamalar ve değişkenlerin değer aralıkları gibi matematiksel ifadelerle Ek 7’de yer verilmiştir.

MS algoritmalarının performanslarının incelenmesi aşamasında niteliksel ve niceliksel birçok farklı değerlendirme kriteri kullanılmaktadır. Nitel kriterler önerilen modelin davranışlarını iki boyutlu olarak gözlemleyebilmeyi sağlayan grafiksel yöntemlerdir. Arama geçmişi, yakınsama eğrileri, keşif sömürü grafikleri bu yöntemlerin en bilinen örnekleridir. Bu yöntemler modellerin farklı tipteki problemler üzerinde yakınsama davranışları, yerel minimum tuzaklarına takılma durumları ve kararlılıkları gibi gözlemler yapılabilmesini sağlar. Ancak modellerin yakın sonuçlar elde etmesi ya da MS yöntemlerin stokastik yapısından dolayı aynı modelin aynı problem üzerinde birden fazla kez yürütülme gereksinimi sayıca fazla sonuç elde edilmesine neden olur. Bu durum da tüm bu sonuçları yorumlamada nitel gözlemlerin tek başına yeterli olmadığı gösterir, çeşitli sayısal ve istatistiksel değerlendirme

kriterleri kullanılır. Çözüm kalitesinin incelenmesi, sağlamlık, hesaplama süresinin ele alınması ve bunları analiz ederken çeşitli istatistiksel testlerden yararlanmak niceliksel değerlendirme kriterlerine örnek gösterilebilir. Aşağıdaki alt başlıklarda önerilen algoritmanın performansının analiz edilmesi ve ileriki bölümlerde diğer optimizasyon yöntemleri ile karşılaştırılması esnasında kullanılan değerlendirme kriterlerine ait detaylara yer verilmiştir.

4.4.1 Arama Geçmişi

Arama geçmişi iki boyutlu olarak tanımlanabilen fonksiyonlarda popülasyondaki arama ajanlarının farklı iterasyonlardaki konumlarını noktasal olarak gösteren grafiklerdir. Bu grafik problemin yerel ve küresel noktalarını görülebilmesini ve arama ajanlarının küresel çözüme ulaşabildiği, yerel çözümlere takıldığı veya keşif sürecinin devam edip etmediği hakkında bilgi verir. Tez kapsamında arama geçmişi diyagramı popülasyondaki düzenleyicilerin dört farklı iterasyondaki konumlarını gösterir. Beyaz renk 0. iterasyon ve siyah renk ise son iterasyonu temsil eder, ara renkler ise ara iterasyonlardaki düzenleyicilerin konumlarıdır. Mavi ve mor renkler fonksiyonların topografik olarak maksimum minimum noktalarını gösterir. Problemin küresel çözümü kırmızı renkli çarpı işareti ile gösterilmiştir.

4.4.2 Yakınsama Eğrileri

Yakınsama eğrileri x eksenini fonksiyonun zaman maliyeti (yani iteration sayısını) ve y eksenini iterasyonlar boyunca elde edilen en iyi uygunluk değerini gösteren grafiklerdir. Yakınsama eğrileri modellerin iterasyonlar boyunca minimuma ne kadar hızlı ilerlediğini gösterir. Yakınsama eğrileri modellerin farklı çalıştırmalardaki tüm sonuçlarını ayrı grafikte gösterecek şekilde çizilebilir. Bu durumda incelenecek modelin her çalıştırması için aynı kararlılıkta davranış sergileyip sergilemediğini ve küresel minimuma ulaşarak ulaşamadığını izleyebilmeyi sağlar. Bazen de farklı yöntemleri karşılaştırmak ve ortalama yakınsama davranışını gözlemlemek için farklı çalıştırmalardaki ortalamalarını da alarak kullanılabilir. Tez kapsamında önerilen modelin değerlendirilmesi sırasında farklı çalıştırmaların yakınsama eğrileri

incelenmiş ve mevcut yöntemler ile karşılaştırılması yapılırken ortalama yakınsama grafiğinden faydalanılmıştır.

4.4.3 Keşif-Sömürü Grafikleri

Keşif/sömürü grafiği modelin keşif ve sömürü olmak üzere sırasıyla mavi ve kırmızı renkle çizdirilmiş iki çizgisel grafiği içeren gösterimdir. Popülasyondaki arama ajanlarının aramayı sürdürme veya belirli bir alana yoğunlaşma davranışları arasındaki dengeyi nasıl kurduğuna ilişkin bilgi verir. Bu grafik modellerin yakınsama davranışları hakkında çıkarımlar yapılmasına olanak tanır. Ayrıca yerel minimum tuzaklarının fazla olduğu problemlerde keşif süreci devam edeceğinden modellerin hangi problemlerde zorlandığını gösterir. Önerilen algoritmanın test fonksiyonları üzerindeki davranışlarının incelenmesi aşamasında bu grafiklerden faydalanılmıştır.

4.4.4 Çözüm Kalitesi

Çözüm kalitesi, bir optimizasyon algoritmasının bir problemin en iyi çözümüne ulaşip ulaşamadığını gösteren bir ölçüttür. Test fonksiyonları belirli bir arama uzayında minimum ya da maksimum noktaları belirli olan matematiksel fonksiyonlar olduğundan hedef değer belirlidir. Bu nedenle bu tip problemlerde bu değere yakınlık bir değerlendirme kriteri olarak kullanılabilir. Ancak en iyi çözümü belli olmayan gerçek hayat problemleri veya mühendislik sorunlarında algoritmalara ait minimum, maksimum ya da ortalama değerler ele alınabilir. Tez kapsamında sonuçlarımızda tüm problemler için minimum, maksimum ya da ortalama değere yer verilmiştir.

4.4.5 Sağlamlık

Sağlamlık, bir optimizasyon algoritmasının kararlılığının göstergesidir. MS algoritmalar her çalıştırmada arama uzayını rastgele popülasyon ile başlatır. Dolayısıyla bir algoritmanın en iyi sonucu elde etmesi, etkin bir yöntem olduğunu göstermez. Farklı çalıştırmalarda en iyi ya da en iyi sonuca yakın değerleri elde edip edilmediği incelenir. Tez kapsamında algoritmaların Bölüm 4.1’de belirtildiği gibi 25

çalıştırma için sonuçları kaydedilmiş ve bu sonuçların standart sapması algoritmanın sağlamlık kriteri olarak belirlenmiştir.

4.4.6 Hesaplama Süresi

Hesaplama süresi bir algoritmanın bir görevi ne kadar zamanda yerine getirdiğinin bir ölçütüdür. Bir yöntemin uygulanabilirliği için elde ettiği iyi performans kadar hesaplama süresinin hızı da önemlidir. Bu nedenle önerilen yöntemin mevcut yöntemler ile karşılaştırılması aşamasında hesaplama sürelerine de yer verilmiştir.

4.4.7 Sıralama Testi

Sıralama testi, bir çeşit çoklu karşılaştırma yöntemidir. İki veya daha fazla model arasından modellerin en iyi performanslarını kıyaslayarak her model için bir sıralama değeri belirler ve daha iyi yöntemleri öne çıkarır. Tez kapsamında yöntemlerin karşılaştırılması aşamasında sıralama testi kullanılmıştır. Yöntemlerin her probleme ait tüm çalıştırmaları küçükten büyüğe sıralandıktan sonra (rastgeleliği azaltmak amacıyla modellerin en iyi performanstan en kötü performansa doğru sonuçları karşılaştırılmıştır) sıralama testi uygulanır. En iyi performans 1. sıra olarak kabul edilir, ikinci en iyi 2. sıra olarak kabul edilir ve bu şekilde kıyaslama devam eder. Beraberlik durumunda, yöntemlere aynı sıra numarası verilir. Her problemin tekrarları tamamlandıktan sonra her model için ortalama alınır. Böylece işlem tamamlandığında, her algoritma için tüm test fonksiyonlarına ait ortalama sıra hesaplanır.

4.4.8 İşaret Testi

İşaret testi, iki örneklem arasındaki farkı değerlendirmek için kullanılır. Genellikle küçük örneklem veya verilerin normal dağılıma uymadığı durumlarda tercih edilir. İşaret testi uygulanırken önce gözlemler arasındaki fark hesaplanır. Farklar pozitif veya negatif olarak işaretlenir. Sıfır olan farklar göz ardı edilir. Test istatistiği, pozitif ve negatif işaretlerin sayısına dayanarak hesaplanır ve belirli bir anlamlılık seviyesinde (genellikle %5) sonuçlar değerlendirilir.

Tez çalışmasında işaret testi karşılaştırılan yöntemleri ikili olarak karşılaştırmak için kullanılmıştır. Bu sayede önerilen yöntemin her çalıştırmada elde ettiği uygunluk değerlerinin karşılaştırılan yöntemlerle elde edilen sonuçlardan gerçek sonuca ne kadar yakın olduğu incelenmiştir. Bölüm 4.1’de belirtildiği gibi optimizasyon sonuçları 25 tekrarlı olarak hesaplanmıştır. 16 klasik test fonksiyonu (16 fonksiyon x 25 çalıştırma = 400 sonuç), 12 bileşik test fonksiyonu (12 fonksiyon x 25 çalıştırma = 300 sonuç) ve 6 mühendislik uygulaması (6 problem x 25 çalıştırma = 150 sonuç) için toplam 850 minimum değer karşılaştırılmış ve pozitif/negatif olmak üzere işaretler kaydedilmiştir. Bu durum yöntemlerin tüm sonuçlarını karşılaştırarak genel performans hakkında sayısal bir skor elde edebilmeyi sağlar. Çalışmada uygulanan test fonksiyonlarının tümü minimizasyon problemi olduğu için daha küçük değere ulaşan yöntem daha başarılı kabul edilir. Bu nedenle önerilen yöntem ile karşılaştırılan yöntem kıyaslamasında negatif işareti göz önünde bulundurularak, toplam 850 çalıştırmadaki galibiyeti hesaplanmıştır. Bu doğrultuda pozitif işaret modelin minimum sonucunun karşılaştırılan yöntemlerden zayıf olduğunu ve negatif işaret ise minimum sonucunun karşılaştırılan yöntemlerden üstün olduğunu gösterir.

4.4.9 Diğer İstatistiksel Testler

Yöntemlere ait sonuçları istatistiksel olarak karşılaştırmak için çalışmada istatistiksel testlerden faydalanılmıştır. İlk aşamada Shapiro-Wilk Testi kullanılarak dağılımların $p < 0,05$ anlamlılık düzeyinde normal dağılım gösterip göstermediği incelenmiştir ve bulgulara göre de parametrik veya parametrik olmayan yöntemler tercih edilmiştir. Sonraki aşamada sonuçların normal dağılım göstermediği görülmüş ve önerilen yöntem ile diğer 64 algoritma sonuçları arasındaki farkları belirlemek için Kruskal-Wallis Testi uygulanmıştır. Kruskal-Wallis testinin sonuçlarında, $p < 0,05$ koşulunu sağlayan durumlar istatistiksel olarak anlamlı olarak kabul edilmiştir. Son aşamada ise olası ilişkileri araştırmak için Post-hoc analizi yapılmıştır.

BÖLÜM BEŞ

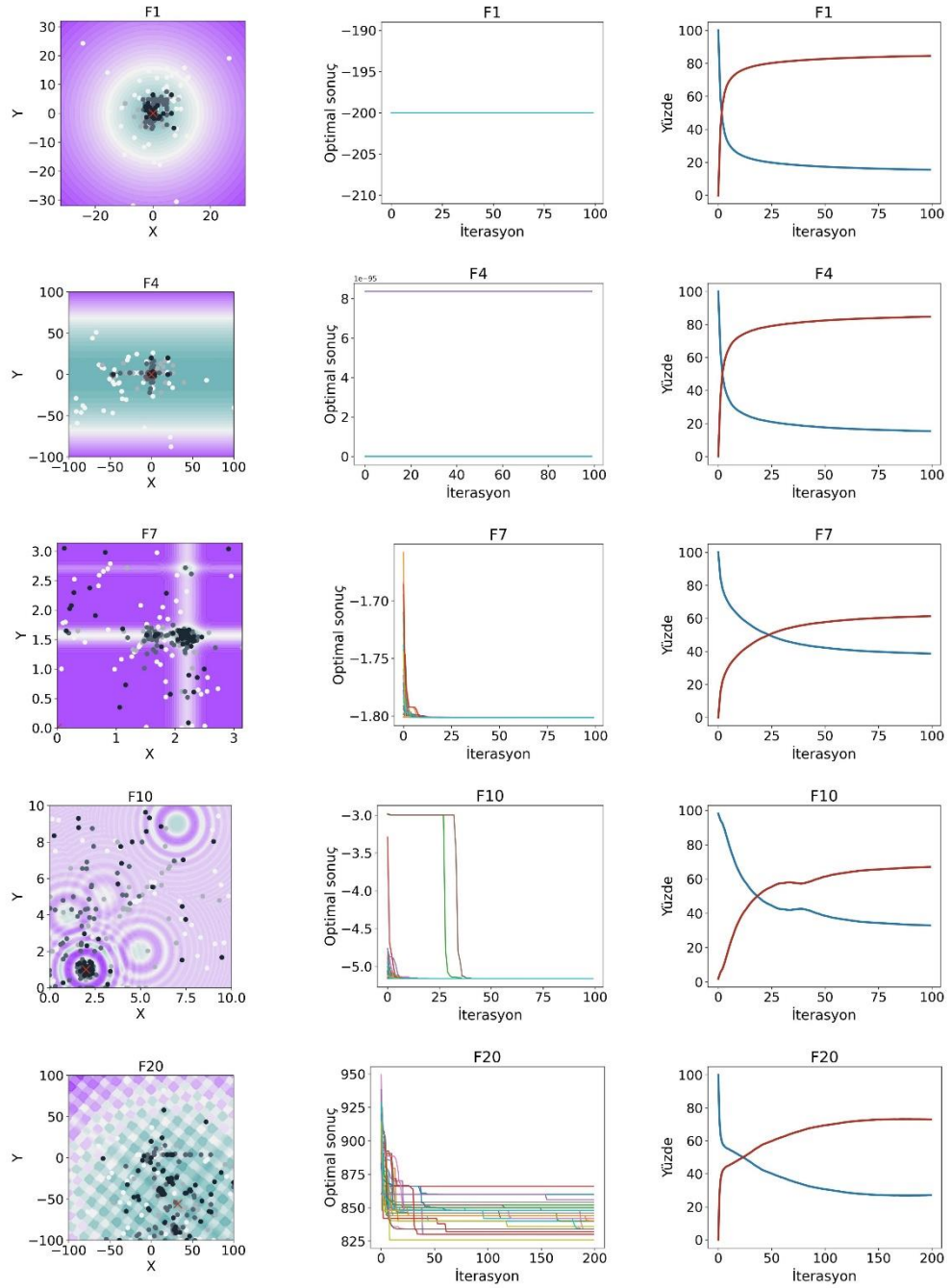
BULGULAR

5.1 Niteliksel Sonuçlar

Değerlendirme sürecinin ilk aşamasında önerilen modelin davranışlarını grafiksel olarak gözlemleyebilmek için iki boyutlu olarak ayarlanabilen test fonksiyonları ele alınmıştır. Bu doğrultuda yöntemin genel davranışlarını yansıtacak şekilde belirlenen beş test fonksiyonu için çizdirilen üç diyagram Şekil 5.1’de gösterilmektedir. Şekil ilk sütununda arama geçmişi grafikleri, ikinci sütununda yakınsama eğrileri ve üçüncü sütununda keşif-sömürü grafiklerini göstermektedir.

F1 fonksiyonu gibi tek yerel minimum noktası olan kıyaslama fonksiyonlarında önerilen model oldukça etkin sonuçlar elde etmiştir. Arama geçmişinde başlangıç popülasyonu dışındaki noktaların fonksiyonun çözümü üzerinde kümелendiği görülmüştür. Yakınsama eğrilerinde tüm çalıştırmalar için ilk iterasyondan çözüme ulaştığı açıktır. Keşif sömürü-grafigi arama ajanlarının başlangıç iterasyonlarında optimal çözüme ulaşıp keşif yüzdesini azaltırken sömürünün hızlı bir şekilde arttırdığını gösterir. Model F2 fonksiyonu için de benzer sonuçlar elde etmiştir.

F4 fonksiyonu gibi bazı problemler tek bir yerel çözümü olmasına rağmen, önerilen modelin kararlı davranış sergileyememesine neden olur. Arama geçmişi farklı çalıştırmaların yalnızca birini yansıtabildiğinden yöntem, çözüme ulaşmış iyi sonuçlar elde etmiş gibi görünebilir. Ancak yakınsama eğrileri incelendiğinde birçok çalıştırma başlangıç iterasyonlarından itibaren çözüme ulaşmış olsa da, bazı çalıştırmalarda modelin herhangi bir iyileştirme göstermeden başlangıç noktasında kaldığı görülmektedir. Bu durumun problemin konveks yapısına rağmen optimal çözümün yakın çevresindeki arama uzayında belirgin bir eğimin olmamasından kaynaklandığı düşünülmektedir. Keşif sömürü eğrilerinde başlangıç iterasyonlarından itibaren dengeli bir davranış izlenmiştir, ancak keşif sömürü yüzdeleri F1 fonksiyonuna göre daha düşüktür. Klasik test fonksiyonlarından F3 ve mühendislik uygulamalarında Hız Düşürücü ve Çoklu Disk Debriyaj Freni Tasarımı problemlerinde benzer eğrilere rastlanmıştır.



Şekil 5.1 Önerilen modele ait grafiksel sonuçlar

Model F7 kıyaslama fonksiyonunda yerel minimumlara rağmen etkin ve kararlı bir performans sergilemiştir. Arama uzayında ajanlar iterasyonlar boyunca çözüme adım adım yaklaşır. Problemlerin çok modlu yapısı nedeniyle ajanlar arama uzayının farklı noktalarını keşfetmeye devam eder. Yakınsama eğrileri karardır, tüm çalıştırmalar iterasyon sayısının ilk çeyreğinde aynı seviyeye ulaşmıştır. Toplam 26 test

fonksiyonunun yarısında ve mühendislik uygulamalarının ikisinde yakınsama davranışı açısından benzer sonuçlar elde edilmiştir. Keşif sömürü eğrileri iterasyon sayısının yarısına ulaşmadan dengeye gelmiştir.

Model F10'un yanısıra F11, F14 ve F16 test fonksiyonlarında arama ajanları optimal çözüm ve çevresinde kümelenirken son iterasyona kadar arama uzayını araştırmaya devam eder. Bu tip problemlerde yerel minimum tuzakları sayıca fazla olabilir. Yakınsama eğrilerinde bazı çalışmalarda modelin yerel minimum noktalara takılrsa da ilerleyen iterasyonlarda bu tuzaklardan kurtulup küresel minimuma yaklaştığı görülmüştür. Keşif sömürü grafiklerinde çalıştırmalar arasındaki bu farklı yakınsama davranışlarından dolayı dalgalanmalar görülmüştür.

F20 gibi çok boyutlu ve yerel minimum tuzaklarının fazla olduğu bazı problemlerde önerilen model zorlanmıştır. Arama uzayı grafiğinde arama ajanının gösterilen iterasyonda çözüme ulaşamadığı ve ajanların farklı bölgelerde küçük kümelenmeler gösterdiği görülmüştür. Yakınsama eğrileri farklı çalıştırmalarda farklı yerel minimumlara takılıp ilerleyen iterasyon sayılarına rağmen çözümlerini geliştirememişlerdir. Model sekiz bileşik fonksiyon ve mühendislik uygulamalarından Kaynaklı Kiriş ve Basınçlı Kap Tasarımı problemlerinde benzer yakınsama davranışları göstermiştir. Keşif sömürü grafiği ortalama davranışları yansıttığı için dengeli bir dağılım göstermektedir.

5.2 Niceliksel Sonuçlar

Niteliksel sonuçlar YKDS algoritmasının başlangıç anından son iterasyona kadar olan süreçte arama uzayındaki hareketliliği, yakınsama hızları ve minimuma ne kadar yaklaşabildikleri, yeni alanları keşfetme eğilimleri ve optimal çözümlerin çevresine ne denli odaklanabildikleri ile ilgili gözlemler sunar. Ancak niteliksel sonuçlar bize yöntemin en iyi çözüme ulaşip ulaşmadığı hakkında net bir bilgi vermez. Önerilen algoritmanın test fonksiyonları ve mühendislik uygulamaları üzerindeki sonuçlarını detaylı olarak analiz etmek için, bu bölümde bulgular niceliksel olarak ele alınmıştır. Algoritmanın otuz kez çalıştırılmasının ardından elde edilen sonuçların minimum, ortamala, maksimum ve standart sapma değerleri hesaplanmış olup klasik test

fonksiyonları, bileşik test fonksiyonları ve mühendislik uygulamaları için sırasıyla Tablo 5.1, 5.2 ve 5.3'te verilmiştir.

Tablo 5.1 YKDS algoritmasının klasik test fonksiyonu sonuçlarının özeti

Fonksiyon	Çözüm	Minimum	Ortalama	Maksimum	Standart sapma
F1	-200,000	-200,000	-200,000	-200,000	0,000
F2	0,000	0,000	0,000	0,000	0,000
F3	0,000	0,000	0,000	0,000	0,000
F4	0,000	0,000	0,000	0,000	0,000
F5	0,000	0,000	0,000	0,000	0,000
F6	0,000	0,000	0,000	0,000	0,000
F7	-1,801	-1,801	-1,801	-1,801	0,000
F8	-2,063	-2,063	-2,063	-2,063	0,000
F9	-2,346	-2,346	-2,346	-2,346	0,000
F10	-5,162	-5,162	-5,162	-5,162	0,000
F11	-176,138	-176,137	-176,128	-176,093	0,011
F12	-1,020	-1,020	-1,020	-1,020	0,000
F13	-6,130	-6,130	-6,129	-6,129	0,000
F14	-176,542	-176,542	-176,533	-176,471	0,014
F15	0,000	0,000	0,000	0,000	0,000
F16	-42,944	-42,944	-42,940	-42,922	0,006

Tablo 5.2 YKDS algoritmasının bileşik test fonksiyonu sonuçlarının özeti

Fonksiyon	Çözüm	Minimum	Ortalama	Maksimum	Standart sapma
F17	300,000	300,069	300,305	300,663	0,174
F18	400,000	400,052	425,292	491,131	33,102
F19	600,000	600,000	600,000	600,000	0,000
F20	800,000	826,000	843,138	866,000	9,389
F21	900,000	900,000	900,820	902,858	0,867
F22	1800,000	1808,639	1895,000	1981,544	28,820
F23	2000,000	2030,895	2050,323	2147,977	22,956
F24	2200,000	2230,316	2261,607	2304,631	47,954
F25	2300,000	2300,429	2558,532	2745,715	187,201
F26	2400,000	2415,370	2601,064	2786,750	61,903
F27	2600,000	2643,826	2804,597	2965,367	53,590
F28	2700,000	2790,406	2852,316	2865,883	18,207

Tablo 5.1'de verilen klasik test fonksiyonlarına ait gerçek çözümler ve sonuçların istatistiksel değerleri karşılaştırıldığında, model on altı fonksiyonun on üçünde tüm çalıştırmalar için en iyi çözüme ulaşmıştır. F14 ve F16 fonksiyonlarında optimal çözüme ulaştığı çalıştırmalar olsa da tümünde başarılı olamamıştır. F11

fonksiyonunda ise optimal çözüme yakın çözümler elde etmiş olsa da hiçbir çalıştırmada gerçek çözüme ulaşamamıştır.

Tablo 5.2’de verilen bileşik test fonksiyonlarına ait gerçek çözümler ve sonuçların istatistiksel değerleri karşılaştırıldığında, model F19 fonksiyonda tüm çalıştırmalar için en iyi çözüme ulaşmıştır. F21 fonksiyonunda bazı çalıştırmalarda küresel çözüme ulaşmıştır ancak tümünde başarılı olamamıştır. F17, F18, F20, F22 ve F28 fonksiyonlarında en iyi sonuca yakın sonuçlar elde etmiş ve standart sapması diğer fonksiyonlara göre düşüktür. F25 fonksiyonundaki performansı kararsız olup yinelemelerin standart sapması en yüksektir.

Tablo 5.3 YKDS algoritmasının mühendislik uygulama sonuçlarının özeti

Fonksiyon	Minimum	Ortalama	Maksimum	Standart sapma
Germe-sıkıştırma yayı	0,013	0,013	0,013	0,000
Basıncılı kap	6095,063	6497,685	7061,823	405,335
Kaynaklı giriş	1,673	1,780	1,985	0,111
Hız düşürücü	3169,177	4158,291	5540,762	765,083
Çoklu disk debriyaj freni	263,896	263,899	263,919	0,014
Üç çubuklu kafes	0,244	0,282	0,331	0,026

Tablo 5.3’de modelin altı mühendislik tasarım problemine ait sonuçların istatistiksel değerleri görülmektedir. Mühendislik problemlerinin kesin çözümleri bilinmediğinden en iyi çözümden söz edilemez. Ancak bu istatistiksel sonuçlardan önerilen algoritmanın dört problem için düşük standart sapma değerlerine sahip olduğu gözlenmiştir. Bu nedenle yöntemin bu problemler için kararlı bir davranış sergilediği söylenebilir. Basıncılı kap ve hız düşürücü tasarım problemleri için model performansının etkinliği zayıftır, bu problemleri çözmek için hassas parametre ayarlamaları gerekebilir.

BÖLÜM ALTI

MEVCUT YÖNTEMLER İLE KARŞILAŞTIRMA

6.1 Çoklu-İstatistiksel Karşılaştırma

MS bir yöntemin potansiyel başarısından söz edebilmek için, belirli optimizasyon zorluklarını etkili bir şekilde ele almasının yanı sıra literatürdeki mevcut yöntemlerle rekabet edebilmesi gerekmektedir. Ayrıca, bu alanda değerlendirilen MS algoritma çeşitliliğini yaygın olarak kullanılan algoritmaların ötesine genişletmek gerekir. Bu nedenle bu bölümde önerilen optimizasyon yönteminin performansını karşılaştırmak amacıyla detaylı bir karşılaştırma yürütülmüştür. Biyolojik canlı tabanlı yöntemler, evrim temelli yöntemler, insan kökenli algoritmalar, fiziksel yasalara dayanan yöntemler, sürü zekasına sahip yöntemler ve diğer yöntemler olmak üzere altı farklı kategoriden 64 MS optimizasyon algoritması ile istatistiksel testler kullanılarak karşılaştırılmıştır. Karşılaştırmada kullanılan yöntemler Bölüm 2.2.1’de ana kategoriler olarak ele alınmış, ilham kaynakları hakkında kısaca bilgi verilmiştir. Metodolojiler Tablo 6.1-6.6’da sunulmuştur. Algoritmalarla ait detaylara, atıfta bulunan yayınlardan ulaşılabilir.

Tablo 6.1 Karşılaştırılan biyolojik canlı tabanlı algoritmaların listesi

No	Orijinal Kısaltma	MS yöntem	Yıl
1	BBO (Simon, 2008)	Biyocoğrafya Tabanlı Optimizasyon	2008
2	BMO (Sulaiman vd., 2020)	Barnakal Çiftleşme Optimizatörü	2018
3	EOA (G.-G. Wang vd., 2018)	Solucan Optimizasyon Algoritması	2018
4	IWO (Mehrabian & Lucas, 2006)	İstilacı Yabancı Ot Optimizasyonu	2006
5	SMA (S. Li vd., 2020)	Balçık Küfü Algoritması	2020
6	SOA (Dhiman & Kumar, 2019)	Martı Optimizasyon Algoritması	2019
7	SOS (Cheng & Prayogo, 2014)	Simbiyotik Organizma Araması	2014
8	WHO (Amali & Dinakaran, 2019)	Antilop Sürüsü Optimizasyonu	2019

Tablo 6.2 Karşılaştırılan evrim temelli yöntemlerin listesi

No	Orijinal Kısaltma	MS yöntem	Yıl
1	CRO (Salcedo-Sanz vd., 2014)	Mercan Resifleri Optimizasyonu	2014
2	DE (Storn & Price, 1997)	Diferansiyel Evrim	1997
3	ES (Rechenberg, 1965)	Evrin Stratejileri	1965
4	FPA (X.-S. Yang, 2012)	Çiçek Tozlaşma Algoritması	2014
5	GA (Holland, 1992)	Genetik Algoritma	1992
6	MA (Moscato, 1989)	Memetik Algoritma	1989

Tablo 6.3 Karşılaştırılan insan kökenli algoritmaların listesi

No	Orijinal Kısaltma	MS yöntem	Yıl
1	BSO (Shi, 2011)	Beyin Fırtınası Optimizasyonu	2011
2	CA (Reynolds, 1994)	Kültür Algoritması	1994
3	CHIO (Al-Betar vd., 2021)	Koronavirüs Sürü Bağışıklığı Optimizasyonu	2021
4	FBIO (Chou & Nguyen, 2020)	Adli Tabanlı Soruşturma Optimizasyonu	2020
5	LCO (Khatiri vd., 2020b)	Yaşam Seçimine Dayalı Optimizasyon	2019
6	SARO (Shabani vd., 2019a)	Arama ve Kurtarma Optimizasyonu	2019
7	SPBO (Das vd., 2020)	Öğrenci Psikolojisi Temelli Optimizasyon	2020
8	SSDO (Tharwat & Gabel, 2020)	Sosyal Kayak Sürücüsü Optimizasyonu	2019
9	TLO (R. V. Rao & Patel, 2012)	Öğretme-Öğrenme Temelli Optimizasyon	2011

Tablo 6.4 Karşılaştırılan fiziksel yasalara dayanan yöntemlerin listesi

No	Orijinal Kısaltma	MS yöntem	Yıl
1	ASO (Zhao vd., 2019)	Atom Arama Optimizasyonu	2019
2	CDO (Shehadeh, 2023)	Çernobil Felaketi Optimizasyonu	2023
3	EFO (Abedinpourshotorban vd., 2015)	Elektromanyetik Alan Optimizasyonu	2015
4	EO (Faramarzi vd., 2020)	Denge Optimizatörü	2020
5	HGSO (Hashim vd., 2019)	Henry Gaz Çözünürlük Optimizasyonu	2019
6	MVO (Mirjalili vd., 2016)	Çoklu Evren Optimizatörü	2016
7	NRO (Wei vd., 2019)	Nükleer Reaksiyon Optimizasyonu	2019
8	SA (Kirkpatrick vd., 1983)	Benzetilmiş Tavlama	1983
9	WDO (Bayraktar vd., 2013)	Rüzgar Güdümlü Optimizasyon	2013

Tablo 6.5 Karşılaştırılan sürü zekâsına sahip yöntemlerin listesi

No	Orijinal Kısaltma	MS yöntem	Yıl
1	ABC (Karaboga & Basturk, 2007)	Yapay Arı Kolonisi	2007
2	ACO (Socha & Dorigo, 2008)	Karınca Kolonisi Optimizasyonu	2008
3	ALO (Mirjalili, 2015b)	Karınca Aslanı Optimizatörü	2015
4	ARO (L. Wang vd., 2022)	Yapay Tavşan Optimizasyonu	2022
5	AVOA (Abdollahzadeh vd., 2021)	Afrika Akbabaları Optimizasyon Algoritması	2021
6	BA (X. S. Yang, 2010a)	Yarasadan İlham Alan Algoritma	2010
7	BES (Alsattar vd., 2020)	Kel Kartal Araması	2020
8	CSA (X.-S. Yang & Deb, 2009)	Guguk Kuşu Arama Algoritması	2009
9	DO (Mirjalili, 2016a)	Yusufçuk Optimizasyonu	2016
10	EHO (G. G. Wang vd., 2015)	Fil Sürüsü Optimizasyonu	2015
11	FFA (X.-S. Yang, 2009)	Ateşböceği Algoritması	2009
12	FOX (Mohammed & Rashid, 2023)	Tilki Optimizatörü	2023
13	GOA (Saremi vd., 2017)	Çekirge Optimizasyon Algoritması	2017
14	GWO (Mirjalili vd., 2014)	Grey Wolf Optimizatörü	2014
15	HHO (Heidari vd., 2019)	Harris Şahinleri Optimizasyonu	2019
16	JA (Venkata Rao, 2016)	Jaya Algoritması	2016
17	MFO (Mirjalili, 2015a)	Güve-Alevi Optimizasyonu	2015
18	NMRA (Salgotra & Singh, 2019)	Çıplak Köstebek Faresi Algoritması	2019
19	PFA (Yapici & Cetinkaya, 2019)	Yol Bulucu Algoritması	2019
20	PSO (Eberhart & Kennedy, 1995)	Parçacık Sürüsü Optimizasyonu	1995

Tablo 6.5 Karşılaştırılan sürü zekâsına sahip yöntemlerin listesi (devamı)

21	SSO (Mirjalili vd., 2017)	Salp Sürü Optimizasyonu	2017
22	STO (Trojovský vd., 2022)	Sibirya Kaplanı Optimizasyonu	2022
23	TDO (Dehghani vd., 2022)	Tazmanya Canavarı Optimizasyonu	2022
24	WOA (Mirjalili & Lewis, 2016)	Balina Optimizasyon Algoritması	2016
25	ZOA (Trojovská vd., 2022)	Zebra Optimizasyon Algoritması	2022

Tablo 6.6 Karşılaştırılan diğer yöntemlerin listesi

No	Kategori	Orijinal Kısaltma	MS yöntem	Yıl
1	Matematik	AOA (Abualigah vd., 2021)	Aritmetik Optimizasyon Algoritması	2021
2		CGO (Talatahari & Azizi, 2021)	Kaos Oyun Optimizasyonu	2021
3		HC (Mitchell vd., 1993)	Tepe Tırmanma	1993
4		SCA (Mirjalili, 2016b)	Sinüs Kosinüs Algoritması	2016
5		TS (Glover, 1990)	Tabu Arama	1990
6	Sistem	AEO (Zhao vd., 2020)	Yapay Ekosistem Tabanlı Optimizasyon	2020
7		GCO (Villaseñor vd., 2018)	Germinal Merkez Optimizasyonu	2018

İstatistiksel testler uygulanırken problemin gerçek değeri ile algoritmalar tarafından üretilen optimum sonuç arasındaki mutlak farkı kullanıldı bu nedenle çözümü bilinen yirmi sekiz test fonksiyonu üzerinden analiz gerçekleştirildi.

Post-hoc testlere göre, önerilen model tarafından elde edilen sonuçlar istatistiksel olarak 42 algoritmanın sonuçlarını geride bırakmıştır (Tablo 6.7). Benzer şekilde, sayısal olarak 9 algoritmadan daha iyi sonuçlar elde etmesine rağmen istatistiksel anlamlı bir fark görülmemiştir (Tablo 6.8).

Tablo 6.7 Optimal sonuçları istatistiksel olarak daha kötü olan modeller

	Yöntem	z	p
1	ACO	11,786	< ,001
2	ALO	3,330	< ,001
3	BA	22,159	< ,001
4	CSA	16,795	< ,001
5	DO	5,876	< ,001
6	EHO	8,728	< ,001
7	FOX	5,493	< ,001
8	JA	12,986	< ,001
9	MFO	13,362	< ,001
10	NMRA	5,384	< ,001
11	PFA	9,826	< ,001
12	PSO	10,593	< ,001

Tablo 6.7 Optimal sonuçları istatistiksel olarak daha kötü olan modeller (devamı)

13	STO	3,415	< ,001
14	BBO	13,045	< ,001
15	BMO	13,873	< ,001
16	EOA	11,449	< ,001
17	IWO	5,543	< ,001
18	SOA	16,533	< ,001
19	AOA	12,926	< ,001
20	HC	9,123	< ,001
21	TS	14,407	< ,001
22	GCO	7,463	< ,001
23	ASO	12,461	< ,001
24	CDO	10,190	< ,001
25	EFO	20,599	< ,001
26	HGSO	18,140	< ,001
27	SA	21,452	< ,001
28	WDO	14,814	< ,001
29	BSO	7,588	< ,001
30	CA	5,522	< ,001
31	CHIO	15,913	< ,001
32	SSDO	11,963	< ,001
33	CRO	10,288	< ,001
34	DE	4,173	< ,001
35	ES	9,700	< ,001
36	FPA	10,756	< ,001
37	GA	13,021	< ,001
38	MA	19,065	< ,001
39	FFA	2,848	0,004
40	HHO	2,229	0,026
41	WOA	2,082	0,037
42	SCA	2,641	0,008

Tablo 6.8 Optimal sonuçları kötü olan modeller

	Yöntem	z	p
1	ABC	1,006	0,314
2	ARO	1,498	0,134
3	BES	0,918	0,358
4	SSO	0,203	0,839
5	CGO	0,73	0,466
6	AEO	0,566	0,571
7	MVO	1,284	0,199
8	LCO	1,350	0,177
9	SPBO	1,752	0,080

Öte yandan, test edilen algoritmaların sadece 13'ü önerilen yöntemden daha iyi sonuçlar üretmiştir. Bunlardan 5'i tarafından elde edilen sonuçlar (Tablo 6.9)

istatistiksel olarak anlamlı bir fark elde etmiştir. 8 yöntem ise istatistiksel bir fark olmamasına karşın niceliksel olarak daha iyi sonuçlara ulaşmıştır (Tablo 6.10).

Tablo 6.9 Optimal sonuçları istatistiksel olarak daha iyi olan modeller

	Yöntem	z	p
1	SOS	-5,678	< ,001
2	FBIO	-4,536	< ,001
3	SARO	-3,751	< ,001
4	TLO	-3,722	< ,001
5	EO	-3,693	< ,001

Tablo 6.10 Optimal sonuçları iyi olan modeller

	Yöntem	z	p
1	AVOA	-0,683	0,494
2	GOA	-0,122	0,903
3	GWO	-1,282	0,200
4	TDO	-0,449	0,654
5	ZOA	-0,118	0,906
6	SMA	-0,759	0,448
7	WHO	-0,286	0,775
8	NRO	-0,849	0,396

6.2 Popüler Algoritmalar ile Karşılaştırma

Bu bölümde sonuçlar çözüm kalitesi ve yakınsama oranı gibi değerlendirme kriterleri kullanılarak detaylı incelemeye tabi tutulmuştur. İzlenebilirlik açısından 64 algorithmadan en popüler olanlarına ait sonuçlara yer verilmiştir. Bu sayede önerilen yöntemin popüler algoritmalar ile rekabet edebilme yeteneği incelenmiştir. Algoritmalar belirlenirken Web of Science'ta "meta-heuristic", "bio-inspired", "optimization" anahtar kelimeleri kullanılmış ve araştırmacılar tarafından en çok atıf almış algoritmalar belirlenmiştir. Seçilen optimizasyon algoritmalarına ait bilgiler Tablo 6.11'de verilmiştir.

Tablo 6.11 Popüler algoritmalara ait detaylar

	Orijinal Kısaltma	Atıf	Yıl	Referans
1	GA	84126	1975	(Holland, 1975)
2	PSO	32956	1995	(Eberhart & Kennedy, 1995)
3	SA	28107	1983	(Kirkpatrick vd., 1983)

Tablo 6.11 Popüler algoritmalarla ait detaylar (devamı)

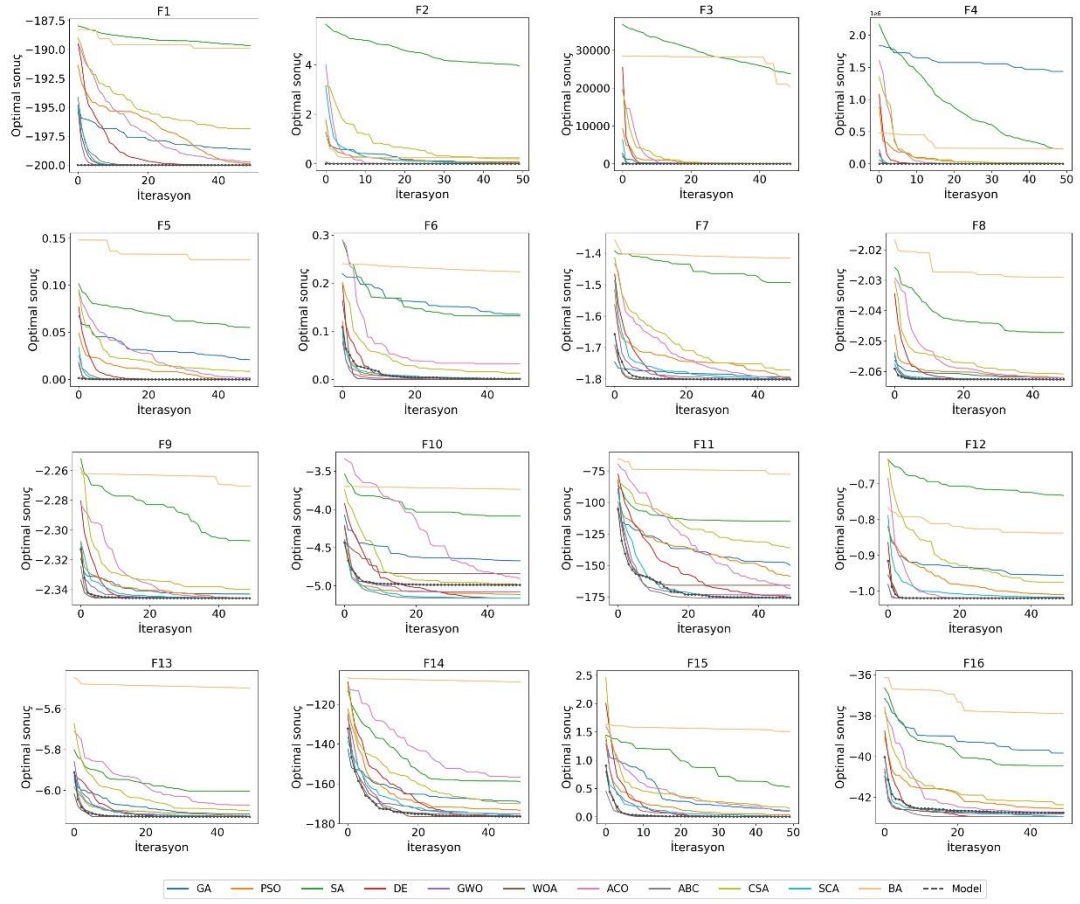
4	DE	18055	1997	(Storn & Price, 1997)
5	GWO	10971	2014	(Mirjalili vd., 2014)
6	WOA	7964	2016	(Mirjalili & Lewis, 2016)
7	ACO	7463	1996	(Dorigo vd., 1996)
8	ABC	5139	2007	(Karaboga & Basturk, 2007)
9	CSA	4786	2009	(X.-S. Yang & Deb, 2009)
10	SCA	3400	2016	(Mirjalili, 2016b)
11	BA	3179	2010	(X. S. Yang, 2010a)

6.2.1 Yakınsama Eğrileri

Bu bölümde önerilen yöntem ve 11 popüler optimizasyon algoritmasının yakınsama davranışları karşılaştırılmıştır. Her bir yakınsama eğrisi algoritmaların 25 çalıştırmasına ait sonuçların ortalaması alınarak çizdirilmiştir. Şekil 6.1 klasik test fonksiyonları, Şekil 6.2 bileşik test fonksiyonları ve Şekil 6.3 mühendislik tasarım problemlerine ait yakınsama grafiklerini gösterir.

Grafiklerden aşağıdaki çıkarımlar elde edilmiştir:

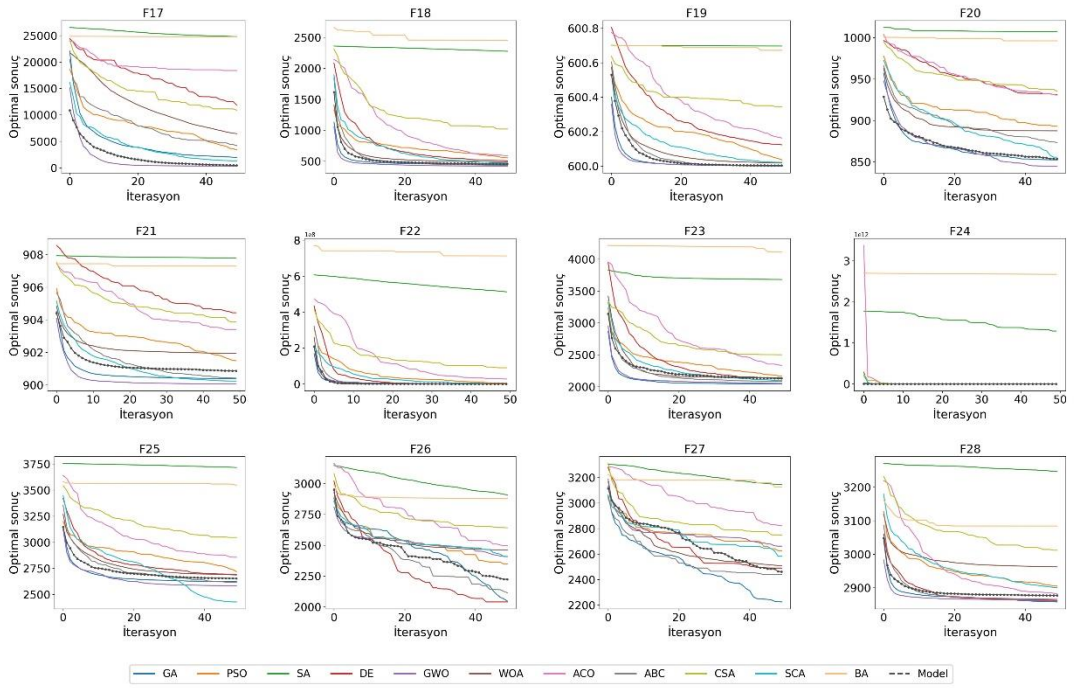
(1) Önerilen modelin klasik test fonksiyonlarında oldukça hızlı bir yakınsama gösterdiği ve F10 fonksiyonu dışında popüler optimizasyon yöntemine kıyasla iyi bir performans sergilediği söylenebilir. Bileşik test fonksiyonlarının büyük bir bölümünde de bu kararlılığı sürdürdüğü söylenebilir ancak 10 boyutlu F21, F23, F25-27 fonksiyonlarında aynı başarı elde edilememiştir. Mühendislik uygulamalarında ise P1 ve P4 problemlerinde başlangıç iterasyonlarından itibaren hızlı bir yakınsama gösterip optimal sonuca yaklaşmıştır. P2, P3 ve P5 problemlerinde aynı başarıyı yakalayamamış olsa da yakınsama performansı popüler algoritmaların büyük bir bölümü ile rekabet edebilir seviyededir. P6 probleminde yakınsama davranışı zayıftır ve optimal sonuca yaklaşamamıştır.



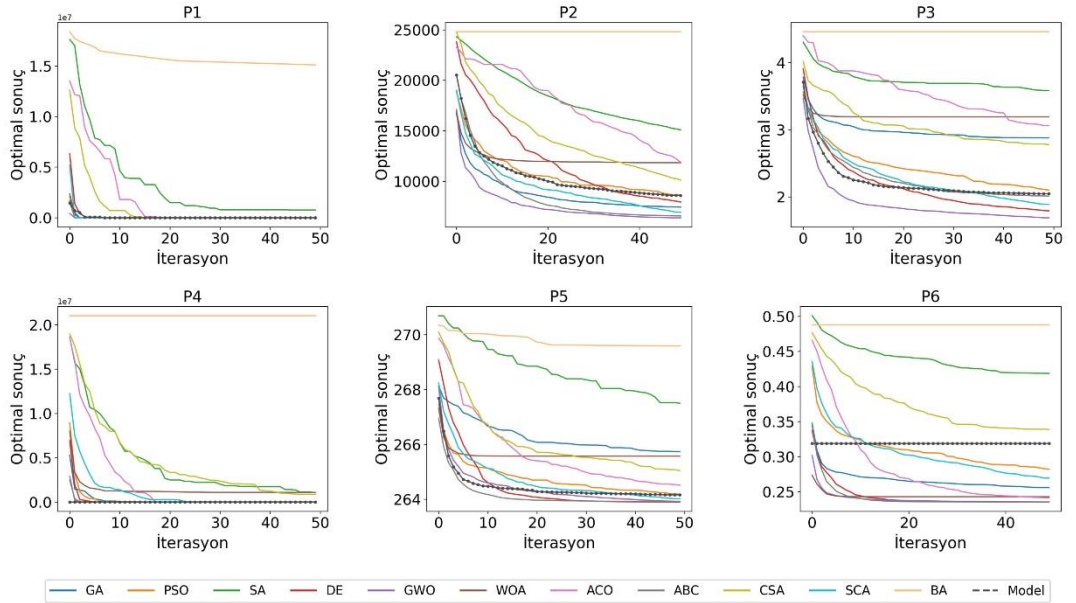
Şekil 6.1 Klasik test fonksiyonlarına ait yakınsama grafikleri

(2) Test fonksiyonları ve mühendislik tasarım problemlerinin neredeyse tümünde yakınsama davranışı en zayıf olan yöntemler SA ve BA'dır. Setler arasında fonksiyonların veya problemlerin türü bakımından bir tutarlılık görülememiş olup karşılaştırılan yöntemler arasında en başarısız algoritmalar olarak yorumlanabilir. ACO ve CSA'nın yakınsama davranışları zayıf olan diğer yöntemler olduğu söylenebilir.

(3) GA algoritması özellikle bileşik test işlevlerinin büyük bir bölümünde rekabetçi bir tavır sergilemiştir. Ancak klasik test fonksiyonları ve mühendislik problemlerinde aynı etkin performansı yakalayamamıştır. GA'nın aksine DE klasik test fonksiyonlarında diğer problemlere göre daha kararlı sonuçlar elde etmiştir.



Şekil 6.2 Bileşik test fonksiyonlarına ait yakınsama grafikleri



Şekil 6.3 Mühendislik tasarım problemlerine ait yakınsama grafikleri

(4) Yakınsama eğrilerinin büyük bölümünde kararlılık gösteren ve önerilen modele rakip olarak gösterilebilecek diğer üç algoritma GWO, WOA ve ABC'dir. WOA klasik test fonksiyonlarında iyi performans gösterirken bileşik test fonksiyonlarında ve mühendislik problemlerinin büyük bölümünde aynı başarıyı gösterememiştir.

6.2.2 Sıralama Testi Sonuçları

Bu bölümde önerilen algoritma popüler algoritmalar ile sıralama testi kullanılarak çoklu karşılaştırmaya tabi tutulmuştur. Klasik test fonksiyonları, bileşik test fonksiyonları ve mühendislik uygulamaları için sıralama sonuçları sırasıyla Tablo 6.12-14' de gösterilmiştir.

Tablo 6.12 Klasik test fonksiyonları için sıralama sonuçları

	GA	PSO	SA	DE	GWO	WOA	ACO	ABC	CSA	SCA	BA	YKDS
F1	9,00	7,00	11,64	6,00	1,00	1,00	8,04	1,12	9,96	2,60	11,36	1,00
F2	9,48	9,64	12,00	1,00	1,00	1,00	1,00	1,00	8,72	8,00	2,96	1,00
F3	9,00	6,52	11,16	1,00	1,00	1,00	8,00	1,00	10,00	1,00	11,84	1,00
F4	11,76	8,00	9,76	6,00	1,00	1,00	7,00	1,00	9,36	1,00	11,12	1,00
F5	9,64	6,76	10,92	1,00	1,00	1,00	8,08	1,00	9,56	1,00	11,80	3,00
F6	10,80	5,92	9,92	1,48	2,60	1,00	9,32	6,56	8,00	5,04	11,96	4,20
F7	8,64	7,84	11,28	1,00	5,00	1,00	7,24	1,00	10,00	6,28	11,72	3,28
F8	7,96	7,40	11,00	3,32	1,08	1,00	8,68	1,00	9,96	5,40	12,00	2,36
F9	9,08	6,36	10,64	1,00	4,36	1,00	7,92	1,00	10,32	6,68	11,96	3,40
F10	8,44	6,80	10,88	4,80	2,72	2,28	9,68	1,76	8,68	5,72	11,92	3,76
F11	9,12	7,92	10,68	5,72	3,04	2,72	6,64	1,52	10,16	4,64	12,00	3,64
F12	9,72	8,08	11,92	1,00	1,12	1,00	5,60	1,00	9,40	7,04	10,84	3,08
F13	8,00	7,00	9,96	3,24	5,24	1,00	10,76	1,00	9,20	5,88	11,96	3,24
F14	7,68	8,00	9,12	4,16	2,96	1,00	10,68	5,84	9,28	4,52	12,00	2,76
F15	9,00	7,00	10,72	4,92	3,96	1,00	8,48	1,40	9,76	6,00	11,96	2,96
F16	9,92	8,32	10,24	2,44	4,56	3,44	6,20	1,00	9,52	4,92	11,92	4,76
Min.	7,68	5,92	9,12	1,00	1,00	1,00	1,00	1,00	8,00	1,00	2,96	1,00
Ort.	9,20	7,41	10,74	3,01	2,60	1,34	7,71	1,76	9,49	4,73	11,21	2,78
Maks.	11,76	9,64	12,00	6,00	5,24	3,44	10,76	6,56	10,32	8,00	12,00	4,76

Tablo 6.12 incelendiğinde, YKDS algoritmasının F1-F4 fonksiyonlarında sıralama değerinin 1 olduğu görülmektedir. Tüm klasik fonksiyonlarındaki ortalama rank değeri ise 2,78'dir. Önerilen model de dahil olmak üzere 6 optimizasyon yönteminin F2 fonksiyonu için ilk sırayı paylaştığı görülmektedir, bu durum algoritmaların tüm tekrarlarında fonksiyonun küresel optimimumuna ulaştığını gösterir ve çözmesi kolay bir problem olarak yorumlanabilir. Klasik test fonksiyonları için WOA, GWO ve ABC algoritmalarının ortalama olarak daha iyi bir sıralamaya sahip olduğu görülmektedir. Ancak GWO ve ABC bazı problemlerde daha kötü sıralamalar elde etmiştir. Dikkat çeken diğer bir nokta GA, SA ve CSA algoritmalarının sıralamalarının problemlerin genelinde geride kaldığıdır. ACO, SCA ve BA ise düşük sıralamalara sahip olmasına

rağmen oldukça kötü sıralamalara da sahiptir, bu nedenle bu üç algoritmanın klasik test fonksiyonlarında kararlılık gösteremediği söylenebilir.

Tablo 6.13 Bileşik test fonksiyonları için sıralama sonuçları

	GA	PSO	SA	DE	GWO	WOA	ACO	ABC	CSA	SCA	BA	YKDS
F17	3,64	5,04	11,36	8,84	2,00	7,00	10,04	5,96	8,16	3,36	11,60	1,00
F18	3,64	7,96	11,16	6,88	2,08	5,28	8,96	3,28	10,00	5,56	11,84	1,36
F19	4,04	6,68	11,68	8,08	1,60	5,08	8,92	3,00	10,04	6,20	11,28	1,36
F20	3,16	6,60	11,96	8,72	1,76	6,00	8,52	5,40	9,52	3,60	11,04	1,72
F21	2,84	6,36	11,72	9,96	1,08	6,64	8,20	3,84	8,84	2,64	11,28	4,60
F22	5,04	8,00	11,00	5,92	1,72	2,52	9,04	4,40	9,96	6,64	12,00	1,76
F23	1,00	7,84	11,04	6,56	2,00	6,24	9,16	3,84	9,84	5,20	11,96	3,32
F24	5,64	7,00	11,00	2,24	2,88	7,28	9,00	1,84	10,04	3,92	11,96	5,20
F25	4,24	7,16	11,92	5,88	3,04	7,00	9,00	3,04	10,00	1,92	11,08	3,72
F26	3,24	5,64	11,76	2,56	6,48	7,92	8,24	3,40	9,72	6,44	11,16	1,44
F27	3,36	5,60	11,44	4,52	7,00	7,32	9,44	3,12	8,28	5,20	11,44	1,28
F28	2,64	7,80	12,00	3,88	1,52	8,48	6,00	3,88	10,08	7,68	10,92	3,12
Min.	1,00	5,04	11,00	2,24	1,08	2,52	6,00	1,84	8,16	1,92	10,92	1,00
Ort.	3,54	6,81	11,50	6,17	2,76	6,40	8,71	3,75	9,54	4,86	11,46	2,49
Maks.	5,64	8,00	12,00	9,96	7,00	8,48	10,04	5,96	10,08	7,68	12,00	5,20

Tablo 6.14 Mühendislik tasarım problemleri için sıralama sonuçları

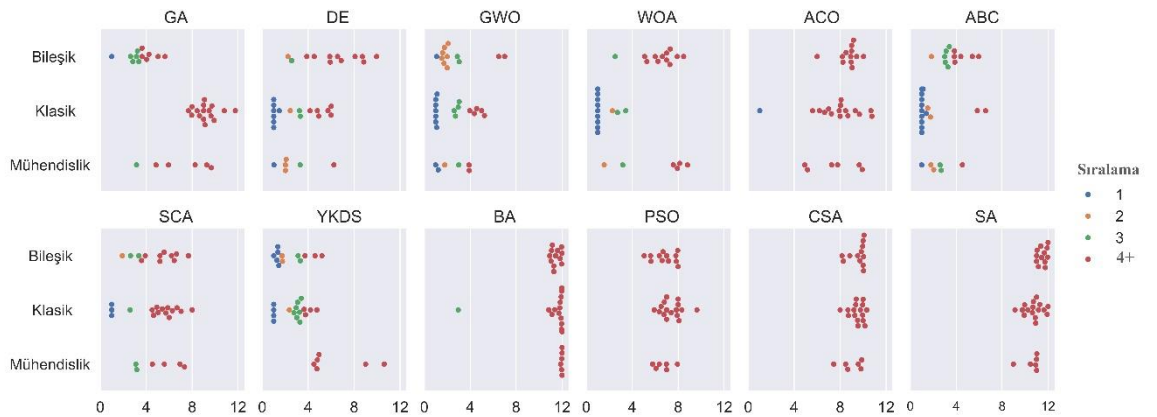
	GA	PSO	SA	DE	GWO	WOA	ACO	ABC	CSA	SCA	BA	YKDS
P1	9,28	6,98	11,00	3,30	1,78	3,18	7,78	2,60	9,58	5,58	11,96	4,94
P2	4,86	7,00	10,46	6,24	1,22	7,94	9,92	1,80	8,66	3,10	12,00	4,80
P3	8,26	5,78	10,92	2,02	1,00	8,82	9,66	4,54	7,44	3,18	11,88	4,50
P4	3,16	6,36	8,98	2,02	3,90	7,58	5,16	1,00	9,88	7,34	12,00	10,62
P5	9,68	6,08	11,00	1,02	3,00	8,14	7,26	2,04	8,48	4,54	12,00	4,76
P6	5,94	7,94	11,02	2,08	3,92	1,56	4,92	2,70	9,82	6,94	11,98	9,00
Min.	3,16	5,78	8,98	1,02	1,00	1,56	4,92	1,00	7,44	3,10	11,88	4,5
Ort.	6,86	6,69	10,56	2,78	2,47	6,20	7,45	2,45	8,98	5,11	11,97	6,44
Maks.	9,68	7,94	11,02	6,24	3,92	8,82	9,92	4,54	9,88	7,34	12,00	10,62

Tablo 6.13, modelin bileşik test fonksiyonlarının altısında sıralamasının en iyi olduğunu göstermektedir. F22 fonksiyonunda da rekabetçi bir yaklaşım sergilediğinden bahsedilebilir ancak F21 ve F24 fonksiyonlarında sıralama olarak geridedir. Bu test grubunda önerilen modelin 2,49'luk sıralamasının yanı sıra GWO algoritmasının 2,76 ile etkili bir performans sergilediği görülmektedir benzer şekilde ABC algoritmasının da istikrarını sürdürdüğü görülmüştür. Ayrıca, ilk test setinde iyi bir sıralamaya sahip olan WOA bu test setinde 6.40'a gerilemiş, klasik test fonksiyonları sıralamasında geride kalan GA ise bu test fonksiyonları için sıralamasını

3,54'e yükseltmiştir. YKDS algoritmasının yanı sıra GA ve ABC'nin problemlerin genelinde düşük sıralamalara sahip olduğu görülmektedir, bundan iyi performans gösterdikleri ve kararlı oldukları sonucu çıkarılabilir. Ancak GWO ve SCA bazı problemlerde minimum değere oldukça yakın çözümler üretirken, bazı problemlerde ise diğer algoritmalarla kıyasla sıralama olarak geride kalmıştır.

Tablo 6.14, modelin mühendislik tasarım problemlerinde klasik test fonksiyonlarındaki etkinliğini sürdüremediğini, genel sıralamasının 6,44 olduğunu göstermiştir. Özellikle P4 ve P6 problemleri için en optimal sonucu bulmada oldukça geride kaldığı görülmüştür. GWO ve ABC bu sette öne çıkan iki yöntemdir. Ayrıca, ilk iki test setinde dikkat çekici bir sıralamaya sahip olmayan DE 2.78 ortalama ile üçüncü en iyi sıralamaya sahip yöntem olmuştur. GWO algoritmasının problemlerin genelini çözerken iyi bir sıralamaya sahipken DE P2, ABC ise P3 kodlu problemler dışında minimum sonuca oldukça yakın çözümler elde etmiştir.

Tablo 12-14'deki modellere ait sıralama testi sonuçları tablolara ek olarak Şekil 6.4'de özetlenmiştir. Her bir grafik için x eksenı sıralama sonucunu gösterirken y eksenı test fonksiyonları ve mühendislik uygulamalarına göre ayrı olarak ele alınmıştır.



Şekil 6.4 En hızlı ve en yavaş yöntemlerin hesaplama süreleri

6.2.3 İşaret Testi Sonuçları

Önceki bölümlerde belirtildiği gibi en etkili optimizasyon algoritması bile problemleri çözmede çok iyi performans göstermesine rağmen her problemi aynı başarı ile çözememektedir. Sıralamalar göz önünde bulundurulduğunda elde edilen bulgular NFL teoremini destekler niteliktedir. Bu nedenle önerilen modelin performansını değerlendirebilmek ve diğer yöntemlere kıyasla üstünlüğünü inceleyebilmek için ek bir teste daha ihtiyaç duyulmuştur. Bu test ikili karşılaştırmaya imkan tanıyan işaret testidir.

İşaret testinde pozitif/negatif işaretlerin nasıl hesaplandığının anlaşılması için Tablo 16.5'te deneysel sonuçların küçük bir bölümüne yer verilmiştir. Tabloda önerilen YKDS algoritması ve rasgele seçilen iki yöntemin F18 kodlu bileşik fonksiyon için ulaştıkları minimum sonuçlar verilmiştir. Seçilen fonksiyon örnek olması için rastgele belirlenmiş olup algoritmaların 5 farklı çalıştırma için elde ettikleri değerler görülmektedir.

Tablo 6.15 F18 fonksiyonu için üç yöntemin ulaştığı beş minimum sonuç

Yöntem	Deneme					Skor	
	1.	2.	3.	4.	5.	(-)	(+)
YKDS	400,75	401,74	402,56	402,85	403,45		
GWO	400,95	401,39	407,79	407,83	408,27	4	1
WOA	405,46	412,07	415,39	416,84	421,20	5	0

F18 kodlu bileşik fonksiyonun minimum değeri 400'dür. YKDS algoritması da dâhil olmak üzere yöntemlerin hiçbirisi gösterilen 5 çalıştırma için minimum sonuca ulaşamamıştır. Buna rağmen minimum sonuca en yakın çözümü elde etme durumu incelenmiştir. YKDS algoritması tabloda görüldüğü gibi GWO algoritmasının ulaştığı sonuçların dördünden daha minimum bir değere ulaşmıştır. Bu yüzden negatif işareti 4 olarak hesaplanmıştır. Yöntemler bu fonksiyon örneğinde 5 kez çalıştırıldığı için pozitif işareti de 1'dir. YKDS algoritmasının elde ettiği sonuçlar benzer şekilde WOA ile karşılaştırıldığında 5 çalıştırma sonuçlarının tümünün WOA'dan daha minimum sonuçlar elde ettiği görülmektedir, bu yüzden negatif işaret 5 ve pozitif işaret 0'dır. Bu örnek küçük bir ön izlemedir, deneysel sürecin tümünde her fonksiyonun 25 kez çalıştırıldığı unutulmamalıdır. 16 klasik test fonksiyonu için 400 sonuç, 12 bileşik test

fonksiyonu için 300 sonuç ve 6 mühendislik uygulaması için 150 sonuç olmak üzere tüm yöntemler için toplam 850 minimum değer kaydedilmiştir. YKDS algoritmasının diğer yöntemler karşısında elde ettiği işaret testi sonuçları Tablo 6.16’da verilmiştir.

Tablo 6.16 İşaret testi sonuçları

	GA	PSO	SA	DE	GWO	WOA	ACO	ABC	CSA	SCA	BA
Klasik	400	400	400	400	400	400	400	400	400	400	400
(+)	5	12	2	244	297	377	36	350	9	123	20
(-)	395	388	398	156	103	23	364	50	391	277	380
Bileşik	300	300	300	300	300	300	300	300	300	300	300
(+)	102	30	1	54	143	51	11	98	5	76	0
(-)	198	270	299	246	157	249	289	202	295	224	300
Mühendislik	150	150	150	150	150	150	150	150	150	150	150
(+)	54	55	16	100	123	51	52	94	19	77	2
(-)	96	95	134	50	27	99	98	56	131	73	148
Toplam	850	850	850	850	850	850	850	850	850	850	850
(+)	161	97	19	398	563	479	99	542	33	276	22
(-)	689	753	831	452	287	371	751	308	817	574	828

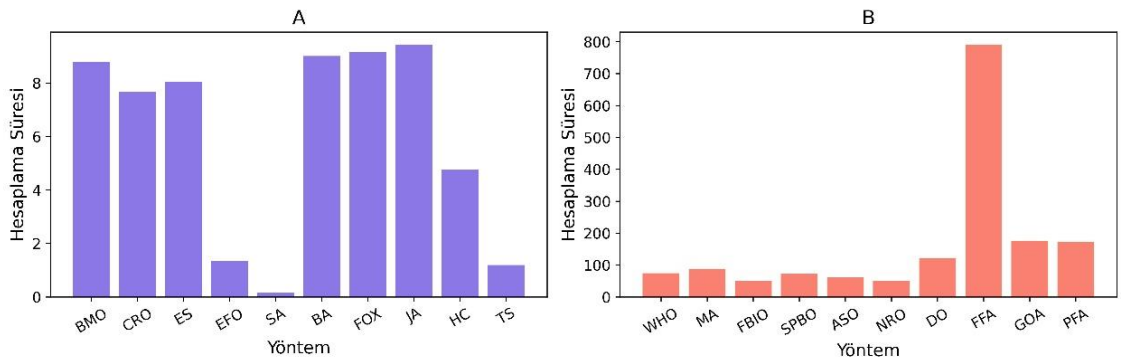
Tabloya göre YKDS algoritması klasik test fonksiyonlarından elde edilen 400 sonuçtan 395’inde GA’ya göre minimum çözüme daha yakın sonuçlara ulaşmıştır. DE, GWO, WOA ve ABC yöntemleri dışındaki diğer yöntemlerde benzer üstünlük söz konusudur. Ancak bu dört yöntem genel olarak önerdiğimiz yöntemden daha optimal çözümler sunmuştur. Bileşik fonksiyonlar incelendiğinde önerilen yöntemin ürettiği optimal çözümlerin küresel minimuma diğer tüm popüler yöntemlerden daha yakın olduğu açıktır. YKDS algoritmasına en yakın çözümler üreten iki yöntem GWO ve GA’dır. Mühendislik uygulamalarında ise DE, GWO ve ABC yöntemlerinin performansı açık ara öndeyken önerilen model SCA yöntemine yakın bir skor elde etmiştir. Ancak tüm test fonksiyonlarından elde edilen toplam skor göz önünde bulundurulduğunda, YKDS algoritmasının literatürde en popüler olan diğer optimize ediciler ile rekabet edebilir düzeyde bir yöntem olduğu görülmüştür.

6.3 Hesaplama Süresi

Hesaplama maliyeti her bir algoritma için tüm problemler çözerken geçirdiği saniye cinsinden çalışma süresini göstermektedir, 25 çalıştırmanın ortalaması alınarak

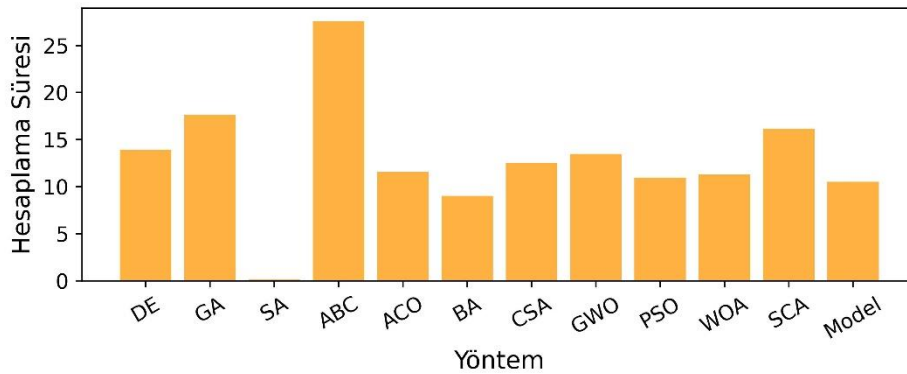
hesaplanmıştır. Şekil 6.5'de en hızlı (A) ve en yavaş (B) on algoritmanın hesaplama süreleri görülmektedir.

SA en hızlı algoritmadır ve tüm problemleri ortalama 0,154 saniyelik kısa bir sürede çözmüştür. SA'nın yanı sıra TS, EFO ve HC yöntemleri de hızlı çalışan diğer algoritmalar. Diğer açıdan FFA tüm problemleri çözmek için en uzun hesaplama süresine sahiptir ve bu süre 791,043 saniyedir. Sonuçlara göre, DO, GOA ve PFA diğer en yavaş algoritmalar.



Şekil 6.5 En hızlı ve en yavaş yöntemlerin hesaplama süreleri

Öte yandan, Şekil 6.6 en popüler optimizasyon yaklaşımlarının hesaplama sürelerini göstermektedir. Bu algoritmaların hesaplama süreleri incelendiğinde, BA'nın 9,014 saniye ile ikinci en hızlı algoritma olduğu görülmüştür. Tez kapsamında önerdiğimiz YKDS algoritmasının 10,515 saniyelik çalışma süresiyle diğer 9 popüler yöntemden daha hızlı işlem yaptığı tespit edilmiştir.



Şekil 6.6 En popüler yöntemlerin hesaplama süreleri

BÖLÜM YEDİ

SONUÇLAR VE TARTIŞMA

Bu tez çalışmasında, doğadan ilham alan yeni bir optimizasyon algoritması önerilmiştir. Önerilen YKDS algoritması, kan dolaşım sisteminin düzenlenmesinde görev alan sinirsel ve hormonal sistemlerin davranışından esinlenilmiştir. Düzenleyicilerin arter basıncı ve kan hacminin kontrolü sırasında denge halini korumak için uyarılma davranışı matematiksel olarak modellenmiştir. Algoritmada, genel performansı etkileyecek teorik parametreler sabit değer olarak belirlenirken farklı optimizasyon problemlerine uyarlanabilirlik açısından ayarlanabilir parametrelere de yer verilmiştir.

Önerilen algoritmanın performansını gözlemlemek için birkaç aşamalı deneysel prosedür izlenmiştir. İlk olarak, YKDS algoritması literatürde sıklıkla kullanılan on altı kıyaslama fonksiyonu ile test edilmiştir. İlk deneyden sonra algoritma on iki bileşik kıyaslama fonksiyonu ile çalıştırıldı, test fonksiyonları 10 boyutlu olarak ayarlandı. Buradaki amaç farklı zorluk seviyelerine sahip fonksiyonlar üzerinde modelin başarısını kıyaslamaktı. Diğer aşamada ise algoritma 6 farklı mühendislik tasarım problemine uygulandı. Bu sayede hem sürekli hem de ayrık unsurlar içeren, karışık değişkenler ve kinematik koşullarla ilgili çeşitli doğrusal olmayan kısıtlamalara sahip gerçek problemler üzerinde algoritmanın etkinliği test edildi.

İncelenebilirlik açısından iki boyutlu olarak ayarlanabilen fonksiyonlar üzerinde algoritma performansı arama geçmişi, yakınsama eğrileri, keşif sömürü diyagramları ile niteliksel olarak analiz edilmiştir. Bunun yanı sıra tüm test fonksiyonları ve mühendislik uygulamalarından küresel minimum ve sağlamlık açısından niceliksel olarak da incelenmiştir. Bu deney ve ilgili tartışmalar aşağıdaki sonuçları desteklemektedir:

(1) Rastgele başlayan arama süreci iterasyonlar boyunca problemin uygunluk değerine göre büyük ölçüde değişikliklere uğrar ve düzenleyiciler arama uzayının gelecek vaat eden bölgelerini etkili bir şekilde keşfeder.

(2) Farklı tipte düzenleyicilerin keşif ve sömürü davranışları arama sürecinin dengeli bir şekilde sürdürülmesini sağlar, hızlı bir yakınsama görülmesine rağmen

algoritma genel itibariyle yerel optimuma hapsolmaz ve küresel optimuma aramayı sürdürür.

(3) YKDS algoritmasının küresel optimuma ulaşması, iki boyutlu klasik test fonksiyonlarını içeren problemlerde oldukça tatmin edicidir. Zorluk seviyesi arttığında bileşik test fonksiyonlarını içeren problemleri çözerken bir dizi test fonksiyonunda önemli ölçüde daha iyi performans gösterme potansiyeline sahiptir. Kısıtlı mühendislik uygulamalarında ise altı problemten dördünün çözümünde etkilidir. Standart sapmalar göz önünde bulundurulduğunda genel itibariyle kararlıdır. Tüm bunlar yöntemin farklı tipteki problemleri ele almada dayanıklı ve sağlam olması olarak yorumlanabilir.

Ayrıca YKDS algoritması bu test fonksiyonları üzerinde aynı deneysel şartlarda 64 optimizasyon yöntemiyle istatistiksel olarak karşılaştırıldı. Performanslar önerilen yöntem ve diğer algoritma sonuçları arasındaki farklar $p < 0,05$ anlamlılık düzeyinde değerlendirildi. Daha sonra deneysel analiz, kapsamlı bir inceleme sağlamak için optimizasyon yöntemlerinden literatürde araştırmacılar tarafından en çok atıf alan 11 algoritmaya odaklandı. Yöntemlerin karşılaştırılması için ikili karşılaştırma testi (işaret testi), çoklu karşılaştırma testi içeren istatistiksel analizlerin yanısıra niteliksel olarak da yakınsama eğrileri kullanıldı. Son olarak tüm algoritmalar gerçek hayat problemlerine uygulanabilirliklerini değerlendirmek için hesaplama süreleri açısından da karşılaştırıldı. Karşılaştırmalı analizin bulguları ve tartışmaları aşağıdaki sonuçları desteklemektedir:

(1) Karşılaştırmalı sonuçlardan YKDS algoritmasının en etkili on beş algoritma arasında yer aldığı görülmektedir. 42 optimizasyon algoritmasına kıyasla istatistiksel açıdan anlamlı bir fark sergilediği açıktır.

(2) Spesifik olarak incelenecek olursa, hem küresel minimizasyon hem de sağlamlık açısından popüler algoritmalara kıyasla oldukça rekabetçidir. Ayrıca, yakınsama davranışı ve hesaplama süresi açısından da birçok yöneme göre daha hızlıdır.

Bununla birlikte, algoritmaların karşılaştırmalı analizi, yaygın olarak bilinen yöntemlerin aksine daha az bilinen çeşitli algoritmaların avantajlarını ve dezavantajlarını ortaya koymaktadır. Bu etkili yöntemlerin gerçek optimizasyon

sorunlarını çözmek için faydalı olduđu sonucuna varılabilir. Endüstri, otomotiv ve inşaat dahil olmak üzere çeşitli mühendislik alanlarındaki zorlukları ele almak için önemli alternatifler olarak hizmet edebilirler.



BÖLÜM SEKİZ

KISIT, ÖNERİ VE GELECEK ÇALIŞMALAR

YKDS algoritmasının test fonksiyonları ve mühendislik tasarım uygulamaları üzerinde kabul edilebilir ve rekabetçi sonuçlar gösterdiği görülmüştür. Ancak önerilen algoritmanın bazı kısıtlamaları ve tez çalışmasının sınırlı kaldığı noktalar bulunmaktadır. Bu kısıtlar, sınırlamaların üstesinden gelebilecek çözüm önerileri ve tez sonrasında yapılması planlanan çalışmalar aşağıda verilmiştir.

(1) Bölüm 1’de bahsedildiği gibi birçok farklı tipte problem bulunmaktadır. Tez çalışmasında benzer türdeki sorunlar üzerinde kıyaslanabilirliği sağlamak ve modelin genel davranışlarını anlamlandırabilmek için tek tip (sürekli karar değişkenlerine sahip) problemler ele alınmıştır.

Algoritmanın ayrık (ikili) ve karma karar değişkenlerine uyarlanabilir versiyonlarının geliştirilmesi farklı tipteki sorunlara uygulanabilirliğini arttıracaktır. Veri madenciliği ve özellikle büyük verilerin kullanıldığı tahmin modellerinde ilgili özelliklerin tespit edilmesi, öznitelik sayısının anlamlı şekilde minimize edilmesi ve tahmin performansının artırılması amacıyla YKDS algoritmasının ikili versiyonunun geliştirilmesi planlanmaktadır.

(2) Yukarıda bahsedilen benzer sebeplerden dolayı tez kapsamında uygulanan problemler tek amaçlı problemlerdir.

Gerçek hayatta karşılaştığımız sorunlar birden fazla durum veya olgunun aynı anda optimizasyonunu gerektirebilir. Bu durumda çalışma gelecekte çok amaçlı problemleri çözmek için yöntemin geliştirilmesine yol göstermektedir.

(3) Diğer açıdan, tezin temel yaklaşımı önerilen yeni MS yöntemin performansını birçok farklı problem üzerinde test etmek ve kapsamlı bir karşılaştırma sağlamaktır. Bu nedenle önerilen yöntem de dahil olmak üzere tüm algoritmaların iterasyon sayısı ve popülasyon boyutu gibi temel parametreleri düşük değerlerde tutulup kısıtlı koşullar altında yöntemlerin performansları analiz edilmiştir. Kullanılan tüm algoritmalar için düşük bir popülasyon boyutu seçmek ve kısıtlı iterasyonlar kullanmak, yöntemlerin

başarısızlığına önemli ölçüde katkıda bulunabilir ve bu da çalışmanın önemli bir sınırlamasını temsil eder.

Bu parametrelerin algoritma performansı üzerindeki etkilerinin araştırılması araştırılmaya değer bir konu olup gelecekte kapsamlı bir araştırmanın önünü açmaktadır.

(4) Ek olarak algoritma çoklu bir test ve uygulama aşamasına tabi tutulduğu için çalışma boyunca algoritmanın ayarlanabilir parametreleri sabit tutulmuş, hassas bir parametre ayarlamasına gidilmemiştir.

Ancak MS algoritmalar kullanılırken performans verimliliğini artırmak için kontrol parametrelerinin tam olarak yapılandırılmasını gerektirir. Bu nedenle YKDS algoritmasının esnekliği ve problemlere uyarlanabilirliğini gözlemleyebilmek için tek bir soruna odaklanıp parametre çalışmada optimizasyonuna da yer verilmesi gelecekte planlanan çalışmalar arasında bulunmaktadır.

(5) Önerilen algoritma genel MS yöntemlerin kısıtlamalarından muzdariptir. Stokastik yapısı gereği kesin sonuç vermez, en iyi olası çözümler üretir bu nedenle her problemde başarılı olacağının bir garantisi yoktur.

Literatürde de desteklendiği üzere bu kısıtlamanın üstesinden gelebilmek için MS algoritmalarla çalışırken tek bir yönetime takılı kalmak yerine karşılaştırmalı çalışmalar yapmak, çalışılan problem üzerinde iyi sonuçlar elde eden yöntemlere odaklanmak performansı ve güvenilirliği artırır.

KAYNAKLAR

- Abdollahzadeh, B., Gharehchopogh, F. S., & Mirjalili, S. (2021). African vultures optimization algorithm: A new nature-inspired metaheuristic algorithm for global optimization problems. *Computers and Industrial Engineering*, 158(May), 107408. <https://doi.org/10.1016/j.cie.2021.107408>
- Abedinpourshotorban, H., Mariyam Shamsuddin, S., Beheshti, Z., & Jawawi, D. N. A. (2015). Electromagnetic field optimization: A physics-inspired metaheuristic optimization algorithm. *Swarm and Evolutionary Computation*, 26, 8–22. <https://doi.org/10.1016/j.swevo.2015.07.002>
- Abualigah, L., Diabat, A., Mirjalili, S., Abd Elaziz, M., & Gandomi, A. H. (2021). The arithmetic optimization algorithm. *Computer Methods in Applied Mechanics and Engineering*, 376, 113609. <https://doi.org/10.1016/j.cma.2020.113609>
- Al-Betar, M. A., Alyasseri, Z. A. A., Awadallah, M. A., & Abu Doush, I. (2021). Coronavirus herd immunity optimizer (CHIO). *Neural Computing and Applications*, 33(10), 5011–5042. <https://doi.org/10.1007/s00521-020-05296-6>
- Alsattar, H. A., Zaidan, A. A., & Zaidan, B. B. (2020). Novel meta-heuristic bald eagle search optimisation algorithm. *Artificial Intelligence Review*, 53(3), 2237–2264. <https://doi.org/10.1007/s10462-019-09732-5>
- Altay, E. V., Altay, O., & Özçevik, Y. (2024). A comparative study of metaheuristic Optimization algorithms for solving real-world engineering design problems. *Computer Modeling in Engineering & Sciences*, 139(1), 1039–1094. <https://doi.org/10.32604/cmescs.2023.029404>
- Amali, D. G. B., & Dinakaran, M. (2019). Wildebeest herd optimization: A new global optimization algorithm inspired by wildebeest herding behaviour. *Journal of Intelligent & Fuzzy Systems*, 37, 8063–8076. <https://doi.org/10.3233/JIFS-190495>
- Atashpaz-Gargari, E., & Lucas, C. (2007). Imperialist competitive algorithm: An algorithm for optimization inspired by imperialistic competition. *2007 IEEE Congress on Evolutionary Computation*, 4661–4667. <https://doi.org/10.1109/CEC.2007.4425083>
- Barricelli, N. A. (1962). Numerical testing of evolution theories - Theoretical introduction and basic tests. *Acta Biotheoretica*, 16(1), 69–98. <https://doi.org/10.1007/BF01556771>
- Bayraktar, Z., Komurcu, M., Bossard, J. A., & Werner, D. H. (2013). The wind driven optimization technique and its application in electromagnetics. *IEEE Transactions on Antennas and Propagation*, 61(5), 2745–2757. <https://doi.org/10.1109/TAP.2013.2238654>
- Bozorg-Haddad, O., Solgi, M., & Loaiciga, H. A. (2017). *Meta-heuristic and evolutionary algorithms for engineering optimization*. Wiley.
- Cheng, M. Y., & Prayogo, D. (2014). Symbiotic Organisms Search: A new

- metaheuristic optimization algorithm. *Computers & Structures*, 139, 98–112. <https://doi.org/10.1016/J.COMPSTRUC.2014.03.007>
- Chou, J. S., & Nguyen, N. M. (2020). FBI inspired meta-optimization. *Applied Soft Computing Journal*, 93, 106339. <https://doi.org/10.1016/j.asoc.2020.106339>
- Cuevas, E., Diaz, P., & Camarena, O. (2021). *Metaheuristic computation: A performance perspective*. Springer.
- Culberson, J. C. (1998). On the futility of blind search: An algorithmic view of “No Free Lunch”. *Evolutionary Computation*, 6(2), 109–127. <https://doi.org/10.1162/evco.1998.6.2.109>
- Das, B., Mukherjee, V., & Das, D. (2020). Student psychology based optimization algorithm: A new population based optimization algorithm for solving optimization problems. *Advances in Engineering Software*, 146(March), 102804. <https://doi.org/10.1016/j.advengsoft.2020.102804>
- Dehghani, M., Hubálovský, Š., & Trojovský, P. (2022). Tasmanian devil optimization: A new bio-inspired optimization algorithm for solving optimization algorithm. *IEEE Access*, 10, 19599–19620. <https://doi.org/10.1177/003754970107600201>
- Dhiman, G., & Kumar, V. (2019). Seagull optimization algorithm: Theory and its applications for large-scale industrial engineering problems. *Knowledge-Based Systems*, 165, 169–196. <https://doi.org/10.1016/j.knosys.2018.11.024>
- Dombi, J., & Jónás, T. (2022). Generalizing the sigmoid function using continuous-valued logic. *Fuzzy Sets and Systems*, 449, 79–99. <https://doi.org/10.1016/j.fss.2022.02.010>
- Dorigo, M. (1992). *Optimization, learning and natural algorithms*. Politecnico di Milano.
- Dorigo, M., Maniezzo, V., & Coloni, A. (1996). Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1), 29–41. <https://doi.org/10.1109/3477.484436>
- Dueck, G. (1993). New optimization heuristics: The great deluge algorithm and the record-to-record travel. *Journal of Computational Physics*, 104(1), 86–92. <https://doi.org/10.1006/jcph.1993.1010>
- Eberhart, R., & Kennedy, J. (1995). New optimizer using particle swarm theory. *Proceedings of the International Symposium on Micro Machine and Human Science*, 39–43. <https://doi.org/10.1109/mhs.1995.494215>
- Erol, O. K., & Eksin, I. (2006). A new optimization method: Big bang–Big crunch. *Advances in Engineering Software*, 37(2), 106–111. <https://doi.org/10.1016/j.advengsoft.2005.04.005>
- Eskandar, H., Sadollah, A., Bahreininejad, A., & Hamdi, M. (2012). Water cycle algorithm - A novel metaheuristic optimization method for solving constrained engineering optimization problems. *Computers and Structures*, 110–111, 151–166.

<https://doi.org/10.1016/j.compstruc.2012.07.010>

- Faramarzi, A., Heidarinejad, M., Stephens, B., & Mirjalili, S. (2020). Equilibrium optimizer: A novel optimization algorithm. *Knowledge-Based Systems*, 191. <https://doi.org/10.1016/j.knosys.2019.105190>
- Feo, T. A., & Resende, M. G. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6, 109–133. https://doi.org/10.1007/978-3-319-68119-1_3
- Fister, I., Yang, X. S., Brest, J., & Fister, D. (2013). A brief review of nature-inspired algorithms for optimization. *Electrotechnical Review*, 80(3), 116–122.
- Fogel, L. J. (1962). Autonomous automata. *Industrial research*, 4, 14–19. [https://scholar.google.com/scholar_lookup?title=Autonomous automata&publication_year=1962&author=L.J. Fogel](https://scholar.google.com/scholar_lookup?title=Autonomous+automata&publication_year=1962&author=L.J.+Fogel)
- Fogel, L. J., Owens, A. J., & Walsh, M. J. (1966). *Artificial intelligence through simulated evolution*. Wiley. <https://books.google.com.tr/books?id=QMLaAAAAMAAJ>
- Fraser, A. (1957a). Simulation of genetic systems by automatic digital computers I. introduction. *Australian Journal of Biological Sciences*, 10, 484–491. <https://doi.org/10.1071/BI9570484>
- Fraser, A. (1957b). Simulation of genetic systems by automatic digital computers II. effects of linkage on rates of advance under selection. *Australian Journal of Biological Sciences*, 10, 492–500. <https://api.semanticscholar.org/CorpusID:54799366>
- Geem, Z. W., Kim, J. H., & Loganathan, G. V. (2001). A new heuristic optimization algorithm: Harmony search. *SIMULATION*, 76(2), 60–68. <https://doi.org/10.1177/003754970107600201>
- Ghosh, S., & Mahanti, A. (1991). Bidirectional heuristic search with limited resources. *Information Processing Letters*, 40, 335–340. [https://doi.org/10.1016/0020-0190\(91\)90203-T](https://doi.org/10.1016/0020-0190(91)90203-T)
- Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5), 533–549. [https://doi.org/10.1016/0305-0548\(86\)90048-1](https://doi.org/10.1016/0305-0548(86)90048-1)
- Glover, F. (1990). Tabu search: A tutorial. *Interfaces*, 20(4), 74–94.
- Goldenberg, M. (2017). The heuristic search research framework. *Knowledge-Based Systems*, 129, 1–3. <https://doi.org/10.1016/j.knosys.2017.05.009>
- Hall, J. H. (2011). Nervous regulation of the circulation, and rapid control of arterial pressure. İçinde *Textbook of Medical Physiology* (s. 204). Elsevier.
- Hall, J. H. . (2011). Role of the Kidneys in Long-Term Control of Arterial Pressure and in Hypertension: The Integrated System for Arterial Pressure Regulation.

- İçinde *Textbook of Medical Physiology* (s. 221). Elsevier.
- Hashim, F. A., Houssein, E. H., Mabrouk, M. S., Al-Atabany, W., & Mirjalili, S. (2019). Henry gas solubility optimization: A novel physics-based algorithm. *Future Generation Computer Systems*, 101, 646–667. <https://doi.org/10.1016/j.future.2019.07.015>
- Hashim, F. A., Hussain, K., Houssein, E. H., Mabrouk, M. S., & Al-Atabany, W. (2021). Archimedes optimization algorithm: a new metaheuristic algorithm for solving optimization problems. *Applied Intelligence*, 51(3), 1531–1551. <https://doi.org/10.1007/s10489-020-01893-z>
- Heidari, A. A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., & Chen, H. (2019). Harris hawks optimization: Algorithm and applications. *Future Generation Computer Systems*, 97, 849–872. <https://doi.org/10.1016/j.future.2019.02.028>
- Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control and artificial intelligence*. University of Michigan Press.
- Holland, J. H. (1992). Genetic algorithms. *Scientific American*, 267(1), 66–72. <https://doi.org/10.1038/scientificamerican0792-66>
- Hooke, R., & Jeeves, T. A. (1961). *Direct search solution of numerical and statistical problems*. 8(2), 212–229. <https://doi.org/10.1145/321062.321069>
- Jamil, M., & Yang, X. S. (2013). A literature survey of benchmark functions for global optimization problems. *International Journal of Mathematical Modelling and Numerical Optimisation*, 4(2), 150–194. <https://doi.org/10.1504/IJMMNO.2013.055204>
- Karaboğa, D. (2005). An idea based on honey bee swarm for numerical optimization. İçinde *Technical Report* (Sayı 9).
- Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, 39(3), 459–471. <https://doi.org/10.1007/s10898-007-9149-x>
- Khatri, A., Gaba, A., Rana, K. P. S., & Kumar, V. (2020a). A novel life choice-based optimizer. *Soft Computing*, 24(12), 9121–9141. <https://doi.org/10.1007/s00500-019-04443-z>
- Khatri, A., Gaba, A., Rana, K. P. S., & Kumar, V. (2020b). A novel life choice-based optimizer. *Soft Computing*, 24(12), 9121–9141. <https://doi.org/10.1007/s00500-019-04443-z>
- Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>
- Koza, J. R. (1989). Hierarchical genetic algorithms operating on populations of computer programs. *Proceedings of the 11th International Joint Conference on*

- Kumar, A., Price, K. V, Mohamed, A. W., Hadi, A. A., & Suganthan, P. N. (2022). Problem definitions and evaluation criteria for the CEC 2022 special session and competition on single objective bound constrained numerical optimization. İçinde *2022 IEEE Congress on Evolutionary Computation (CEC)*. <https://github.com/P-N-Suganthan>
- Kumar, A., Wu, G., Ali, M. Z., Mallipeddi, R., Suganthan, P. N., & Das, S. (2020). A test-suite of non-convex constrained optimization problems from the real-world and some baseline results. *Swarm and Evolutionary Computation*, 56. <https://doi.org/10.1016/j.swevo.2020.100693>
- Laarhoven, V. P. J. M., & Aarts, E. H. L. (1987). Simulated annealing: theory and applications. *Springer*, 33–38.
- Li, G., Zhang, T., Tsai, C.-Y., Yao, L., Lu, Y., & Tang, J. (2024). Review of the metaheuristic algorithms in applications: Visual analysis based on bibliometrics. *Expert Systems with Applications*, 255, 124857. <https://doi.org/10.1016/j.eswa.2024.124857>
- Li, M. D., Zhao, H., Weng, X. W., & Han, T. (2016). A novel nature-inspired algorithm for optimization: Virus colony search. *Advances in Engineering Software*, 92, 65–88. <https://doi.org/10.1016/j.advengsoft.2015.11.004>
- Li, S., Chen, H., Wang, M., Heidari, A. A., & Mirjalili, S. (2020). Slime mould algorithm: A new method for stochastic optimization. *Future Generation Computer Systems*, 111, 300–323. <https://doi.org/10.1016/j.future.2020.03.055>
- Li, Y., Xiong, G., Mirjalili, S., & Wagdy Mohamed, A. (2024). Optimal equivalent circuit models for photovoltaic cells and modules using multi-source guided teaching–learning-based optimization. *Ain Shams Engineering Journal*, 102988. <https://doi.org/10.1016/J.ASEJ.2024.102988>
- Liu, Y., & Passino, K. M. (2002). Biomimicry of social foraging bacteria for distributed optimization: Models, principles, and emergent behaviors. *Journal of Optimization Theory and Applications*, 115(3), 603–628. <https://doi.org/10.1023/A:1021207331209>
- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20, 130–141. [https://doi.org/10.1175/1520-0469\(1963\)020<0130:DNF>2.0.CO;2](https://doi.org/10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2)
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. İçinde F. Glover & G. A. Kochenberger (Ed.), *Handbook of Metaheuristics* (ss. 320–353). Springer US. https://doi.org/10.1007/0-306-48056-5_11
- Mehrabian, A. R., & Lucas, C. (2006). A novel numerical optimization algorithm inspired from weed colonization. *Ecol. Informatics*, 1, 355–366. <https://doi.org/10.1016/j.ecoinf.2006.07.003>

- Mirjalili, S. (2015a). Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Systems*, 89, 228–249. <https://doi.org/10.1016/j.knosys.2015.07.006>
- Mirjalili, S. (2015b). The ant lion optimizer. *Advances in Engineering Software*, 83, 80–98. <https://doi.org/10.1016/j.advengsoft.2015.01.010>
- Mirjalili, S. (2016a). Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Computing and Applications*, 27(4), 1053–1073. <https://doi.org/10.1007/s00521-015-1920-1>
- Mirjalili, S. (2016b). SCA: A sine cosine algorithm for solving optimization problems. *Knowledge-Based Systems*, 96, 120–133. <https://doi.org/10.1016/j.knosys.2015.12.022>
- Mirjalili, S., Gandomi, A. H. ., Mirjalili, S. Z., Saremi, S., Faris, H., & Mirjalili, S. M. (2017). Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems. *Advances in Engineering Software*, 114, 163–191. <https://doi.org/10.1016/j.advengsoft.2017.07.002>
- Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, 95, 51–67. <https://doi.org/10.1016/j.advengsoft.2016.01.008>
- Mirjalili, S., Mirjalili, S. M., & Hatamlou, A. (2016). Multi-verse optimizer: a nature-inspired algorithm for global optimization. *Neural Computing and Applications*, 27(2), 495–513. <https://doi.org/10.1007/s00521-015-1870-7>
- Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, 69, 46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
- Mitchell, M., Holland, J. H., & Forrest, S. (1993). When will a genetic algorithm outperform hill climbing? *NIPS Papers*.
- Mohammed, H., & Rashid, T. (2023). FOX: a FOX-inspired optimization algorithm. *Applied Intelligence*, 53(1), 1030–1050. <https://doi.org/10.1007/s10489-022-03533-0>
- Molina, D., Poyatos, J., Ser, J. Del, García, S., Hussain, A., & Herrera, F. (2020). Comprehensive taxonomies of nature- and bio-inspired optimization: Inspiration versus algorithmic behavior, critical analysis recommendations. *Cognitive Computation*, 12(5), 897–939. <https://doi.org/10.1007/s12559-020-09730-8>
- Moscato, P. (1989). *On evolution, search, optimization, genetic algorithms and martial arts : Towards memetic algorithms*. <https://api.semanticscholar.org/CorpusID:1264156>
- Özcan, N., & Utku, S. (2024). *Doctoral Thesis Document*. Github Repository. <https://github.com/Nrmnzc/Doctoral-Thesis-Document>

- Passos, C. A. S. (2000). A beam search based algorithm to solve flowshop scheduling problems with constraints on shared resources. *IFAC Proceedings Volumes*, 33(17), 675–679. [https://doi.org/10.1016/S1474-6670\(17\)39484-3](https://doi.org/10.1016/S1474-6670(17)39484-3)
- Qais, M. H., Hasanien, H. M., Turkey, R. A., Alghuwainem, S., Tostado-Véliz, M., & Jurado, F. (2022). Circle search algorithm: A geometry-based metaheuristic optimization algorithm. *Mathematics*, 10(10), 1–27. <https://doi.org/10.3390/math10101626>
- Raja, B. D., Patel, V. K., Yildiz, A. R., & Kotecha, P. (2023). Performance of scientific law-inspired optimization algorithms for constrained engineering applications. *Engineering Optimization*, 55(10), 1798–1812. <https://doi.org/10.1080/0305215X.2022.2127698>
- Rajwar, K., Deep, K., & Das, S. (2023). An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges. *Artificial Intelligence Review*, 56(11), 13187–13257. <https://doi.org/10.1007/s10462-023-10470-y>
- Rao, R. V., & Patel, V. (2012). An elitist teaching-learning-based optimization algorithm for solving complex constrained optimization problems. *International Journal of Industrial Engineering Computations*, 3(4), 535–560. <https://doi.org/10.5267/j.ijiec.2012.03.007>
- Rao, R. V., Savsani, V. J., & Vakharia, D. P. (2011). Teaching–learning-based optimization: A novel method for constrained mechanical design optimization problems. *Computer-Aided Design*, 43(3), 303–315. <https://doi.org/10.1016/j.cad.2010.12.015>
- Rashedi, E., Nezamabadi-pour, H., & Saryazdi, S. (2009). GSA: A gravitational search algorithm. *Information Sciences*, 179(13), 2232–2248. <https://doi.org/10.1016/j.ins.2009.03.004>
- Ray, T., & Liew, K. M. (2003). Society and civilization: An optimization algorithm based on the simulation of social behavior. *IEEE Transactions on Evolutionary Computation*, 7(4), 386–396. <https://doi.org/10.1109/TEVC.2003.814902>
- Rechenberg, I. (1965). Cybernetic solution path of an experimental problem. İçinde *Royal Aircraft Establishment*.
- Renard, P., Alcolea, A., & Ginsbourger, D. (2013). Stochastic versus deterministic approaches. İçinde *Environmental Modelling: Finding Simplicity in Complexity* (ss. 133–149). Wiley.
- Reynolds, R. G. (1994). An introduction to cultural algorithms. In *proceedings of the third annual conference on evolutionary programming*, 24, 131–139. <https://doi.org/10.1136/ewjm.172.5.335>
- Salcedo-Sanz, S., Del Ser, J., Landa-Torres, I., Gil-López, S., & Portilla-Figueras, J. A. (2014). The coral reefs optimization algorithm: A novel metaheuristic for efficiently solving optimization problems. *Scientific World Journal*, 2014.

<https://doi.org/10.1155/2014/739768>

- Salgotra, R., & Singh, U. (2019). The naked mole-rat algorithm. *Neural Computing and Applications*, 31(12), 8837–8857. <https://doi.org/10.1007/s00521-019-04464-7>
- Saremi, S., Mirjalili, S., & Lewis, A. (2017). Grasshopper optimisation algorithm: Theory and application. *Advances in Engineering Software*, 105, 30–47. <https://doi.org/10.1016/j.advengsoft.2017.01.004>
- Savsani, P., & Savsani, V. (2016). Passing vehicle search (PVS): A novel metaheuristic algorithm. *Applied Mathematical Modelling*, 40(5–6), 3951–3978. <https://doi.org/10.1016/j.apm.2015.10.040>
- Schwefel, H. P. (1984). Evolution Strategies: A family of non-linear optimization techniques based on imitating some principles of organic evolution. *Annals of Operations Research*, 1(2), 165–167.
- Sen, A. K., Bagchi, A., & Zhang, W. (2004). Average-case analysis of best-first search in two representative directed acyclic graphs. *Artificial Intelligence*, 155(1–2), 183–206. <https://doi.org/10.1016/j.artint.2004.01.001>
- Shabani, A., Asgarian, B., Gharebaghi, S. A., Salido, M. A., & Giret, A. (2019a). A new optimization algorithm based on search and rescue operations. *Mathematical Problems in Engineering*, 2019. <https://doi.org/10.1155/2019/2482543>
- Shabani, A., Asgarian, B., Gharebaghi, S. A., Salido, M. A., & Giret, A. (2019b). A new optimization algorithm based on search and rescue operations. *Mathematical Problems in Engineering*, 2019, 23. <https://doi.org/10.1155/2019/2482543>
- Shehadeh, H. A. (2023). Chernobyl disaster optimizer (CDO): a novel meta-heuristic method for global optimization. *Neural Computing and Applications*, 35(15), 10733–10749. <https://doi.org/10.1007/s00521-023-08261-1>
- Shi, Y. (2011). Brain storm optimization algorithm. In *International conference in swarm intelligence*, 303–309. https://doi.org/10.1007/978-3-642-21515-5_36
- Siddique, N., & Adeli, H. (2015). Nature inspired computing: An overview and some future directions. *Cognitive Computation*, 7(6), 706–714. <https://doi.org/10.1007/s12559-015-9370-8>
- Simon, D. (2008). Biogeography-based optimization. *IEEE Transactions on Evolutionary Computation*, 12(6), 702–713. <https://doi.org/10.1109/TEVC.2008.919004>
- Sivaraj, H., & Gopalakrishnan, G. (2003). Random walk based heuristic algorithms for distributed memory model checking. *Electronic Notes in Theoretical Computer Science*, 89(1). [https://doi.org/10.1016/S1571-0661\(05\)80096-9](https://doi.org/10.1016/S1571-0661(05)80096-9)
- Socha, K., & Dorigo, M. (2008). Ant colony optimization for continuous domains. *European Journal of Operational Research*, 185(3), 1155–1173. <https://doi.org/10.1016/j.ejor.2006.06.046>

- Sörensen, K. (2015). Metaheuristics-the metaphor exposed. *International Transactions in Operational Research*, 22(1), 3–18. <https://doi.org/10.1111/itor.12001>
- Storn, R., & Price, K. (1997). Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4), 341–359. <https://doi.org/10.1023/A:1008202821328>
- Storn, R., & Price, K. (2009). Differential evolution - A simple and efficient heuristic for global optimization over continuous spaces. *Australasian Plant Pathology*, 38(3), 284–287. <https://doi.org/10.1071/AP09004>
- Sulaiman, M. H., Mustaffa, Z., Saari, M. M., & Daniyal, H. (2020). Barnacles Mating Optimizer: A new bio-inspired algorithm for solving engineering optimization problems. *Engineering Applications of Artificial Intelligence*, 87, 103330. <https://doi.org/10.1016/J.ENGAPPAI.2019.103330>
- Talatahari, S., & Azizi, M. (2021). Chaos game optimization: a novel metaheuristic algorithm. *Artificial Intelligence Review*, 54(2), 917–1004. <https://doi.org/10.1007/s10462-020-09867-w>
- Tharwat, A., & Gabel, T. (2020). Parameters optimization of support vector machines for imbalanced data using social ski driver algorithm. *Neural Computing and Applications*, 32(11), 6925–6938. <https://doi.org/10.1007/s00521-019-04159-z>
- Thieu, N. Van. (2024). Opfunu: An Open-source Python Library for Optimization Benchmark Functions. *Journal of Open Research Software*, 12. <https://doi.org/10.5334/JORS.508>
- Trojovská, E., Dehghani, M., & Trojovský, P. (2022). Zebra optimization algorithm: A new bio-inspired optimization algorithm for solving optimization algorithm. *IEEE Access*, 10, 49445–49473. <https://doi.org/10.1109/ACCESS.2022.3172789>
- Trojovský, P., Dehghani, M., & Hanuš, P. (2022). Siberian tiger optimization: A new bio-inspired metaheuristic algorithm for solving engineering optimization problems. *IEEE Access*, 10, 132396–132431. <https://doi.org/10.1109/ACCESS.2022.3229964>
- Van Thieu, N. (2023). *ENOPPY: A Python Library for Engineering Optimization Problems*. <https://doi.org/10.5281/zenodo.7953206>
- Van Thieu, N., & Mirjalili, S. (2023). *MEALPY: An open-source library for latest meta-heuristic algorithms in Python*. Elsevier. <https://doi.org/10.1016/j.sysarc.2023.102871>
- Venkata Rao, R. (2016). Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems. *International Journal of Industrial Engineering Computations*, 7(1), 19–34. <https://doi.org/10.5267/j.ijiec.2015.8.004>
- Villaseñor, C., Arana-Daniel, N., Alanis, A. Y., López-Franco, C., & Hernandez-

- Vargas, E. A. (2018). Germinal center optimization algorithm. *International Journal of Computational Intelligence Systems*, 12(1), 13–27. <https://doi.org/10.2991/ijcis.2018.25905179>
- Wang, G.-G., Deb, S., & Coelho, L. D. S. (2018). Earthworm optimisation algorithm: a bio-inspired metaheuristic algorithm for global optimisation problems. *Int. J. Bio-Inspired Comput.*, 12(1), 1–22. <https://doi.org/10.1504/IJBIC.2018.093328>
- Wang, G. G., Deb, S., & Coelho, L. D. S. (2015). Elephant herding optimization. *Proceedings - 2015 3rd International Symposium on Computational and Business Intelligence, ISCBI 2015*, 1–5. <https://doi.org/10.1109/ISCBI.2015.8>
- Wang, L., Cao, Q., Zhang, Z., Mirjalili, S., & Zhao, W. (2022). Artificial rabbits optimization: A new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. *Engineering Applications of Artificial Intelligence*, 114, 105082. <https://doi.org/10.1016/j.engappai.2022.105082>
- Wei, Z., Huang, C., Wang, X., Han, T., & Li, Y. (2019). Nuclear reaction optimization: A novel and powerful physics-based algorithm for global optimization. *IEEE Access*, 7, 1–9. <https://doi.org/10.1109/ACCESS.2019.2918406>
- Wolpert, D. H. (1996). The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7), 1341–1390. <https://doi.org/10.1162/neco.1996.8.7.1341>
- Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), 67–82. <https://doi.org/10.1109/4235.585893>
- Xie, X.-F., Zhang, W.-J., & Yang, Z.-L. (2002). Social cognitive optimization for nonlinear programming problems. *Proceedings. International Conference on Machine Learning and Cybernetics*, 2, 779–783 c.2. <https://doi.org/10.1109/ICMLC.2002.1174487>
- Yang, X.-S. (2009). Firefly algorithms for multimodal optimization. İçinde O. Watanabe & T. Zeugmann (Ed.), *Stochastic Algorithms: Foundations and Applications* (ss. 169–178). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-04944-6_14
- Yang, X.-S. (2012). Flower pollination algorithm for global optimization. İçinde J. Durand-Lose & N. Jonoska (Ed.), *Unconventional Computation and Natural Computation* (ss. 240–249). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-32894-7_27
- Yang, X.-S., & Deb, S. (2009). Cuckoo Search via Lévy flights. *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, 210–214. <https://doi.org/10.1109/NABIC.2009.5393690>
- Yang, X. S. (2010a). A new metaheuristic bat-inspired algorithm. *Studies in Computational Intelligence*, 284, 65–74. https://doi.org/10.1007/978-3-642-12538-6_6

- Yang, X. S. (2010b). *Engineering optimization: An introduction with metaheuristic applications* (1. baskı). Wiley.
- Yapici, H., & Cetinkaya, N. (2019). A new meta-heuristic optimizer: Pathfinder algorithm. *Applied Soft Computing Journal*, 78, 545–568. <https://doi.org/10.1016/j.asoc.2019.03.012>
- Zhang, Z., Huang, C., Dong, K., & Huang, H. (2019). Birds foraging search: a novel population-based algorithm for global optimization. *Memetic Computing*, 11(3), 221–250. <https://doi.org/10.1007/s12293-019-00286-1>
- Zhao, W., Wang, L., & Zhang, Z. (2019). Atom search optimization and its application to solve a hydrogeologic parameter estimation problem. *Knowledge-Based Systems*, 163, 283–304. <https://doi.org/10.1016/j.knosys.2018.08.030>
- Zhao, W., Wang, L., & Zhang, Z. (2020). Artificial ecosystem-based optimization: a novel nature-inspired meta-heuristic algorithm. *Neural Computing and Applications*, 32(13), 9383–9425. <https://doi.org/10.1007/s00521-019-04452-x>
- Zolghadr-Aslı, B. (2024). *Computational intelligence-based optimization algorithms from theory to practice*. CRC Press.

EKLER

Ek 1: Klasik Test Fonksiyonları Matematiksel İfadeleri

Kod	Uygunluk Fonksiyonu
F1	$f(x) = -200 e^{-0.02\sqrt{X_1^2+X_2^2}}$
F2	$f(x) = (X_1 + 10)^2 + (X_2 + 10)^2 + e^{(-X_1^2-X_2^2)}$
F3	$f(x) = \left(\sum_{i=1}^D X_i^2\right)^2$
F4	$f(x) = X_1^2 + 10^6 \sum_{i=1}^D X_i^2$
F5	$f(x) = 0.26 (X_1^2 + X_2^2) - 0.48X_1X_2$
F6	$f(x) = (1 - X_1)^2 + 100(X_2 - X_1^2)^2$
F7	$f(x) = -X_1 + 2X_2 + (X_1 - X_2)^2 + \sin(X_1 + X_2) + 1$
F8	$f(x) = -0.0001 \left(\left e^{\left 100 - \frac{\sqrt{X_1^2+X_2^2}}{\pi} \right } \sin(X_1) \sin(X_2) \right + 1 \right)^{0.1}$
F9	$f(x) = \left(1 - 8X_1 + 7X_1^2 - \frac{7}{3}X_1^3 + \frac{1}{4}X_1^4 \right) X_2^2 e^{-X_1}$
F10	$f(x) = - \sum_{i=1}^5 \frac{c_i \cos\{\pi[(X_1 - a_i)^2 + (X_2 - b_i)^2]\}}{e^{\frac{(X_1-a_i)^2+(X_2-b_i)^2}{\pi}}}$
F11	$f(x) = \sum_{i=1}^5 i \cos[(i-1)X_1 + i] \sum_{j=1}^5 j \cos[(j+1)X_2 + j] + (X_1 + 1.4251)^2 + (X_2 + 0.80032)^2$
F12	$f(x) = [\sin^2((\cos(X_1) + \cos(X_2))^2 + \cos^2((\sin(X_1) + \sin(X_2))^2 + X_1)^2 + 0.01(X_1 + X_2)$
F13	$f(x) = \prod_{i=1}^n \sqrt{X_i} \sin(X_i)$
F14	$f(x) = \left[\sum_{i=0}^4 (i+1) \cos(iX_1 + i + 1) \right] \left[\sum_{j=0}^4 (j+1) \cos[(j+2)X_2 + j + 1] \right]$
F15	$f(x) = (X_1^2 + X_2 - 11)^2 + (X_1 + X_2^2 - 7)^2$
F16	$f(x) = X_1^2 - 12X_1 + 8\sin\left(\frac{5}{2}\pi X_1\right) + 10\cos\left(\frac{1}{2}\pi X_1\right) + 11 - 0.2 \frac{\sqrt{5}}{e^{\frac{1}{2}(X_2-0.5)^2}}$

Ek 2: Bileşik Test Fonksiyonları Matematiksel İfadeleri

Kod	Uygunluk Fonksiyonu
F17	Kaydırılmış ve Rotasyona Uğratılmış Zakharov Fonksiyonu
F18	Kaydırılmış ve Rotasyona Uğratılmış Rosenbrock's Fonksiyonu
F19	Kaydırılmış ve Rotasyona Uğratılmış Schaffer's Fonksiyonu
F20	Kaydırılmış ve Rotasyona Uğratılmış Sürekli olamayan Rastrigin's Fonksiyonu
F21	Kaydırılmış ve Rotasyona Uğratılmış Levy Fonksiyonu
F22	Hibrit Fonksiyon (n=3)
F23	Hibrit Fonksiyon (n=6)
F24	Hibrit Fonksiyon (n=5)
F25	Kompozit Fonksiyon (n=5)
F26	Kompozit Fonksiyon (n=4)
F27	Kompozit Fonksiyon (n=5)
F28	Kompozit Fonksiyon (n=6)

Ek 3: Germe-Sıkıştırma Yayı Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = X1^2X2(X3 + 2)$
Kısıtlar	$g1(x) = 1 - \frac{(X2^3X3)}{(71785X1^4)}$ $g2(x) = \frac{(4X2^2 - X1X2)}{(12566(X2X1^3 - X1^4))} + \frac{1}{(5108X1^2)} - 1$ $g3(x) = 1 - \frac{140.45X1}{(X2^2X3)}$ $g4(x) = \frac{(X1 + X2)}{1.5} - 1$
Değer Aralıkları	$0.05 \leq X1 \leq 2.0$ $0.25 \leq X2 \leq 1.3$ $2.0 \leq X3 \leq 15.0$

Ek 4. Basınçlı Kap Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = 0.6224z_1X_3X_4 + 1.7781z_2X_3^2 + 3.1661z_1^2X_4 + 19.84z_1^2X_3$
Kısıtlar	$g_1(x) = 0.00954X_3 - z_2$ $g_2(x) = 0.0193X_3 - z_1$ $g_3(x) = X_4 - 240$ $g_4(x) = 1296000 - \pi X_3^2X_4 - \frac{4}{3}\pi X_3^3$
Değer Aralıkları	$1.0 \leq X_1 \leq 99.99$ $1.0 \leq X_2 \leq 99.99$ $10.0 \leq X_3 \leq 200$ $10.0 \leq X_4 \leq 200$
Sabitler	$z_1 = 0.0625X_1$, and $z_2 = 0.0625X_2$

Ek 5: Kaynaklı Kiriş Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = 1.10471X_1^2X_2 + 0.04811X_3X_4 (14 + X_2)$
Kısıtlar	$g_1(x) = t - T_{max}$ $g_2(x) = \sigma - \sigma_{max}$ $g_3(x) = X_1 - X_4$ $g_4(x) = \delta - \delta_{max}$ $g_5(x) = P - P_c$
Değer Aralıkları	$0.125 \leq X_1 \leq 2.0$ $0.1 \leq X_2 \leq 10.0$ $0.1 \leq X_3 \leq 10.0$ $0.1 \leq X_4 \leq 2.0$
Sabitler	$P = 6000, L = 14, \delta_{max} = 0.25, E = 30e^6, G = 12e^6,$ $T_{max} = 13600, \sigma_{max} = 30000$ $P_c = 4.013 E \frac{\sqrt{\frac{X_3^2 X_4^6}{30}}}{L^2} \left(1 - \frac{X_3}{2L} \sqrt{\frac{E}{4G}}\right)$ $\sigma = \frac{6PL}{X_4 X_3^2}$ $\delta = \frac{6PL^3}{EX_3^2 X_4}$ $J = 2 \left[\sqrt{2} X_1 X_2 \left(\frac{X_2^2}{4} + \left(\frac{(X_1 + X_3)}{2} \right)^2 \right) \right]$ $R = \sqrt{\left(\frac{X_2^2}{4} + \left(\frac{(X_1 + X_3)}{2} \right)^2 \right)}$ $M = P \left(L + \frac{X_2}{2} \right)$ $t'' = \frac{MR}{J}$ $t' = \frac{P}{\sqrt{2} X_1 X_2}$ $t = \sqrt{(t')^2 + 2 t' t'' \frac{X_2}{2R} + (t'')^2}$

Ek 6. Hız Düşürücü Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = 0.7854X_1X_2^2(3.3333X_3^2 + 14.9334X_3 - 43.0934) - 1.508X_1(X_6^2 + X_7^2) + 7.477(X_6^3 + X_7^3) + 0.7854(X_4X_6^2 + X_5X_7^3)$
Kısıtlar	$g_1(x) = -X_1X_2^2X_3 + 27$ $g_2(x) = -X_1X_2^2X_3^2 + 397.5$ $g_3(x) = -X_2X_6^4X_3X_4^{-3} + 1.93$ $g_4(x) = \frac{-X_2X_7^4X_3}{X_4^3} + 1.93$ $g_5(x) = 10X_6^{-3} \sqrt{(16.91 \cdot 10^6 + \left(\frac{745X_4}{X_2X_3}\right)^2} - 1100$ $g_6(x) = 10X_7^{-3} \sqrt{(157.5 \cdot 10^6 + \left(\frac{745X_5}{X_2X_3}\right)^2} - 850$ $g_7(x) = X_2X_3 - 40$ $g_8(x) = \frac{-X_1}{X_2} + 5$ $g_9(x) = \frac{X_1}{X_2} - 12$ $g_{10}(x) = 1.5X_6 - X_4 + 1.9$ $g_{11}(x) = 1.1X_7 - X_5 + 1.9$
Değer Aralıkları	$2.6 \leq X_1 \leq 3.6$ $0.7 \leq X_2 \leq 0.8$ $17.0 \leq X_3 \leq 28.0$ $7.3 \leq X_4 \leq 8.3$ $7.3 \leq X_5 \leq 8.3$ $2.9 \leq X_6 \leq 3.9$ $5.0 \leq X_7 \leq 5.5$

Ek 7. Çoklu Disk Debriyaj Freni Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = \pi(X_2^2 - X_1^2) X_3 (X_5 + 1) \rho$
Kısıtlar	$g_1(x) = Prz - p_{max}$ $g_2(x) = Prz * Vsr - p_{max} * V_{srmax}$ $g_3(x) = X_1 + deIR - X_2$ $g_4(x) = (X_5 + 1)(X_3 + \delta) - L_{max}$ $g_5(x) = s * Ms - Mh$ $g_6(x) = -T$ $g_7(x) = Vsr - V_{srmax}$ $g_8(x) = T - T_{max}$
Değer Aralıkları	$60.0 \leq X_1 \leq 80.99$ $90.0 \leq X_2 \leq 110.99$ $1.0 \leq X_3 \leq 3.99$ $0.0 \leq X_4 \leq 1000.99$ $2.0 \leq X_5 \leq 9.99$
Sabitler	Mf = 3, Ms = 40, Iz = 55, n = 250, Tmax = 15, s = 1.5, delta = 0.5, Vsrmax = 10, rho = 0.0000078, pmax = 1, mu = 0.6, Lmax = 30, deIR = 20 $Rsr = \frac{2}{3} \frac{(X_2^3 - X_1^3)}{(X_2^2 X_1^2)}$ $Vsr = \frac{\pi * Rsr * n}{30}$ $A = \pi(X_2^2 - X_1^2)$ $Prz = \frac{X_4}{A}$ $w = \frac{\pi * n}{30}$ $Mh = \frac{2}{3} \frac{\mu * X_4 X_5 (X_2^3 - X_1^3)}{(X_2^2 X_1^2)}$ $T = \frac{Iz * w}{Mh * Mf}$

Ek 8. Üç Çubuklu Kafes Tasarımının Matematiksel İfadeleri

Amaç Fonksiyonu	$f(x) = (2\sqrt{2}X_1 + X_2) L$
Kısıtlar	$g1(x) = \frac{(X_2)}{(\sqrt{2}X_1^2 + 2X_1X_2)} P - \sigma \leq 0$ $g2(x) = \frac{\sqrt{2} X_1 + X_2}{(\sqrt{2}X_1^2 + 2X_1X_2)} P - \sigma \leq 0$ $g3(x) = \frac{1}{(\sqrt{2} X_2 + X_1)} P - \sigma \leq 0$
Değer Aralıkları	$0.0 \leq X_1 \leq 1.0$ $0.0 \leq X_2 \leq 1.0$
Sabitler	$P = 2, L = 100, \sigma = 2$